

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Dương Hoàng Hồng Phúc - Lê Võ Minh
Phương - Nguyễn Mạnh Phương - Lê Minh
Quân - Võ Hoàng Anh Quân - Nguyễn Minh
Thiện

XÂY DỰNG HỆ THỐNG PHÂN TÍCH DỮ
LIỆU ON-CHAIN SOLANA VÀ TRỰC
QUAN HÓA HÀNH VI VÍ BLOCKCHAIN

ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHÍNH QUY

Tp. Hồ Chí Minh, tháng 07/2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Dương Hoàng Hồng Phúc - 22120271

Lê Võ Minh Phương - 22120286

Nguyễn Mạnh Phương - 22120287

Lê Minh Quân - 22120290

Võ Hoàng Anh Quân - 22120293

Nguyễn Minh Thiện - 22120344

**XÂY DỰNG HỆ THỐNG PHÂN TÍCH DỮ
LIỆU ON-CHAIN SOLANA VÀ TRỰC
QUAN HÓA HÀNH VI VÍ BLOCKCHAIN**

THỰC TẬP DỰ ÁN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHÍNH QUY

GIẢNG VIÊN HƯỚNG DẪN

ThS. Phạm Nguyễn Sơn Tùng

ThS. Hồ Tuấn Thanh

Tp. Hồ Chí Minh, tháng 07/2026

Lời cam đoan

Chúng tôi xin cam đoan báo cáo đồ án tốt nghiệp này là công trình nghiên cứu khoa học được thực hiện hoàn toàn độc lập và nghiêm túc của nhóm dưới sự định hướng chuyên môn và hướng dẫn trực tiếp của ThS. Phạm Nguyễn Sơn Tùng và ThS. Hồ Tuấn Thanh. Tất cả các kết quả nghiên cứu, dữ liệu thu thập cùng toàn bộ các số liệu khảo sát, thực nghiệm được trình bày trong cuốn báo cáo đều đảm bảo tính khách quan, trung thực và chưa từng được công bố trong bất kỳ công trình, luận văn hay tạp chí khoa học nào khác trước đây.

Mọi sự giúp đỡ, đóng góp của các cá nhân và tổ chức trong suốt quá trình nhóm triển khai đề tài này đều đã được ghi nhận và bày tỏ lòng biết ơn một cách trân trọng trong phần lời cảm ơn. Bên cạnh đó, toàn bộ các nguồn thông tin, tài liệu tham khảo, các ý tưởng hay kết quả nghiên cứu của các tác giả khác được sử dụng trong báo cáo đều được nhóm tuân thủ nghiêm ngặt quy chế học thuật, tiến hành trích dẫn nguồn gốc một cách rõ ràng, minh bạch. Chúng tôi xin chịu hoàn toàn trách nhiệm trước Hội đồng và Nhà trường về tính trung thực cũng như tính chính danh của toàn bộ nội dung trong công trình nghiên cứu này

Thuyết minh chỉnh sửa đề tài

Nhận xét của giảng viên hướng dẫn

Nhận xét của giảng viên phản biện

Lời cảm ơn

Trước hết, nhóm thực hiện đề tài xin được bày tỏ lòng biết ơn chân thành và sâu sắc đến ThS. Phạm Nguyễn Sơn Tùng và ThS. Hồ Tuấn Thanh, hai giảng viên hướng dẫn đã trực tiếp định hướng, góp ý và đồng hành cùng nhóm trong suốt quá trình thực hiện đồ án tốt nghiệp. Những chỉ dẫn của quý Thầy không chỉ giúp nhóm hoàn thiện hơn về mặt chuyên môn, mà còn giúp nhóm hình thành cách tiếp cận hệ thống hơn trong việc phân tích bài toán, lựa chọn giải pháp kỹ thuật, thiết kế kiến trúc và trình bày kết quả nghiên cứu một cách rõ ràng, khoa học.

Nhóm xin chân thành cảm ơn Ban Giám hiệu Trường Đại học Khoa học Tự nhiên, Đại học Quốc gia Thành phố Hồ Chí Minh, cùng toàn thể quý Thầy, Cô thuộc Khoa Công nghệ Thông tin đã tạo điều kiện học tập, rèn luyện và nghiên cứu thuận lợi trong suốt thời gian qua. Những kiến thức nền tảng về lập trình, cơ sở dữ liệu, công nghệ phần mềm, hệ thống phân tán, an toàn thông tin và phương pháp nghiên cứu được tích lũy trong quá trình học tập là cơ sở quan trọng để nhóm có thể triển khai đề tài “Xây dựng hệ thống phân tích dữ liệu on-chain Solana và trực quan hóa hành vi ví blockchain”.

Nhóm cũng xin ghi nhận vai trò của các nền tảng, thư viện mã nguồn mở và dịch vụ bên thứ ba đã hỗ trợ quá trình nghiên cứu, phát triển và thử nghiệm hệ thống. Các nguồn dữ liệu và dịch vụ như Helius, Moralis, CoinGecko, Birdeye, Google Gemini, Stripe và hạ tầng RPC của Solana đã cung cấp những cơ sở quan trọng để nhóm có thể tiếp cận dữ liệu on-chain, dữ liệu thị trường, chức năng phân tích hỗ trợ bởi trí tuệ nhân tạo

cũng như các luồng thanh toán thử nghiệm trong phạm vi đồ án. Bên cạnh đó, hệ sinh thái mã nguồn mở với các công nghệ như React, Hono, PostgreSQL, Drizzle ORM, ECharts và Carbon Design System cũng đóng vai trò quan trọng trong việc giúp nhóm xây dựng, kiểm thử và hoàn thiện sản phẩm một cách hiệu quả hơn.

Bên cạnh đó, nhóm cũng xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn quan tâm, động viên và tạo điều kiện để các thành viên có thể tập trung hoàn thành đồ án. Sự hỗ trợ về tinh thần, sự cảm thông trong những giai đoạn áp lực và niềm tin từ những người thân xung quanh là nguồn động lực quan trọng giúp nhóm duy trì tinh thần làm việc nghiêm túc trong suốt quá trình thực hiện đề tài.

Cuối cùng, nhóm xin cảm ơn sự nỗ lực, trách nhiệm và tinh thần hợp tác của tất cả các thành viên trong nhóm: Dương Hoàng Hồng Phúc, Lê Võ Minh Phương, Nguyễn Mạnh Phương, Lê Minh Quân, Võ Hoàng Anh Quân và Nguyễn Minh Thiện. Trong quá trình thực hiện đồ án, mỗi thành viên đã cùng nhau trao đổi, phân chia công việc, giải quyết các khó khăn kỹ thuật và hoàn thiện sản phẩm theo từng giai đoạn. Chính sự phối hợp và hỗ trợ lẫn nhau giữa các thành viên là yếu tố quan trọng giúp nhóm hoàn thành đề tài này.

Mặc dù nhóm đã cố gắng thực hiện đề tài một cách nghiêm túc và cẩn trọng, báo cáo khó tránh khỏi những thiếu sót nhất định. Nhóm rất mong nhận được những ý kiến đóng góp quý báu từ quý Thầy, Cô và Hội đồng để có thể tiếp tục hoàn thiện sản phẩm cũng như rút ra thêm nhiều kinh nghiệm cho quá trình học tập, nghiên cứu và phát triển phần mềm trong tương lai.

Mục lục

Thuyết minh chỉnh sửa đề tài	ii
Nhận xét hướng dẫn	iii
Nhận xét phản biện	iv
Lời cảm ơn	v
Đề cương chi tiết	vii
Mục lục	vii
Danh sách hình	xi
Danh sách bảng	xiv
Bảng thuật ngữ	xv
Tóm tắt	xxi
1 Giới thiệu	1
1.1 Đặt vấn đề	1
1.1.1 Ngữ cảnh và vấn đề thực tiễn	1
1.1.2 Tính cấp thiết và độ khó của bài toán	2
1.1.3 Hạn chế của các giải pháp hiện tại	3
1.2 Mục tiêu đề tài	4

1.3	Đối tượng và phạm vi ứng dụng	5
1.3.1	Đối tượng người dùng mục tiêu	5
1.3.2	Phạm vi ứng dụng	6
1.4	Giải pháp và cách tiếp cận	7
1.5	Đóng góp của đề tài	7
1.6	Bố cục của báo cáo	9
2	Hệ thống và nguồn dữ liệu liên quan	10
2.1	Các hệ thống liên quan	10
2.1.1	Arkham Intelligence	11
2.1.2	Birdeye và CoinGecko	12
2.1.3	Dune Analytics và Nansen	12
2.2	Đối chiếu và bài toán đặt ra cho Yoca	13
2.3	Các nhà cung cấp dữ liệu được sử dụng trong Yoca	15
2.3.1	Dữ liệu thị trường, token và pool	15
2.3.2	Dữ liệu ví và giao dịch	16
2.4	Tổng kết chương	18
3	Phân tích và đặc tả yêu cầu	19
3.1	Người dùng và hành trình phân tích	19
3.2	Yêu cầu chức năng	20
3.2.1	Khám phá thị trường, token và pool	21
3.2.2	Phân tích ví và giao dịch	21
3.2.3	Phát hiện dấu hiệu wash trading	21
3.2.4	Tài khoản và không gian cá nhân	22
3.2.5	Cảnh báo và theo dõi	22
3.2.6	AI và giới hạn sử dụng	23
3.3	Yêu cầu chất lượng	23
3.4	Sơ đồ Use Case	24
3.5	Kế hoạch kiểm thử và tiêu chí chấp nhận	27
3.6	Tổng kết chương	29

4	Kiến trúc và thiết kế hệ thống	30
4.1	Kiến trúc tổng thể	30
4.2	Hợp đồng dữ liệu từ giao diện đến nhà cung cấp	32
4.3	Luồng truy xuất, chuẩn hóa và lưu đệm	34
4.4	Thiết kế cơ sở dữ liệu	35
4.4.1	Mục tiêu và đặc điểm dữ liệu	35
4.4.2	Phân nhóm dữ liệu theo miền nghiệp vụ	36
4.4.3	Sơ đồ thực thể mối quan hệ	38
4.4.4	Các bảng trọng tâm	44
4.4.5	Cache và quản lý độ mới	47
4.4.6	Toàn vẹn và bảo mật dữ liệu	48
4.4.7	Tổng kết thiết kế cơ sở dữ liệu	48
4.5	Lựa chọn công nghệ và giới hạn của thiết kế	49
4.6	Ranh giới tin cậy và các điểm cần bảo vệ	50
4.7	Kết chương	51
5	Triển khai thực nghiệm và Đánh giá	53
5.1	Môi trường và công cụ triển khai	53
5.1.1	Tổ chức môi trường triển khai	53
5.1.2	Tích hợp nguồn dữ liệu	54
5.2	Quy trình tích hợp và triển khai hệ thống	55
5.2.1	Quản lý mã nguồn và môi trường	55
5.2.2	Tích hợp liên tục bằng GitHub Actions	55
5.2.3	Triển khai và kiểm tra sau triển khai	56
5.3	Kết quả triển khai các chức năng cốt lõi	59
5.3.1	Xác thực tài khoản	59
5.3.2	Trang tổng quan thị trường	59
5.3.3	Tìm kiếm	61
5.3.4	Trang tổng quan token	62
5.3.5	Trang chi tiết pool	65
5.3.6	Trang phân tích ví	66

5.3.7	Trang so sánh nhiều ví	68
5.3.8	Trang cảnh báo và theo dõi ví	69
5.3.9	Trang hồ sơ cá nhân	71
5.3.10	Mô-đun phát hiện dấu hiệu wash trading	72
5.3.11	Quá trình đồng nhất hóa giao diện	73
5.3.12	Localization cho người dùng Việt Nam	74
5.4	Giải quyết các thách thức kỹ thuật	76
5.4.1	Xử lý giới hạn truy vấn và điều phối khóa truy cập	76
5.4.2	Tái sử dụng bộ đệm theo khoảng thời gian	77
5.4.3	Từ tự tổng hợp giao dịch đến dịch vụ phân tích ví .	78
5.4.4	Migration Mobula-Zerion và phân tách ý nghĩa PnL	79
5.4.5	Giá trị kỹ thuật sau khi thu hẹp phạm vi tự tính toán	80
5.4.6	Chuẩn hóa dữ liệu từ nhiều nguồn	80
5.4.7	Duy trì tính giải thích của kết quả AI	81
5.4.8	Giảm thiểu prompt injection và kết quả AI sai cấu trúc	82
5.5	Kết quả kiểm thử và đánh giá hệ thống	82
5.5.1	Phạm vi kiểm thử hiện có	83
5.5.2	Kết quả chạy test và vấn đề còn tồn tại	84
5.5.3	Đánh giá và phạm vi kiểm thử còn thiếu	84
5.6	Tổng kết chương	85
6	Kết luận và hướng phát triển	86
6.1	Kết quả và đóng góp chính	86
6.2	Giới hạn còn tồn tại	88
6.3	Hướng phát triển	89
	Tài liệu tham khảo	91
A	Phân tích chi phí vận hành và mô hình kinh doanh	94
A.1	Bản đồ chi phí dữ liệu hiện tại	94
A.2	Điều chỉnh TTL và đòn bẩy phía nhà cung cấp	95

A.3	Kịch bản chi phí theo quy mô người dùng	96
A.4	Giả thuyết chuyển sang mô hình AI tự vận hành	97
A.5	Mô hình giá và dòng tiền giả định	98
A.5.1	Bảng giá theo gói, chỉ ghi phần khác biệt	98
A.5.2	Dòng tiền giả định theo quy mô	99
A.5.3	Nguồn vốn	99
A.6	Phạm vi và giới hạn của mô hình	99

Danh sách hình

2.1	Logo nền tảng Arkham Intelligence	11
2.2	Logo nền tảng Birdeye và CoinGecko	12
2.3	Logo nền tảng Dune Analytics và Nansen	12
3.1	Sơ đồ Use Case tổng quát của hệ thống Yoca	25
3.2	Sơ đồ Use Case của Khách	26
3.3	Sơ đồ Use Case của Người dùng đã đăng ký	27
4.1	Kiến trúc tổng thể của hệ thống Yoca: backend là điểm trung chuyển duy nhất giữa client và sáu nhóm phụ thuộc ngoài, cùng DevOps là quan hệ triển khai chứ không phải một lệnh gọi runtime	31
4.2	Các miền dữ liệu chính và quan hệ khái niệm	37
4.3	ERD nhóm tài khoản và thanh toán	38
4.4	ERD nhóm cảnh báo và lịch sử cảnh báo	39
4.5	ERD nhóm token và dữ liệu thị trường	39
4.6	ERD nhóm nội dung và AI cache theo token	40
4.7	ERD nhóm tổng quan và phân tích ví	41
4.8	ERD nhóm portfolio và lịch sử số dư	42
4.9	ERD nhóm lịch sử transfer và swap	43
4.10	ERD nhóm chi tiết giao dịch Helius	44
5.1	Kiến trúc tích hợp và triển khai của Yoca	56

5.2	Một lần chạy GitHub Actions thành công trên <code>main</code> : bẫy job typecheck, lint, test server/client, build server/client và deploy đều đạt, tổng thời gian 3m2s	57
5.3	Render Static Site <code>yoca-frontend</code> ở trạng thái Live, phục vụ tại tên miền <code>yoca.id.vn</code>	58
5.4	Render Web Service <code>yoca-backend</code> ở trạng thái Live tại <code>yoca-backend.onrender.com</code> , log cho thấy các request <code>GET /api</code> thật trả về 200	58
5.5	Market Overview: nhóm Trending, bộ lọc theo khung thời gian và bảng token/pool có thể điều hướng tiếp	61
5.6	Token Overview: identity, thống kê giá/thị trường và biểu đồ giá của một token	63
5.7	Tab Holders: danh sách địa chỉ nắm giữ lớn và biểu đồ phân bố nguồn cung theo bốn lớp	64
5.8	Trợ lý AI trả lời câu hỏi về token kèm trạng thái giới hạn lượt hỏi còn lại trong ngày (gói Free)	64
5.9	Token Pool: pool selector, market metrics và biểu đồ nền của cặp giao dịch đang chọn	66
5.10	Wallet Overview: win rate, tổng tài sản, PnL, khối lượng giao dịch, biểu đồ số dư và phân bổ tài sản của một ví demo	67
5.11	Phân tích chi tiết theo từng vị thế token: số dư, lãi/lỗ đã và chưa thực hiện, khối lượng và giá mua/bán trung bình	68
5.12	Tạo một alert rule thật: theo dõi swap của một ví trong khoảng giá trị 10–100 USD	70
5.13	Email cảnh báo nhận được thật sau khi rule ở Hình 5.12 kích hoạt: sự kiện SWAP trên ví theo dõi, kèm nguồn, số lượng và transaction signature	71
5.14	Cùng trang Wallet Overview ở hai locale: chuỗi giao diện, định dạng ngày/thời lượng và giá trị tài chính được quy đổi và làm tròn theo VND	76
5.15	Quá trình điều chỉnh nguồn dữ liệu cho mô-đun Wallet	79

Danh sách bảng

2.1	Đối chiếu định hướng của các nền tảng phân tích dữ liệu blockchain	14
2.2	Vai trò của các nhà cung cấp dữ liệu trong phiên bản Yoca hiện tại	17
3.1	Kịch bản và tiêu chí chấp nhận các luồng chính	28
4.1	Ví dụ hợp đồng request/response cho endpoint phân tích ví	33
4.2	Contract đại diện của API Alert History	34
4.3	Các bảng dữ liệu trọng tâm của Yoca	44
5.1	Kết quả chạy test trên source hiện tại	84
A.1	Giới hạn miễn phí và gói trả phí gần nhất theo provider .	96
A.2	Chi phí dữ liệu và AI ước tính theo mức quy mô người dùng	97
A.3	Giá Gemini API theo token (USD/1 triệu token) [11] . . .	97
A.4	Bảng giá theo gói, chỉ ghi phần khác biệt so với gói liền trước	98
A.5	Doanh thu và lợi nhuận gộp giả định theo mức quy mô (chỉ tính chi phí dữ liệu/AI)	99

Bảng thuật ngữ

Báo cáo sử dụng một số thuật ngữ chuyên ngành thuộc lĩnh vực blockchain, tài chính số và kiến trúc phần mềm. Phần lớn các thuật ngữ này đã được dịch hoặc diễn giải bằng tiếng Việt ngay trong nội dung từng chương; bảng dưới đây tổng hợp lại những thuật ngữ thông dụng nhất để tiện tra cứu trong suốt quá trình đọc báo cáo.

Thuật ngữ	Giải thích
Adapter	Lớp trung gian trong backend, chuyển dữ liệu từ định dạng riêng của một nhà cung cấp bên ngoài sang định dạng nội bộ thống nhất của hệ thống.
AI Assistant (Trợ lý AI)	Thành phần sử dụng mô hình ngôn ngữ lớn để tóm tắt và diễn giải dữ liệu ví, token hoặc tín hiệu wash trading dựa trên ngữ cảnh đã được hệ thống chuẩn hóa.
API (Application Programming Interface)	Giao diện lập trình ứng dụng, tập hợp quy tắc để các thành phần phần mềm giao tiếp và trao đổi dữ liệu với nhau.
ATH / ATL (All-Time High / All-Time Low)	Mức giá cao nhất và thấp nhất mà một token từng đạt được tính đến thời điểm hiện tại.
Backend	Phần máy chủ của ứng dụng, đảm nhiệm xác thực, xử lý nghiệp vụ, truy cập cơ sở dữ liệu và tích hợp với các nhà cung cấp dữ liệu bên ngoài.
Blockchain	Chuỗi khối, một dạng sổ cái phân tán lưu trữ dữ liệu giao dịch một cách công khai, có thể kiểm chứng và khó bị thay đổi.

(còn tiếp ở trang sau)

(Bảng thuật ngữ - tiếp theo)

Thuật ngữ	Giải thích
Cache / Caching (Lưu trữ đệm)	Cơ chế lưu lại dữ liệu đã truy xuất để tái sử dụng cho các lần yêu cầu sau, giúp giảm số lượt gọi đến nhà cung cấp bên ngoài và cải thiện tốc độ phản hồi.
DeFi (Decentralized Finance)	Tài chính phi tập trung, các dịch vụ tài chính như cho vay, giao dịch hay tạo thanh khoản được vận hành trực tiếp trên blockchain, không qua trung gian truyền thống.
DEX (Decentralized Exchange)	Sàn giao dịch phi tập trung, nơi người dùng hoán đổi token trực tiếp thông qua hợp đồng thông minh thay vì thông qua một đơn vị trung gian.
Drawdown	Mức suy giảm giá trị tài sản của một ví so với đỉnh giá trị gần nhất, dùng để đánh giá mức độ rủi ro.
DTO (Data Transfer Object)	Đối tượng dữ liệu được định nghĩa để truyền dữ liệu giữa các lớp của hệ thống theo một cấu trúc thống nhất.
Endpoint	Một địa chỉ cụ thể trong API mà frontend hoặc dịch vụ khác gửi yêu cầu đến để lấy hoặc gửi dữ liệu.
ERD (Entity Relationship Diagram)	Sơ đồ thực thể mối quan hệ, mô tả các nhóm bảng chính và mối liên hệ giữa chúng trong cơ sở dữ liệu.
FDV (Fully Diluted Valuation)	Định giá pha loãng hoàn toàn, giá trị vốn hóa ước tính của một token nếu toàn bộ nguồn cung tối đa được lưu hành.
Frontend	Phần giao diện người dùng của ứng dụng, đảm nhiệm hiển thị dữ liệu, tiếp nhận thao tác và trực quan hóa thông tin.
Full-stack	Kiến trúc ứng dụng bao gồm cả phần giao diện (frontend) và phần máy chủ (backend) được phát triển trong cùng một hệ thống.
GCN / GAT / GraphSAGE	Các kiến trúc mạng nơ-ron đồ thị (Graph Neural Network) dùng để học đặc trưng và phát hiện quan hệ bất thường trên dữ liệu dạng đồ thị ví-giao dịch.
Holder	Địa chỉ ví đang nắm giữ một loại token nhất định.

(còn tiếp ở trang sau)

(Bảng thuật ngữ - tiếp theo)

Thuật ngữ	Giải thích
Heuristic	Quy tắc suy luận thực nghiệm dùng để đưa ra kết quả gần đúng khi dữ liệu không cung cấp trực tiếp thông tin cần thiết; kết quả cần được đối chiếu và không bảo đảm đúng cho mọi trường hợp.
Indexer	Hệ thống thu thập, giải mã và lập chỉ mục dữ liệu blockchain để có thể truy vấn hiệu quả mà không phải đọc lại toàn bộ dữ liệu thô từ node.
Lazy Loading (Tải dữ liệu động)	Kỹ thuật chỉ tải dữ liệu khi thực sự có nhu cầu sử dụng, thay vì tải toàn bộ dữ liệu ngay từ đầu.
Internationalization (i18n)	Quá trình thiết kế phần mềm để có thể hỗ trợ nhiều ngôn ngữ và quy tắc trình bày dữ liệu mà không phải viết lại từng màn hình.
Localization (Bản địa hóa)	Việc điều chỉnh chuỗi giao diện, số, ngày giờ, tiền tệ và cách trình bày cho một ngôn ngữ hoặc khu vực cụ thể; Yoco hiện hỗ trợ locale tiếng Anh và tiếng Việt.
LLM (Large Language Model)	Mô hình ngôn ngữ lớn, được sử dụng để diễn giải các chỉ số đã tính toán thành nội dung bằng ngôn ngữ tự nhiên.
Monolith	Kiến trúc backend được triển khai như một ứng dụng thống nhất duy nhất, nhưng bên trong được chia thành nhiều lớp và miền nghiệp vụ riêng biệt.
Monorepo	Cách tổ chức mã nguồn trong đó nhiều thành phần của dự án như frontend, backend và tài liệu kỹ thuật cùng nằm trong một kho mã nguồn duy nhất.
NFT (Non-Fungible Token)	Token không thể thay thế, đại diện cho một tài sản số có tính duy nhất trên blockchain.
On-chain / Off-chain	On-chain là dữ liệu được ghi nhận trực tiếp trên blockchain; off-chain là dữ liệu nằm ngoài blockchain, không được ghi nhận trực tiếp trên chuỗi.

(còn tiếp ở trang sau)

(Bảng thuật ngữ - tiếp theo)

Thuật ngữ	Giải thích
ORM (Object-Relational Mapping)	Công cụ ánh xạ giữa cấu trúc bảng trong cơ sở dữ liệu quan hệ với các đối tượng trong mã nguồn, giúp truy vấn dữ liệu an toàn hơn về kiểu dữ liệu.
Partial-range cache reuse	Kỹ thuật tái sử dụng phần dữ liệu đã đồng bộ trong một khoảng thời gian, chỉ truy xuất bổ sung phần dữ liệu còn thiếu khi người dùng thay đổi khoảng thời gian phân tích.
PnL (Profit and Loss)	Lãi/lỗ, chỉ số phản ánh kết quả tăng hoặc giảm giá trị tài sản của một ví theo thời gian.
Realized / Unrealized PnL	Lãi/lỗ đã thực hiện là kết quả của phần vị thế đã bán hoặc đóng; lãi/lỗ chưa thực hiện là phần biến động của tài sản vẫn đang được nắm giữ.
Pool (Pool thanh khoản)	Quỹ thanh khoản chứa cặp tài sản, dùng để thực hiện giao dịch hoán đổi trên các sàn phi tập trung.
Provider (Nhà cung cấp dữ liệu)	Nhà cung cấp dữ liệu bên thứ ba, ví dụ Helius, CoinGecko hoặc Birdeye, nơi hệ thống truy xuất dữ liệu on-chain hoặc dữ liệu thị trường.
Fail-fast	Nguyên tắc dừng xử lý và báo lỗi ngay khi phát hiện dữ liệu hoặc trạng thái không hợp lệ, thay vì tiếp tục với kết quả không chắc chắn.
Hono RPC	Cơ chế tạo client có kiểu dữ liệu từ định nghĩa route Hono, giúp client và server dùng chung hợp đồng về tham số và phản hồi API.
Runtime validation	Việc kiểm tra cấu trúc và giá trị dữ liệu khi chương trình đang chạy; trong Yoca, lớp kiểm tra này chủ yếu dùng Zod tại biên API.
Rate limit (Giới hạn truy vấn)	Giới hạn về số lượng hoặc tần suất yêu cầu mà một API cho phép trong một khoảng thời gian nhất định.
Request coalescing	Cơ chế gộp nhiều yêu cầu dữ liệu giống nhau phát sinh đồng thời thành một lần gọi duy nhất đến nhà cung cấp.

(còn tiếp ở trang sau)

(Bảng thuật ngữ - tiếp theo)

Thuật ngữ	Giải thích
RPC (Remote Procedure Call)	Giao thức gọi hàm từ xa, được dùng để tương tác trực tiếp với node của mạng lưới Solana.
Schema (Lược đồ dữ liệu)	Mô tả cấu trúc các bảng, cột và mối quan hệ trong cơ sở dữ liệu.
Sequence Diagram (Sơ đồ tuần tự)	Sơ đồ mô tả trình tự tương tác giữa các thành phần của hệ thống theo thời gian.
Tiêu chí chấp nhận (Acceptance Criteria)	Điều kiện cụ thể dùng để xác định một chức năng hoặc luồng sử dụng đã đáp ứng yêu cầu đặt ra hay chưa.
Shard / Sharding	Kỹ thuật phân chia dữ liệu hoặc cấu hình, ví dụ danh sách địa chỉ theo dõi, thành nhiều nhóm nhỏ hơn để dễ quản lý và tránh vượt giới hạn của nhà cung cấp.
Sparkline	Biểu đồ nhỏ, đơn giản, dùng để thể hiện nhanh xu hướng biến động của một chỉ số theo thời gian.
Stablecoin	Loại token có giá trị được neo theo một tài sản ổn định, phổ biến nhất là đồng USD.
Swap	Giao dịch hoán đổi trực tiếp giữa hai loại token thông qua giao thức phi tập trung.
Token	Đơn vị tài sản số được phát hành và giao dịch trên blockchain, có thể đại diện cho tiền mã hóa, quyền sở hữu hoặc một loại tài sản khác.
TTL (Time To Live)	Khoảng thời gian một dữ liệu cache được xem là còn hợp lệ trước khi hệ thống cần cập nhật lại từ nhà cung cấp.
Use Case (Tình huống sử dụng)	Mô tả một tương tác cụ thể giữa tác nhân, có thể là người dùng hoặc hệ thống bên ngoài, với hệ thống đang xây dựng.
User Journey (Hành trình người dùng)	Chuỗi bước mà người dùng thực hiện qua nhiều màn hình hoặc chức năng để hoàn thành một mục tiêu trong hệ thống.
Wallet (Ví)	Địa chỉ dùng để lưu trữ, quản lý và thực hiện giao dịch tài sản số trên blockchain.

(còn tiếp ở trang sau)

(Bảng thuật ngữ - tiếp theo)

Thuật ngữ	Giải thích
Wash trading	Hành vi giao dịch được phối hợp thực hiện nhằm tạo ra khối lượng hoặc mức độ hoạt động giả cho một tài sản.
Watchlist (Danh sách theo dõi)	Danh sách token hoặc ví được người dùng lưu lại để tiện theo dõi.
Web3	Thế hệ ứng dụng web được xây dựng trên nền tảng blockchain, hướng đến việc phi tập trung hóa dữ liệu và quyền kiểm soát.
Webhook	Cơ chế cho phép một dịch vụ tự động gửi dữ liệu đến một địa chỉ đã đăng ký ngay khi có sự kiện xảy ra, thay vì phải chủ động truy vấn liên tục.

Tóm tắt

Việc phân tích dữ liệu trên Solana thường yêu cầu người dùng kết hợp nhiều công cụ cho dữ liệu thị trường, token, pool, ví và giao dịch. Đề tài xây dựng Yoca nhằm tổ chức các đối tượng này thành một luồng sử dụng thống nhất, từ khám phá thị trường đến phân tích ví và hỗ trợ diễn giải bằng trí tuệ nhân tạo.

Yoca được triển khai dưới dạng ứng dụng web full-stack TypeScript gồm React/Vite ở phía giao diện, Hono ở phía máy chủ và PostgreSQL/Drizzle cho dữ liệu có cấu trúc cùng các read model cache. Hệ thống tích hợp nhiều nhà cung cấp theo miền nghiệp vụ, kiểm tra response bằng Zod và tái sử dụng dữ liệu theo TTL hoặc khoảng lịch sử đã đồng bộ. Trong quá trình phát triển, mô-đun Wallet được chuyển từ việc phụ thuộc nhiều vào Birdeye và tự suy luận giao dịch Helius sang kết hợp Mobula cho phân tích/tổng tài sản, Zerion cho lịch sử theo token và Helius cho portfolio, webhook cùng transaction detail.

Kết quả của đề tài là luồng Market–Pool–Token–Wallet–User–AI, các chức năng tài khoản và theo dõi, trực quan hóa giao dịch, Alert History, tín hiệu wash trading và lớp localization tiếng Anh/tiếng Việt có định dạng USD/VND. Hai test suite client/server đều đạt trong lần chạy cuối; các luồng AI chính có system instruction, ranh giới dữ liệu không tin cậy và validation đầu ra. Tuy nhiên, hệ thống vẫn phụ thuộc vào chất lượng và quota provider; giao diện chưa được đồng nhất hoàn toàn; E2E và load test vẫn cần thêm bằng chứng vận hành. Báo cáo vì vậy tập trung trình bày cả kết quả đạt được lẫn giới hạn cần tiếp tục xử lý.

Từ khóa: Solana, phân tích on-chain, dữ liệu nhiều nhà cung cấp, phân tích ví, PnL, localization, wash trading.

Chương 1

Giới thiệu

Chương này trình bày bối cảnh của tài chính phi tập trung (DeFi) và những rào cản mà người dùng gặp phải khi tiếp cận, phân tích dữ liệu trên hệ sinh thái Solana. Trên cơ sở nhận diện các vấn đề thực tiễn và khoảng trống của những giải pháp hiện có, chương xác định mục tiêu, đối tượng, phạm vi ứng dụng và hướng tiếp cận của nền tảng Yoca, đồng thời khái quát những đóng góp chính của đề tài.

1.1 Đặt vấn đề

1.1.1 Ngữ cảnh và vấn đề thực tiễn

Trong những năm gần đây, công nghệ blockchain đã trở thành nền tảng cho sự phát triển của nhiều ứng dụng tài chính phi tập trung. Trong đó, Solana là một hệ sinh thái đáng chú ý nhờ khả năng xử lý giao dịch với thông lượng cao và chi phí tương đối thấp, tạo điều kiện cho sự hình thành của nhiều token, pool thanh khoản, sàn giao dịch phi tập trung và các ứng dụng DeFi.

Cùng với sự phát triển đó, khối lượng dữ liệu được ghi nhận trên mạng lưới ngày càng gia tăng. Mặc dù dữ liệu on-chain có tính công khai và có thể kiểm chứng, việc khai thác và diễn giải dữ liệu vẫn tương đối phức

tập. Một giao dịch có thể liên quan đến nhiều tài khoản, nhiều lần chuyển token và nhiều chương trình khác nhau. Để hiểu đầy đủ bối cảnh của một token hoặc ví, người dùng thường phải kết hợp dữ liệu từ nền tảng theo dõi thị trường, công cụ phân tích ví, trình khám phá blockchain và các nguồn thông tin khác nhau.

Sự phân tán giữa các công cụ khiến quá trình phân tích dễ bị gián đoạn và đòi hỏi người dùng phải có kiến thức kỹ thuật nhất định. Các chỉ số về giá, thanh khoản, phân bố holder, lịch sử giao dịch, biến động số dư và dòng tiền thường được trình bày riêng lẻ, trong khi mối liên hệ giữa token, pool, ví và giao dịch chưa được thể hiện trong một luồng phân tích thống nhất.

Bên cạnh đó, dữ liệu thị trường có thể chịu ảnh hưởng bởi wash trading, tức các hoạt động giao dịch mang tính phối hợp nhằm tạo ra cảm nhận sai lệch về khối lượng hoặc mức độ hoạt động của một tài sản. Nếu chỉ quan sát những chỉ số tổng hợp, người dùng khó nhận biết cấu trúc giao dịch và mối quan hệ giữa các ví đứng sau những biến động này.

1.1.2 Tính cấp thiết và độ khó của bài toán

Bài toán xây dựng một nền tảng phân tích dữ liệu Solana không chỉ nằm ở việc tổng hợp và hiển thị dữ liệu mà còn liên quan đến hiệu năng, tính nhất quán và khả năng duy trì dữ liệu theo thời gian. Dữ liệu blockchain phát sinh liên tục, trong khi mỗi nhà cung cấp có phạm vi bao phủ, chu kỳ cập nhật, cấu trúc phản hồi và chính sách giới hạn truy vấn khác nhau.

Một trang phân tích token hoặc ví có thể cần đồng thời nhiều nhóm dữ liệu như thông tin nhận diện, giá, biểu đồ lịch sử, pool, holder, danh mục tài sản, swap và chuyển token. Nếu các yêu cầu này được gửi trực tiếp và lặp lại đến nhà cung cấp, hệ thống dễ gặp giới hạn truy vấn, làm tăng thời gian phản hồi và ảnh hưởng đến trải nghiệm sử dụng.

Đối với dữ liệu lịch sử, hệ thống cần có khả năng tái sử dụng phần dữ liệu đã được đồng bộ khi người dùng thay đổi khoảng thời gian phân tích.

Việc định giá danh mục ví tại một thời điểm trong quá khứ còn yêu cầu tái dựng số dư token và kết hợp với dữ liệu giá tương ứng. Những token mới, có thanh khoản thấp hoặc thiếu dữ liệu giá lịch sử cần được xử lý thận trọng để tránh tạo ra kết quả thiếu căn cứ.

Bài toán phát hiện wash trading cũng có độ phức tạp riêng. Một giao dịch đơn lẻ thường không đủ để phản ánh dấu hiệu bất thường; thay vào đó, cần xem xét quan hệ giữa nhiều ví, hướng luân chuyển token, mức độ lặp lại về thời gian và số lượng, cũng như sự xuất hiện của các chu trình giao dịch. Vì vậy, dữ liệu cần được phân tích trong bối cảnh của toàn bộ mạng lưới giao dịch thay vì chỉ được xem như những bản ghi độc lập.

Ngoài ra, việc tích hợp trí tuệ nhân tạo vào phân tích dữ liệu tài chính cần bảo đảm nội dung diễn giải có cơ sở đối chiếu. Mô hình ngôn ngữ cần sử dụng dữ liệu đã được hệ thống tính toán và chuẩn hóa, trong khi bảng, biểu đồ và giao dịch nguồn vẫn đóng vai trò là căn cứ chính của quá trình phân tích.

1.1.3 Hạn chế của các giải pháp hiện tại

Thị trường hiện có nhiều nền tảng hỗ trợ theo dõi và phân tích dữ liệu blockchain như Arkham Intelligence, Birdeye, CoinGecko, Dune Analytics và Nansen. Mỗi nền tảng có thể mạnh riêng, từ dữ liệu giá, khối lượng và thanh khoản đến truy vết dòng tiền, gắn nhãn ví hoặc xây dựng dashboard truy vấn dữ liệu.

Tuy nhiên, khi xét trong phạm vi của đề tài, vẫn tồn tại khoảng trống đối với một hệ thống tập trung vào Solana, có khả năng liên kết dữ liệu thị trường với dữ liệu token, pool, ví và giao dịch trong cùng một giao diện. Một số công cụ yêu cầu người dùng có kiến thức về truy vấn dữ liệu, trong khi các công cụ khác chủ yếu cung cấp những nhóm chỉ số riêng biệt và chưa tập trung vào việc diễn giải mối quan hệ giữa chúng.

Khả năng nhận diện dấu hiệu wash trading cũng thường được tách khỏi quá trình phân tích token và hành vi ví. Việc tổ chức giao dịch thành đồ

thị, phát hiện các chu trình đáng chú ý và trình bày cơ sở hình thành tín hiệu có thể bổ sung một góc nhìn hữu ích cho quá trình đánh giá dữ liệu thị trường.

Từ những vấn đề trên, Yoca được định hướng như một nền tảng tích hợp, cho phép người dùng khám phá thị trường, phân tích token và pool, quan sát hành vi ví, kiểm tra giao dịch, nhận diện dấu hiệu wash trading và sử dụng AI để hỗ trợ diễn giải dữ liệu.

1.2 Mục tiêu đề tài

Xuất phát từ sự phân tán của các công cụ phân tích, rào cản kỹ thuật khi đọc dữ liệu on-chain và nhu cầu nhận diện các hoạt động bất thường, đề tài đặt ra mục tiêu nghiên cứu, thiết kế và triển khai nền tảng web Yoca, chuyên biệt cho việc thu thập, xử lý, phân tích và trực quan hóa dữ liệu trên mạng lưới Solana.

Trước hết, hệ thống hướng đến việc xây dựng một luồng phân tích thống nhất, bắt đầu từ tìm kiếm và khám phá thị trường, tiếp tục đến phân tích token, pool, ví và giao dịch. Các đối tượng được liên kết để người dùng có thể đi từ góc nhìn tổng quan đến dữ liệu chi tiết mà không phải chuyển đổi liên tục giữa nhiều công cụ.

Đề tài đồng thời hướng đến việc giảm truy vấn lặp và cải thiện khả năng phản hồi thông qua cơ chế lưu trữ, tái sử dụng dữ liệu đã đồng bộ và điều phối yêu cầu đến các nhà cung cấp bên ngoài. Dữ liệu từ nhiều nguồn cần được kiểm tra và chuẩn hóa trước khi sử dụng trong bảng, biểu đồ hoặc các chức năng phân tích.

Một mục tiêu quan trọng khác là xây dựng mô-đun phát hiện dấu hiệu wash trading dựa trên đồ thị giao dịch. Mô-đun hướng đến việc nhận diện các chu trình và đặc điểm giao dịch đáng chú ý, sau đó thể hiện kết quả thông qua điểm rủi ro, danh sách ví và đồ thị quan hệ để người dùng có thể quan sát và đối chiếu.

Song song với đó, đề tài tích hợp mô hình ngôn ngữ lớn nhằm hỗ trợ

diễn giải dữ liệu token, ví và các tín hiệu bất thường. Dữ liệu được xử lý thành ngữ cảnh có cấu trúc trước khi chuyển đến mô hình AI, qua đó giúp nội dung phản hồi gắn với đúng đối tượng và phạm vi dữ liệu đang được phân tích.

Cuối cùng, Yoca hướng đến việc cung cấp trải nghiệm sử dụng nhất quán thông qua biểu đồ tương tác, chức năng so sánh ví, watchlist, cảnh báo, xuất báo cáo, quản lý hồ sơ và hỗ trợ đa ngôn ngữ. Đối với người dùng Việt Nam, localization không chỉ thay chuỗi giao diện mà còn điều chỉnh cách viết số, ngày giờ và quy đổi giá trị USD sang VND ở lớp hiển thị. Các chức năng này được tổ chức trong cùng một nền tảng nhằm giảm sự phân tán và rào cản ngôn ngữ khi khai thác dữ liệu on-chain.

1.3 Đối tượng và phạm vi ứng dụng

1.3.1 Đối tượng người dùng mục tiêu

Yoca hướng đến ba nhóm người dùng chính. Nhóm thứ nhất là người nghiên cứu dữ liệu blockchain và người tìm kiếm token tiềm năng, có nhu cầu theo dõi thanh khoản, pool, phân bố holder, hoạt động của các ví lớn và những dấu hiệu giao dịch bất thường.

Nhóm thứ hai là nhà giao dịch cá nhân cần một không gian làm việc tích hợp để quan sát dữ liệu thị trường, danh mục ví, lịch sử giao dịch, biến động số dư và kết quả lãi/lỗ theo thời gian. Các chức năng so sánh ví, cảnh báo và xuất báo cáo hỗ trợ nhóm người dùng này trong quá trình tổng hợp và đối chiếu thông tin.

Nhóm thứ ba là người dùng mới tiếp cận thị trường tài sản số. Đối với nhóm này, giao diện trực quan và trợ lý AI giúp đơn giản hóa việc đọc các chỉ số kỹ thuật, đồng thời duy trì khả năng kiểm tra thông tin thông qua bảng, biểu đồ và dữ liệu nguồn.

1.3.2 Phạm vi ứng dụng

Đề tài tập trung khai thác dữ liệu on-chain và dữ liệu thị trường thuộc hệ sinh thái Solana. Các nhóm dữ liệu chính bao gồm thông tin token, giá, vốn hóa, khối lượng, pool thanh khoản, holder, giao dịch, chuyển token, swap, danh mục ví và lịch sử số dư.

Hệ thống tích hợp nhiều nguồn dữ liệu theo từng miền nghiệp vụ. Helius và hạ tầng RPC của Solana được sử dụng cho dữ liệu giao dịch và sự kiện on-chain; CoinGecko, GeckoTerminal và Birdeye hỗ trợ dữ liệu thị trường, giá và pool; Moralis và Mobula bổ sung các thông tin liên quan đến token, holder và tokenomics. Dữ liệu được kiểm tra và chuẩn hóa trước khi đưa vào các chức năng phân tích.

Về chức năng, Yoca bao gồm quản lý tài khoản và hồ sơ; tìm kiếm và khám phá thị trường; phân tích token và pool; phân tích ví, so sánh ví và trực quan hóa giao dịch; phát hiện dấu hiệu wash trading; hỗ trợ diễn giải bằng AI; quản lý watchlist, cảnh báo, gói dịch vụ, thanh toán và xuất báo cáo.

Hệ thống đóng vai trò là công cụ quan sát và hỗ trợ phân tích. Phạm vi đề tài không bao gồm việc tự động đặt lệnh, thực hiện chiến lược đầu tư hoặc thay mặt người dùng ký giao dịch. Các kết quả AI và tín hiệu wash trading được trình bày cùng dữ liệu liên quan để người dùng có thể kiểm tra và đối chiếu.

Vì dữ liệu khai thác gắn với địa chỉ ví thật trên Solana, nhóm chúng em đặt ra một số nguyên tắc khi thiết kế và trình bày kết quả. Yoca không suy diễn danh tính hoặc chủ sở hữu thật đứng sau một địa chỉ ví; hệ thống chỉ làm việc với nhãn do người dùng tự gán hoặc dữ liệu công khai trên chuỗi. Tín hiệu wash trading và nội dung do AI diễn giải luôn được trình bày như gợi ý cần đối chiếu, không phải kết luận cuối cùng về hành vi gian lận hay một cá nhân cụ thể. Dữ liệu cá nhân mà hệ thống thu thập được giới hạn ở mức cần thiết để vận hành tài khoản, watchlist, cảnh báo và thanh toán; người dùng có thể yêu cầu xóa tài khoản cùng dữ liệu liên

quan theo quy trình xác nhận trong phần hồ sơ cá nhân.

1.4 Giải pháp và cách tiếp cận

Yoca được xây dựng theo kiến trúc ứng dụng web full-stack trong một monorepo, sử dụng TypeScript ở cả phía giao diện và phía máy chủ. Giao diện được phát triển bằng React và Vite, trong khi máy chủ sử dụng Hono để tổ chức API và xử lý nghiệp vụ. PostgreSQL kết hợp với Drizzle ORM được sử dụng để quản lý dữ liệu có cấu trúc; ECharts và React Flow hỗ trợ trực quan hóa biểu đồ và đồ thị giao dịch.

Hệ thống được tổ chức theo các lớp giao diện, API, xử lý nghiệp vụ, chuẩn hóa, lưu trữ và tích hợp dịch vụ. Giao diện không truy cập trực tiếp các nhà cung cấp dữ liệu mà gửi yêu cầu đến máy chủ để kiểm tra dữ liệu đã có, đồng bộ phần cần thiết và chuyển đổi kết quả về cấu trúc thống nhất. Cách tiếp cận này giúp hạn chế sự phụ thuộc trực tiếp giữa giao diện với từng nguồn dữ liệu.

Đối với dữ liệu ví, hệ thống chuẩn hóa các hoạt động giao dịch và kết hợp dữ liệu số dư với giá lịch sử để phục vụ quá trình tổng hợp và trực quan hóa. Đối với wash trading, dữ liệu chuyển token được biểu diễn thành đồ thị có hướng để phân tích các chu trình và đặc điểm giao dịch đáng chú ý. Đối với AI, máy chủ xây dựng ngữ cảnh có cấu trúc từ dữ liệu đã xử lý trước khi yêu cầu mô hình tạo phần giải thích.

Cách tiếp cận trên giúp Yoca kết hợp nhiều lớp dữ liệu trong một luồng sử dụng thống nhất, đồng thời tạo cơ sở để mở rộng nguồn dữ liệu và chức năng phân tích trong tương lai.

1.5 Đóng góp của đề tài

Về mặt kỹ thuật, đề tài xây dựng một kiến trúc tích hợp dữ liệu phù hợp với hệ thống phân tích blockchain sử dụng nhiều nhà cung cấp. Cơ chế điều phối yêu cầu và tái sử dụng dữ liệu theo phạm vi thời gian giúp

giảm truy vấn lặp, duy trì dữ liệu lịch sử và cải thiện khả năng phản hồi trong những lần truy cập tiếp theo.

Đề tài đồng thời xây dựng quy trình phân tích ví kết hợp dữ liệu giao dịch, sổ dư và giá lịch sử. Dữ liệu được chuẩn hóa trước khi sử dụng để tính giá trị danh mục, PnL, khối lượng và các chỉ số so sánh, qua đó tạo nguồn dữ liệu thống nhất cho bảng, biểu đồ, báo cáo và các chức năng phân tích hỗ trợ bởi AI.

Một đóng góp nổi bật của đề tài là mô-đun phân tích dấu hiệu wash trading dựa trên cấu trúc đồ thị giao dịch. Hệ thống kết hợp mối quan hệ giữa các ví với những đặc điểm về chu trình, thời gian, số lượng và khối lượng giao dịch để xác định các địa chỉ hoặc cụm hoạt động đáng chú ý. Kết quả được trình bày trên đồ thị tương tác và đi kèm thông tin giải thích, giúp người dùng quan sát được cả tín hiệu rủi ro và cơ sở hình thành tín hiệu.

Về mặt trải nghiệm, Yoca hướng đến nhóm người dùng mới hoặc chưa có nhiều kiến thức chuyên sâu về blockchain. Thay vì yêu cầu người dùng tự tổng hợp dữ liệu từ nhiều nền tảng và diễn giải các chỉ số kỹ thuật riêng lẻ, hệ thống liên kết dữ liệu thị trường, token, pool, ví và giao dịch trong một luồng sử dụng thống nhất. Trợ lý AI hỗ trợ chuyển các dữ liệu đã được xử lý thành nội dung dễ tiếp cận hơn, trong khi bảng, biểu đồ và giao dịch nguồn vẫn được duy trì để người dùng kiểm tra và từng bước hình thành khả năng đọc dữ liệu on-chain.

Trong bối cảnh lĩnh vực blockchain tại Việt Nam đang phát triển và ngày càng thu hút sự quan tâm của cộng đồng, Yoca góp phần mở rộng khả năng tiếp cận các công cụ phân tích dữ liệu on-chain cho người dùng trong nước. Nhóm chúng em xây dựng một lớp localization dùng chung thay vì để từng màn hình tự dịch và định dạng dữ liệu. Việc hỗ trợ tiếng Việt, định dạng ngày giờ phù hợp, quy đổi giá trị sang VND và diễn giải các chỉ số bằng ngôn ngữ tự nhiên giúp giảm rào cản về ngôn ngữ và kiến thức chuyên môn. Qua đó, đề tài góp phần thúc đẩy việc ứng dụng dữ liệu blockchain trong học tập, nghiên cứu và theo dõi thị trường, đồng thời tạo

nền tảng cho sự phát triển của các công cụ phân tích tài sản số phù hợp hơn với nhu cầu của người dùng Việt Nam.

1.6 Bố cục của báo cáo

Báo cáo được cấu trúc thành sáu chương. Chương 1 trình bày bối cảnh, vấn đề, mục tiêu, phạm vi và hướng tiếp cận của đề tài. Chương 2 khảo sát các hệ thống liên quan, làm rõ khoảng trống mà Yoca hướng đến và giới thiệu những nguồn dữ liệu đang được sử dụng. Chương 3 phân tích hành trình người dùng, các nhóm yêu cầu chức năng, yêu cầu chất lượng và tiêu chí chấp nhận. Chương 4 trình bày kiến trúc tổng thể, hợp đồng dữ liệu, cơ chế truy xuất-lưu đệm và thiết kế cơ sở dữ liệu. Chương 5 mô tả quá trình triển khai, kết quả hiện thực, những khó khăn kỹ thuật và đánh giá hệ thống. Cuối cùng, Chương 6 tổng kết kết quả đạt được, trình bày đóng góp, giới hạn và các hướng phát triển tiếp theo.

Chương 2

Hệ thống và nguồn dữ liệu liên quan

Chương này trình bày những hệ thống mà nhóm chúng em đã khảo sát khi xác định hướng phát triển cho Yoca, sau đó đi vào các nguồn dữ liệu được sử dụng trong phiên bản hiện tại. Hai nội dung có liên hệ với nhau nhưng không hoàn toàn đồng nhất: Arkham, Dune hay Nansen giúp nhóm hình dung cách một sản phẩm phân tích on-chain tổ chức thông tin cho người dùng, còn CoinGecko, Birdeye, Helius, Mobula, Zerion và Moralis cung cấp dữ liệu để hệ thống vận hành. Việc tách hai góc nhìn này giúp báo cáo làm rõ cả khoảng trống sản phẩm lẫn những ràng buộc thực tế đã ảnh hưởng đến kiến trúc của Yoca.

2.1 Các hệ thống liên quan

Trong giai đoạn đầu của đề án, từ ngày 07/09/2025 đến ngày 21/09/2025, nhóm chúng em khảo sát năm nền tảng gồm Arkham Intelligence, Birdeye, CoinGecko, Dune Analytics và Nansen. Đây không phải những sản phẩm giống Yoca ở mọi chức năng. Mỗi nền tảng đại diện cho một cách tiếp cận khác nhau: điều tra thực thể và dòng tiền, theo dõi token theo thời gian thực, tổng hợp thị trường, truy vấn dữ liệu tùy biến

hoặc phân tích nhóm ví có ảnh hưởng. Nhờ đặt các cách tiếp cận cạnh nhau, nhóm xác định rõ hơn phần nào nên học hỏi và phần nào cần giảm lược để phù hợp với người dùng của đề tài.

2.1.1 Arkham Intelligence



Hình 2.1: Logo nền tảng Arkham Intelligence

URL: <https://intel.arkm.com/>

Arkham Intelligence tập trung vào việc liên kết địa chỉ blockchain với thực thể, trực quan hóa quan hệ giữa các ví và hỗ trợ truy vết dòng tiền [2]. Điểm nhóm chúng em quan tâm không chỉ là số lượng dữ liệu được hiển thị, mà là cách Arkham biến những địa chỉ khó đọc thành một mạng lưới có ngữ cảnh. Người dùng có thể bắt đầu từ một ví, theo dõi tài sản di chuyển qua các địa chỉ trung gian và quan sát mối quan hệ giữa những thực thể đã được gắn nhãn.

Cách tiếp cận này có giá trị cao đối với điều tra chuyên sâu, nhưng cũng khiến giao diện chứa nhiều thông tin và đòi hỏi người dùng hiểu mục đích của từng công cụ. Yoca không đặt mục tiêu xây dựng một cơ sở định danh thực thể có quy mô tương đương. Bài học nhóm rút ra là dữ liệu ví chỉ thực sự hữu ích khi được đặt trong ngữ cảnh; vì vậy, Yoca ưu tiên danh mục, lịch sử giao dịch, biến động tài sản và những tín hiệu hành vi có thể giải thích trực tiếp cho người dùng.

2.1.2 Birdeye và CoinGecko



Hình 2.2: Logo nền tảng Birdeye và CoinGecko

URL: <https://birdeye.so/> và <https://www.coingecko.com/>

Birdeye và CoinGecko cùng giúp người dùng tiếp cận dữ liệu token, nhưng có trọng tâm khác nhau. Birdeye nổi bật ở dữ liệu DEX, pool, giao dịch và biến động ngắn hạn, đặc biệt trong hệ sinh thái Solana [3]. CoinGecko có phạm vi thị trường rộng hơn, cung cấp danh mục coin, metadata, giá, vốn hóa, khối lượng và lịch sử thị trường trên nhiều hệ sinh thái [8]. Với người dùng, cả hai đều rút ngắn đáng kể quá trình tìm token, quan sát biểu đồ và đánh giá tình trạng thị trường.

Qua hai nền tảng này, nhóm chúng em nhận thấy một trang thị trường tốt phải giúp người dùng đi từ thông tin tổng quan đến một token hoặc pool cụ thể mà không bị đứt mạch. Tuy nhiên, góc nhìn thị trường chưa đủ để trả lời tài sản trong một ví đã thay đổi như thế nào, giao dịch nào tạo ra lãi hoặc lỗ, hay hành vi của ví có điểm gì đáng chú ý. Yếu tố sử dụng thị trường và token làm điểm bắt đầu, rồi nối chúng với phân tích ví thay vì xây dựng hai nhóm chức năng rời nhau.

2.1.3 Dune Analytics và Nansen



Hình 2.3: Logo nền tảng Dune Analytics và Nansen

URL: <https://dune.com/> và <https://www.nansen.ai/>

Dune Analytics cho phép cộng đồng truy vấn dữ liệu blockchain đã được lập chỉ mục và xây dựng dashboard bằng SQL [10]. Sức mạnh của Dune nằm ở khả năng tùy biến: cùng một tập dữ liệu có thể được dùng để trả lời nhiều câu hỏi nghiên cứu khác nhau. Đổi lại, người dùng phải hiểu cấu trúc dữ liệu và biết cách xây dựng hoặc lựa chọn truy vấn phù hợp. Đây không phải rào cản lớn với nhà phân tích, nhưng khó trở thành trải nghiệm mặc định cho người dùng chỉ muốn xem nhanh một token hoặc ví.

Nansen tiếp cận bài toán từ dữ liệu ví đã được gắn nhãn và các nhóm hành vi như smart money [20]. Thay vì yêu cầu người dùng tự đọc từng giao dịch, nền tảng làm nổi bật dòng tiền và hoạt động của những nhóm ví đáng chú ý. Hướng này gần với mong muốn của nhóm chúng em về việc đưa dữ liệu đã được xử lý đến gần người dùng hơn, nhưng Nansen thiên về người dùng chuyên nghiệp và lớp dữ liệu độc quyền. Yoca lựa chọn phạm vi hẹp hơn: chuẩn bị sẵn các luồng phân tích phổ biến, giữ khả năng kiểm chứng các chỉ số nền và dùng AI như lớp hỗ trợ diễn giải chứ không thay thế dữ liệu.

2.2 Đối chiếu và bài toán đặt ra cho Yoca

Các nền tảng được khảo sát không tạo thành một thang đo hơn-kém duy nhất. Arkham và Nansen có lợi thế về ngữ cảnh thực thể; Birdeye và CoinGecko mạnh ở dữ liệu thị trường; Dune trao nhiều quyền tùy biến cho người phân tích. Vì vậy, nhóm chúng em chọn các tiêu chí đối chiếu dựa trên hành trình mà Yoca muốn hỗ trợ: khám phá thị trường, đi sâu vào token hoặc pool, xem hoạt động của ví, lưu lại đối tượng quan tâm và nhận phần diễn giải dễ đọc hơn.

Bảng 2.1: Đối chiếu định hướng của các nền tảng phân tích dữ liệu blockchain

Tiêu chí	Arkham	Birdeye / CoinGecko	Dune	Nansen	Yoca
Khám phá thị trường và token	Có, nhưng không phải trọng tâm	Là chức năng trọng tâm	Phụ thuộc dashboard	Có tín hiệu thị trường	Là điểm bắt đầu của luồng phân tích
Phân tích pool và giao dịch	Theo hướng dòng tiền	Mạnh về DEX, pool và trade	Tùy biến bằng truy vấn	Có dữ liệu đã tổng hợp	Nổi pool, token và hoạt động ví
Phân tích ví	Mạnh về thực thể và truy vết	Có tra cứu, độ sâu tùy sản phẩm	Có thể tự xây bằng SQL	Mạnh về nhân ví và smart money	Danh mục, PnL, lịch sử và biểu đồ ví
Mức độ tùy biến	Theo công cụ có sẵn	Theo màn hình và bộ lọc có sẵn	Rất cao	Theo gói phân tích có sẵn	Theo các luồng nghiệp vụ định trước
Khả năng tiếp cận	Thiên về điều tra dữ liệu	Dễ tiếp cận ở góc nhìn thị trường	Cần hiểu dữ liệu và truy vấn	Thiên về người dùng chuyên nghiệp	Hướng đến luồng đọc dữ liệu trực quan
Cá nhân hóa	Watchlist và thực thể quan tâm	Watchlist/cảnh báo tùy nền tảng	Người dùng tự xây dashboard	Theo dõi ví và nhóm nhân	Watchlist, nhân ví, cảnh báo và hồ sơ
Diễn giải dữ liệu	Ngữ cảnh thực thể	Chủ yếu trình bày chỉ số	Do tác giả dashboard quyết định	Tín hiệu được đóng gói sẵn	AI diễn giải trên dữ liệu đã kiểm chứng
Phát hiện hành vi bất thường	Hỗ trợ điều tra quan hệ ví	Không phải trọng tâm chung	Có thể tự xây bằng truy vấn	Có nhân và tín hiệu chuyên sâu	Có mô-đun wash trading trong phạm vi đề tài
Ngôn ngữ và đơn vị hiển thị	Giao diện tiếng Anh	Giao diện tiếng Anh, giá niêm yết USD	Giao diện tiếng Anh	Giao diện tiếng Anh	Tiếng Anh và tiếng Việt, quy đổi hiển thị sang VND

Bảng 2.1 cho thấy Yoca không thay thế hoàn toàn một nền tảng nào ở trên. Phạm vi của đề tài nằm ở việc nối các góc nhìn vốn thường bị tách rời trong một luồng sử dụng thống nhất. Người dùng có thể bắt đầu từ tổng quan thị trường, mở pool và token liên quan, sau đó chuyển sang ví để xem danh mục, lịch sử hoạt động và kết quả phân tích. Các chức năng theo dõi, cảnh báo và AI được đặt sau lớp dữ liệu này để hỗ trợ sử dụng lâu dài, không phải để che đi nguồn số liệu ban đầu.

Khi bắt tay vào hiện thực hóa luồng trên, khó khăn lớn nhất của nhóm không còn nằm ở việc vẽ thêm một biểu đồ. Dữ liệu cần thiết được phân tán giữa nhiều dịch vụ; cùng một khái niệm có thể có khóa định danh, độ sâu lịch sử và trường dữ liệu khác nhau. Một phản hồi hợp lệ về mặt HTTP vẫn có thể thiếu symbol, giá hoặc phần mô tả giao dịch mà hệ thống mong đợi. Bên cạnh đó, quota của các gói miễn phí và chi phí của gói trả phí buộc Yoca phải cân nhắc dữ liệu nào cần cập nhật ngay, dữ liệu nào

có thể dùng lại và chức năng nào không thể phụ thuộc vào một nguồn duy nhất. Đây là bài toán dẫn trực tiếp đến lớp tích hợp provider, kiểm tra dữ liệu khi chạy, cache và các read model được trình bày trong chương Kiến trúc và thiết kế hệ thống.

2.3 Các nhà cung cấp dữ liệu được sử dụng trong Yoca

Yoca không tự vận hành một blockchain indexer hoàn chỉnh. Trong phạm vi thời gian và hạ tầng của đề án, nhóm chúng em sử dụng các API chuyên biệt rồi chuẩn hóa kết quả về mô hình nội bộ. Danh sách dưới đây phản ánh mã nguồn tại thời điểm rà soát ngày 11/07/2026, không dựa đơn thuần vào những utility hoặc thử nghiệm từng tồn tại trong repository. Chẳng hạn, CoinMarketCap và Dune SIM không còn được các service chính sử dụng nên không được xem là nguồn dữ liệu vận hành của phiên bản báo cáo này.

2.3.1 Dữ liệu thị trường, token và pool

CoinGecko hiện là nguồn có phạm vi sử dụng rộng nhất ở miền thị trường. Yoca dùng các API coin cho danh sách token, metadata, tìm kiếm, giá, vốn hóa và lịch sử; đồng thời dùng nhóm Onchain DEX do GeckoTerminal cung cấp cho pool, giao dịch và tìm kiếm theo địa chỉ. CoinGecko cung cấp REST API trong cả gói Demo miễn phí và các gói trả phí; nhóm endpoint công khai bao phủ dữ liệu coin, market và một phần dữ liệu on-chain [7]. Điều này phù hợp với quá trình phát triển và thử nghiệm, nhưng các giới hạn theo phút và phạm vi lịch sử khiến dữ liệu vẫn cần được cache thay vì gọi lại theo từng tương tác.

Birdeye tiếp tục được sử dụng cho trending token, recent trades, trader gainers/losers và một số đường lấy lịch sử giá. Ở giai đoạn trước, Birdeye còn đảm nhiệm nhiều dữ liệu ví hơn. Nhóm chúng em từng được hỗ trợ

gói Lite trong ba tháng, nhờ đó có thể thử nghiệm các chức năng yêu cầu lịch sử sâu hơn. Khi điều kiện hỗ trợ thay đổi, việc duy trì phần phân tích ví trên cùng nguồn này không còn phù hợp với ngân sách đề án. Theo bảng giá tại thời điểm khảo sát, gói Standard miễn phí có phạm vi API giới hạn; gói Lite có giá 39 USD/tháng, gồm 2,5 triệu compute unit và phần lớn API [5]. Quá trình này khiến nhóm nhìn quota và chi phí như một ràng buộc kiến trúc, thay vì chỉ là thông tin cấu hình.

Mobula cũng được dùng để lấy thống kê holder của token. Tuy nhiên, vai trò quan trọng hơn của dịch vụ này nằm ở miền ví, được trình bày ở phần tiếp theo.

2.3.2 Dữ liệu ví và giao dịch

Helius là nguồn dữ liệu gắn chặt với Solana trong Yoca. Hệ thống dùng Helius Wallet API cho portfolio, lịch sử, transfer, funded-by và identity; Enhanced Transactions được dùng cho giao dịch đã giải mã, webhook và một số luồng phân tích chi tiết. API Enhanced Transactions chuyển transaction signature thành cấu trúc dễ đọc hơn với token transfer, native transfer, account data và instruction [13]. Lớp trừu tượng này giúp nhóm bắt đầu nhanh, nhưng không phải mọi swap đều có event hoặc trường dẫn xuất đầy đủ. Kinh nghiệm tự tổng hợp PnL từ dữ liệu này, cùng giới hạn của cách suy luận bằng balance change và program data, sẽ được phân tích trong chương Kiến trúc và thiết kế hệ thống.

Mobula được bổ sung trong giai đoạn cuối và hiện là nguồn chính cho wallet analysis, PnL theo giai đoạn, vị thế theo token, lịch sử hoạt động và biểu đồ tổng giá trị ví. API của Mobula cung cấp wallet activity, position, historical net worth và trading analysis trên một mô hình dữ liệu đã được tổng hợp [17]. Đây là phần từng đòi hỏi nhóm thu thập và giải mã một lượng giao dịch khó dự đoán theo từng ví. Free tier công bố 10.000 credit/tháng; riêng endpoint wallet analysis tiêu tốn 5 credit, trong khi một số endpoint wallet khác tính theo số chain [18]. Mức miễn phí này

phù hợp với quy mô đồ án, nhưng vẫn đòi hỏi cache và giới hạn refresh để tránh lãng phí credit.

Zerion được sử dụng cho lịch sử số dư ở cấp token. Trước đó nhóm từng dùng Zerion cho cả biểu đồ tổng giá trị ví, nhưng kết quả có thời điểm sai lệch lớn so với portfolio Helius do khác biệt trong cách nhận diện tài sản có giá trị. Sau khi đối chiếu, nhóm chuyển total balance sang Mobula và giữ Zerion ở phần lịch sử từng token, nơi dữ liệu vẫn đáp ứng được yêu cầu. Zerion cung cấp gói Developer miễn phí và API cho portfolio, transaction, position cùng chart dữ liệu ví [25]. Quota thực tế phụ thuộc key và chính sách tại thời điểm cấp, vì vậy báo cáo không xem đây là một giới hạn cố định lâu dài.

Moralis hiện giữ vai trò hẹp hơn, chủ yếu bổ sung metadata token và phục vụ một đường lấy swap còn lại. Dịch vụ cung cấp nhiều nhóm Wallet, Token, NFT và Price API theo cơ chế compute unit; free tier tại thời điểm khảo sát công bố 40.000 compute unit mỗi ngày [19]. Nhóm không triển khai fallback qua nhiều provider cho mọi service. Mặc dù fallback có thể tăng khả năng tìm thấy dữ liệu, sự khác nhau về response và semantics làm tăng đáng kể chi phí duy trì schema, kiểm thử và xử lý lỗi. Yoca chỉ kết hợp hoặc thay nguồn ở những miền có lợi ích rõ ràng.

Bảng 2.2: Vai trò của các nhà cung cấp dữ liệu trong phiên bản Yoca hiện tại

Nhà cung cấp	Dữ liệu Yoca đang sử dụng	Điểm cần kiểm soát
CoinGecko	Token list, metadata, search, market, lịch sử giá, pool và pool trade	Rate limit, định danh CoinGecko và trường enrichment có thể thiếu
Birdeye	Trending, recent trades, trader ranking và một số lịch sử giá	Compute unit, phạm vi gói và các đường tích hợp kế thừa
Helius	Portfolio Solana, transaction, transfer, identity, funded-by và webhook	Độ sâu lịch sử và transaction abstraction không luôn đủ cho PnL
Mobula	Wallet analysis, PnL, position, activity, total balance history và holder	Credit theo endpoint/chain và coupling của read model phân tích ví
Zerion	Lịch sử số dư theo token	Sai khác trong phân loại tài sản và quota theo key
Moralis	Metadata token và một đường dữ liệu swap	Vai trò hẹp, chi phí duy trì nếu mở rộng thành fallback tổng quát

Sự phân công trong Bảng 2.2 là trạng thái hiện tại, không phải thiết kế cố định từ đầu. Khoảng một tháng cuối của quá trình phát triển, việc

chuyển các luồng Balance Chart, PnL và Wallet Overview sang Mobula và Zerion đã làm gián đoạn phần lớn màn hình phân tích ví. Trải nghiệm này giúp nhóm chúng em nhận ra rằng một provider không chỉ ảnh hưởng đến hàm gọi API; nó còn ảnh hưởng đến schema kiểm tra khi chạy, cache, bảng dữ liệu và contract mà frontend đang sử dụng. Chương Kiến trúc và thiết kế hệ thống sẽ trình bày việc thay nguồn như một quá trình thay thế tăng dần, trong đó mô hình mới được xây dựng và kiểm tra trước khi phần cũ được loại bỏ.

2.4 Tổng kết chương

Qua việc khảo sát các sản phẩm hiện có, nhóm chúng em xác định Yoca nên tập trung vào một hành trình phân tích liên mạch thay vì cố đạt độ sâu chuyên biệt của từng nền tảng. Hệ thống kết nối thị trường, token, pool và ví, sau đó bổ sung theo dõi và AI trên dữ liệu đã được chuẩn hóa. Điểm khác biệt mà đề tài hướng đến không nằm ở số lượng dashboard, mà ở cách các góc nhìn này hỗ trợ nhau trong cùng một trải nghiệm.

Việc rà soát mã nguồn cũng cho thấy dữ liệu của Yoca được hình thành từ nhiều provider với vai trò khác nhau và đã thay đổi trong quá trình phát triển. CoinGecko và Birdeye cung cấp phần lớn dữ liệu thị trường; Helius cung cấp lớp dữ liệu Solana; Mobula đảm nhiệm các phân tích ví nặng; Zerion và Moralis bổ sung những miền hẹp hơn. Những lựa chọn này tạo ra các yêu cầu về kiểm tra dữ liệu, cache, khả năng thay nguồn và thiết kế cơ sở dữ liệu. Chương tiếp theo tiếp tục xác định yêu cầu của hệ thống từ góc nhìn người dùng; các quyết định kiến trúc phát sinh từ những ràng buộc dữ liệu trên sẽ được trình bày trong chương Kiến trúc và thiết kế hệ thống.

Chương 3

Phân tích và đặc tả yêu cầu

Chương 2 đã làm rõ khoảng trống mà Yoca hướng đến và những nguồn dữ liệu được sử dụng để hiện thực hóa sản phẩm. Từ nền tảng đó, chương này chuyển sang góc nhìn người dùng: Yoca phục vụ ai, người dùng đi qua hệ thống như thế nào và những yêu cầu nào cần được đáp ứng. Các quyết định về framework, cache, provider adapter và cơ sở dữ liệu được tách sang chương Kiến trúc và thiết kế hệ thống để phân đặc tả không bị trộn với cách cài đặt.

3.1 Người dùng và hành trình phân tích

Yoca được xây dựng cho người dùng muốn quan sát thị trường và hoạt động ví Solana nhưng không muốn tự ghép dữ liệu từ nhiều công cụ. Hệ thống có hai nhóm người dùng chính. Khách có thể truy cập các dữ liệu công khai như thị trường, token, pool, giao dịch và ví. Người dùng đã đăng ký có thêm một không gian cá nhân để lưu đối tượng quan tâm, quản lý nhân ví, thiết lập cảnh báo, theo dõi gói sử dụng và dùng các chức năng AI có giới hạn theo tài khoản.

Thay vì xem những chức năng trên như các trang độc lập, nhóm chúng

em tổ chức chúng thành một hành trình phân tích. Người dùng có thể bắt đầu ở Market Overview để nhận biết token hoặc pool đáng chú ý, mở Pool Detail để xem thanh khoản và giao dịch, rồi chuyển đến Token Overview để đọc dữ liệu thị trường, holder và lịch sử biến động. Khi một địa chỉ ví xuất hiện trong quá trình tìm hiểu, Wallet Overview tiếp tục cung cấp danh mục, lịch sử hoạt động, PnL và các biểu đồ liên quan. Nếu muốn theo dõi lâu dài, người dùng đăng nhập để lưu ví hoặc token, đặt nhãn, cấu hình cảnh báo và sử dụng AI để diễn giải tập dữ liệu đã chọn.

Luồng này không buộc mọi phiên sử dụng phải đi qua tất cả màn hình. Người dùng vẫn có thể tìm thẳng một địa chỉ hoặc mở trực tiếp trang ví. Tuy nhiên, việc xác định một hành trình chung giúp nhóm tránh phát triển các chức năng rời rạc: dữ liệu thị trường tạo bối cảnh cho token, token dẫn đến pool và giao dịch, còn dữ liệu ví cho thấy một chủ thể đã hành động như thế nào trong bối cảnh đó.

Ngoài luồng chính, Yoca còn hỗ trợ so sánh nhiều ví, xem chi tiết giao dịch và phân tích dấu hiệu wash trading. Đây là các nhánh dành cho người dùng cần đi sâu hơn sau bước quan sát tổng quan. Mô-đun wash trading trình bày quan hệ ví-giao dịch dưới dạng đồ thị và các chỉ số bất thường; kết quả chỉ đóng vai trò tín hiệu hỗ trợ kiểm tra, không phải kết luận chắc chắn về hành vi gian lận.

3.2 Yêu cầu chức năng

Từ hành trình trên và các chức năng đã được đối chiếu với mã nguồn, yêu cầu của Yoca được gom thành sáu nhóm. Cách gom này mô tả năng lực người dùng nhận được, không bám theo từng route hoặc component cụ thể.

3.2.1 Khám phá thị trường, token và pool

Hệ thống cần cho phép người dùng xem tổng quan thị trường, các token hoặc pool nổi bật và những nhóm dữ liệu như trending, khối lượng, số giao dịch, tăng giảm giá và pool mới. Người dùng có thể tìm kiếm bằng tên, symbol hoặc địa chỉ; từ kết quả tìm kiếm, hệ thống cần dẫn đến đúng trang token, pool hoặc ví. Với một token, Yoca cần trình bày metadata, dữ liệu thị trường, biểu đồ lịch sử, holder và các pool liên quan. Với một pool, hệ thống cần hiển thị cặp tài sản, thanh khoản, biến động và giao dịch gần đây ở mức dữ liệu nguồn cho phép.

3.2.2 Phân tích ví và giao dịch

Khi người dùng nhập một địa chỉ ví hợp lệ, hệ thống cần cung cấp phần tổng quan, danh mục tài sản, phân bổ token, lịch sử transfer và swap, PnL cùng các chuỗi thời gian liên quan đến số dư và hoạt động giao dịch. Người dùng có thể thay đổi khoảng thời gian, lọc hoặc phân trang những tập dữ liệu lớn mà không phải tải toàn bộ lịch sử vào một lần. Yoca cũng cần cho phép mở chi tiết một giao dịch và xem dữ liệu đã được diễn giải bên cạnh thông tin nguồn cần thiết để đối chiếu.

Chức năng so sánh nhiều ví sử dụng cùng các khái niệm ở cấp độ tổng hợp. Người dùng có thể chọn nhiều địa chỉ và đặt các chỉ số chính cạnh nhau, thay vì mở từng trang rồi ghi chép thủ công. Với các dữ liệu không đủ độ phủ, giao diện phải thể hiện giới hạn của kết quả thay vì ngẫm xem phần dữ liệu hiện có là toàn bộ lịch sử.

3.2.3 Phát hiện dấu hiệu wash trading

Yoca cần cho phép người dùng chọn token hoặc dữ liệu giao dịch phù hợp để quan sát các mẫu đáng chú ý như chu trình chuyển tài sản, cấu trúc tập trung quanh một số ví và biến động khối lượng bất thường. Hệ thống trình bày kết quả bằng đồ thị, bảng và phần giải thích để người dùng có

thể lần ngược về các ví hoặc giao dịch liên quan. Các tín hiệu phải được diễn đạt theo mức độ nghi vấn; hệ thống không được tự động đồng nhất một mẫu bất thường với kết luận gian lận.

3.2.4 Tài khoản và không gian cá nhân

Hệ thống cần hỗ trợ đăng ký và đăng nhập bằng email/mật khẩu, tài khoản Google hoặc chữ ký ví Solana. Sau khi xác thực, người dùng có thể cập nhật thông tin cá nhân, quản lý phương thức đăng nhập, liên kết ví, thay đổi mật khẩu hoặc xóa tài khoản theo các bước xác nhận tương ứng.

Trong không gian cá nhân, người dùng có thể lưu token và ví vào watchlist, gắn nhãn riêng cho địa chỉ ví và xem lại những đối tượng đã quan tâm. Người dùng cũng có thể xem trạng thái subscription và lịch sử thanh toán. Những dữ liệu thuộc sở hữu cá nhân chỉ được truy cập bởi đúng tài khoản đã xác thực.

3.2.5 Cảnh báo và theo dõi

Người dùng đã đăng ký cần có khả năng tạo, xem, chỉnh sửa trạng thái và xóa cảnh báo token hoặc cảnh báo hoạt động giao dịch. Một cảnh báo gồm đối tượng theo dõi, điều kiện, thời hạn và kênh nhận được hệ thống hỗ trợ. Với cảnh báo ví, webhook có thể được dùng để tiếp nhận hoạt động mới mà không cần truy vấn liên tục.

Yêu cầu này bao gồm cả việc lưu lại lịch sử thông báo theo tài khoản, đánh dấu trạng thái đã đọc/chưa đọc và cơ chế chống gửi trùng khi webhook gửi lại cùng một sự kiện; đây không phải các mục tùy chọn mà là điều kiện để tính năng cảnh báo đáng tin cậy trong sử dụng thực tế. Mức độ hoàn thiện thật của các yêu cầu này được trình bày ở chương Triển khai, kiểm thử và đánh giá.

3.2.6 AI và giới hạn sử dụng

AI trong Yoca nhận ngữ cảnh đã được hệ thống chuẩn bị từ dữ liệu token, ví hoặc kết quả wash trading. Người dùng đã đăng ký có thể đặt câu hỏi, yêu cầu tóm tắt và quản lý phiên chat hoặc prompt mẫu. Phản hồi cần giữ liên hệ với dữ liệu đầu vào, tránh tự tạo số liệu và cho phép người dùng quay lại bảng, biểu đồ hoặc giao dịch nguồn khi cần kiểm tra.

Do mỗi lần gọi mô hình phát sinh chi phí, hệ thống cần theo dõi lượt dùng theo tài khoản và gói dịch vụ. Khi vượt quyền lợi, người dùng phải nhận được thông báo rõ ràng và đường dẫn đến thông tin gói phù hợp, thay vì gặp một lỗi kỹ thuật không giải thích được.

3.3 Yêu cầu chất lượng

Yêu cầu đầu tiên là tính nhất quán và khả năng kiểm chứng của dữ liệu. Dữ liệu bên ngoài phải được kiểm tra trước khi đi vào các phép tính hoặc giao diện. Những trường enrichment như symbol, logo hoặc chỉ số biến động có thể thiếu; hệ thống cần có trạng thái thay thế phù hợp, trong khi dữ liệu bắt buộc cho nghiệp vụ phải tạo lỗi rõ ràng. Các kết quả PnL, biểu đồ và AI cần chỉ ra phạm vi thời gian hoặc nguồn dữ liệu đủ để người dùng hiểu giới hạn của chúng.

Yêu cầu thứ hai là khả năng phản hồi trong điều kiện phụ thuộc API bên ngoài. Các trang không nên gọi lại cùng một dữ liệu đắt đỏ ở mọi lần mở. Hệ thống cần tái sử dụng dữ liệu còn phù hợp, chỉ đồng bộ phần thiếu và thể hiện trạng thái tải ở từng khu vực. Báo cáo không đặt một ngưỡng thời gian phản hồi cố định khi nhóm chưa có benchmark đầy đủ; thay vào đó, hiệu quả cache hit và cache miss sẽ được đánh giá bằng số liệu thực tế nếu phép đo được thực hiện ở chương Triển khai, kiểm thử và đánh giá.

Yêu cầu thứ ba là xử lý lỗi có thể hiểu được. Địa chỉ hoặc tham số sai phải được phát hiện trước khi gọi provider. Khi nguồn bên ngoài trả lỗi, response rỗng hoặc sai cấu trúc, backend cần chuyển chúng thành lỗi có

phân loại để frontend hiển thị trạng thái phù hợp. Hệ thống không được tiếp tục tính toán trên dữ liệu chưa qua kiểm tra chỉ để giữ màn hình có kết quả.

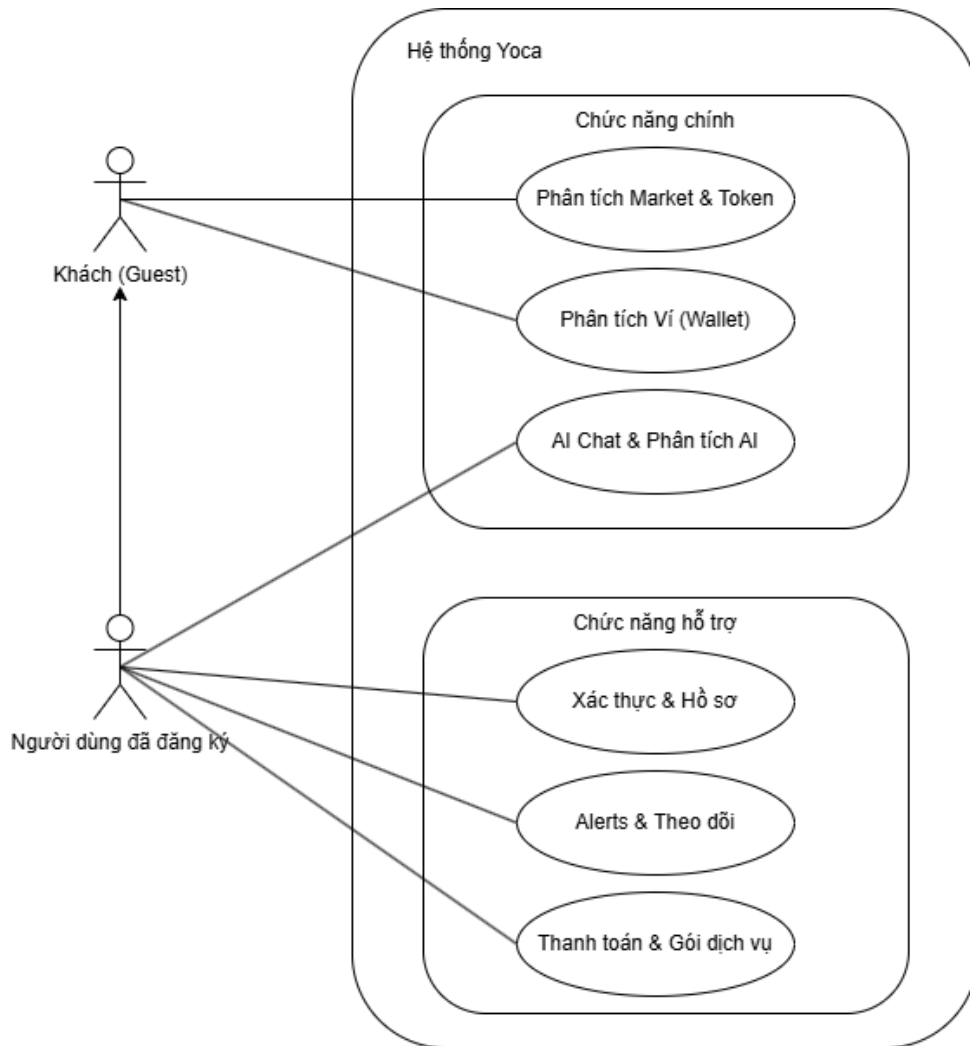
Yêu cầu thứ tư là bảo vệ dữ liệu và thao tác cá nhân. Phiên đăng nhập, watchlist, nhân ví, cảnh báo, lịch sử thanh toán và chức năng AI theo gói phải được kiểm tra quyền truy cập ở phía server. API key, secret xác thực và thông tin thanh toán không được đưa vào bundle phía client. Những thao tác nhạy cảm như liên kết ví hoặc xóa tài khoản cần có bước chứng minh quyền sở hữu hoặc xác nhận bổ sung. Với chức năng AI, câu hỏi người dùng, lịch sử hội thoại và dữ liệu do công cụ trả về phải được xem là dữ liệu không tin cậy; chúng không được phép thay đổi system instruction hoặc yêu cầu mô hình tiết lộ ngữ cảnh ẩn.

Yêu cầu cuối cùng là khả năng sử dụng và bảo trì. Một người dùng mới cần tìm được token hoặc ví, hiểu trạng thái loading/error/empty và đọc được các chỉ số chính mà không phải biết cấu trúc API. Các màn hình cốt lõi cần dùng thuật ngữ, màu sắc và hành vi component nhất quán. Khi chuyển giữa tiếng Việt và tiếng Anh, hệ thống phải thay đổi đồng bộ chuỗi giao diện, định dạng số, ngày giờ và đơn vị tiền tệ; giá trị gốc bằng USD chỉ được quy đổi sang VND tại lớp trình bày để không làm thay đổi dữ liệu nghiệp vụ. Ở phía phát triển, contract dữ liệu giữa client và server phải đủ rõ để thay đổi provider không buộc giao diện phụ thuộc vào response thô của từng dịch vụ.

3.4 Sơ đồ Use Case

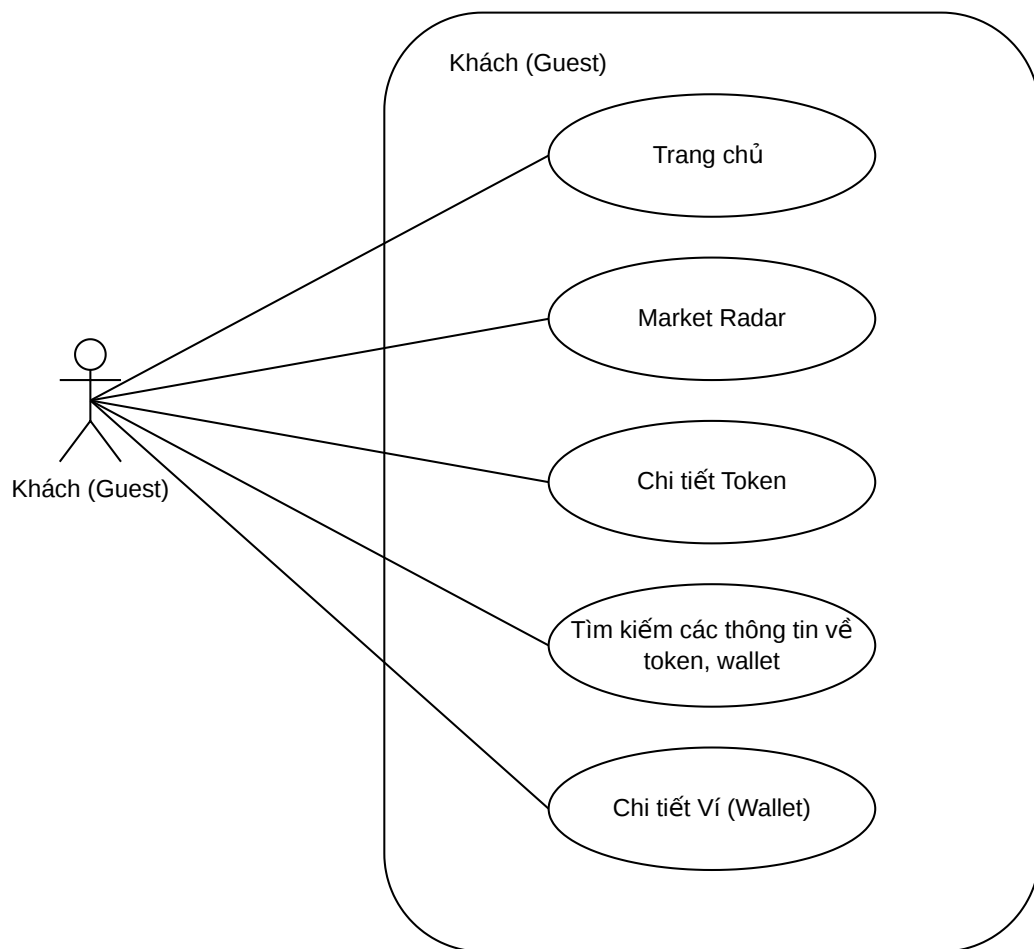
Sơ đồ tổng quát trong Hình 3.1 mô tả hai tác nhân Khách và Người dùng đã đăng ký. Người dùng đã đăng ký kế thừa các chức năng công khai của Khách, đồng thời có thêm những chức năng cần dữ liệu cá nhân hoặc phát sinh chi phí như quản lý tài khoản, cảnh báo, subscription và AI. Các provider, webhook hoặc cổng thanh toán là hệ thống hỗ trợ ở phía sau; chúng sẽ xuất hiện trong sơ đồ kiến trúc hoặc sơ đồ tuần tự thay vì được

xem như người dùng của Yoca.



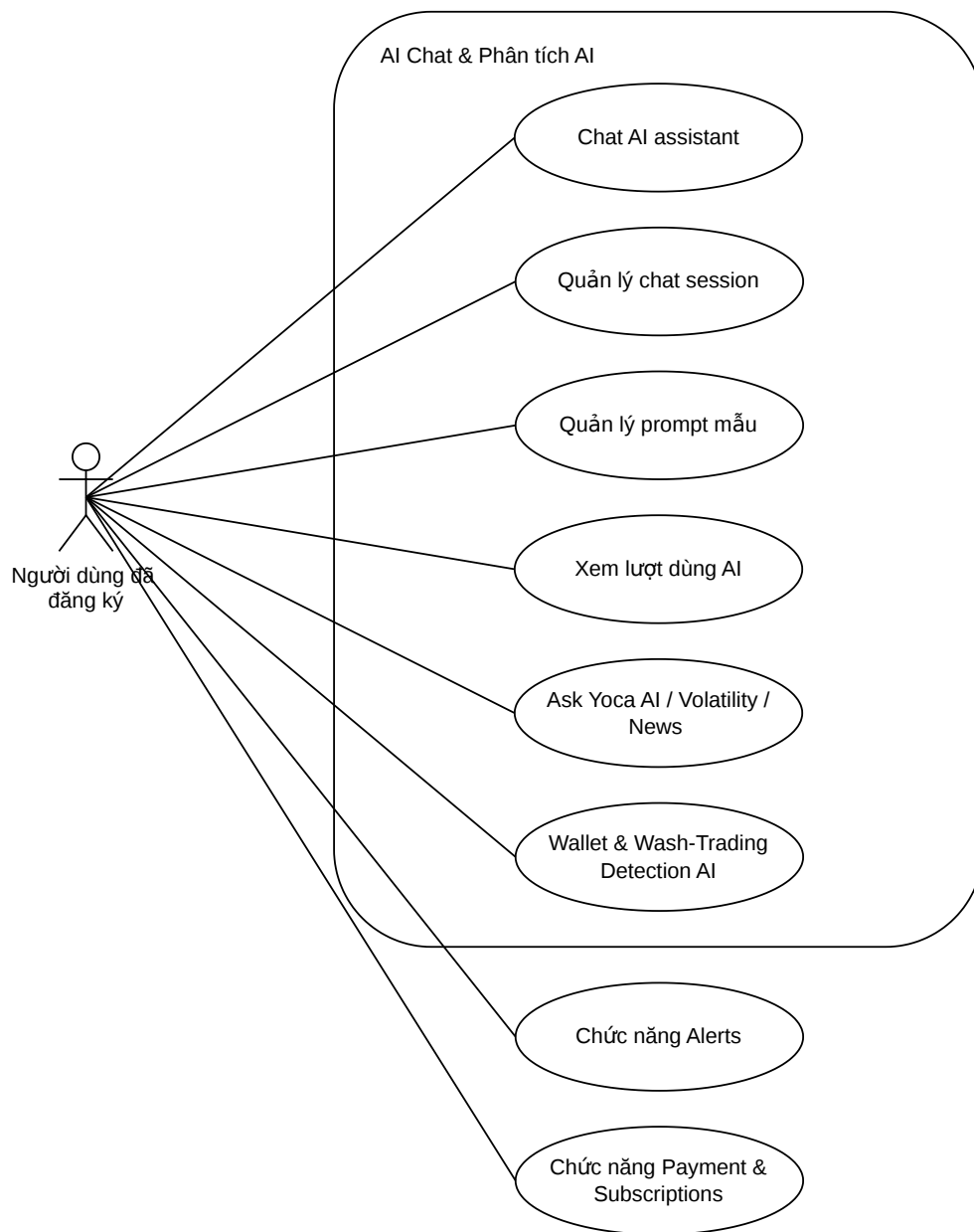
Hình 3.1: Sơ đồ Use Case tổng quát của hệ thống Yoca

Hình 3.2 triển khai chi tiết những điểm vào công khai: trang chủ, Market Radar, tìm kiếm, chi tiết token và chi tiết ví. Sơ đồ này được giản lược theo mục tiêu sử dụng; pool, giao dịch và các nhánh phân tích sâu được truy cập từ những điểm vào trên thay vì tách thành quá nhiều use case nhỏ.



Hình 3.2: Sơ đồ Use Case của Khách

Hình 3.3 tập trung vào các chức năng bổ sung sau đăng nhập, gồm AI chat, quản lý phiên và prompt, theo dõi lượt dùng, cảnh báo, payment và subscription. Các thao tác profile, watchlist và nhãn ví được gom trong nhóm quản lý tài khoản ở sơ đồ tổng quát để hình không trở thành danh sách toàn bộ route của hệ thống.



Hình 3.3: Sơ đồ Use Case của Người dùng đã đăng ký

3.5 Kế hoạch kiểm thử và tiêu chí chấp nhận

Ở giai đoạn đặc tả, nhóm chúng em xác định các kịch bản cần kiểm tra trước khi xem một luồng là có thể trình diễn. Đây là kế hoạch và tiêu chí

chấp nhận, chưa phải tuyên bố rằng toàn bộ kịch bản đã được kiểm thử. Kết quả thực tế, môi trường chạy và những phần chưa bao phủ sẽ được trình bày riêng trong chương Triển khai, kiểm thử và đánh giá.

Bảng 3.1: Kịch bản và tiêu chí chấp nhận các luồng chính

Luồng	Kịch bản cần kiểm tra	Tiêu chí chấp nhận
Khám phá thị trường	Mở Market Overview, tìm token/pool/ví và chuyển giữa các trang liên quan	Kết quả dẫn đến đúng đối tượng; loading, empty và upstream error được hiển thị rõ; dữ liệu chính có đơn vị và thời điểm phù hợp
Token và pool	Mở token, xem chart/holder/pool rồi mở một pool và giao dịch gần đây	Address được giữ nhất quán qua các trang; biểu đồ và bảng không dùng dữ liệu sai schema; trường tùy chọn bị thiếu không làm hỏng toàn trang
Phân tích ví	Mở ví, thay khoảng thời gian, xem portfolio, PnL, balance, transfer và swap	Dữ liệu thuộc đúng ví và khoảng thời gian; danh sách lớn được phân trang/lọc; kết quả có giới hạn độ phủ phải được ghi rõ
So sánh và wash trading	So sánh nhiều ví; mở phân tích wash trading của một token và lần theo giao dịch liên quan	Các chỉ số dùng cùng định nghĩa; đồ thị liên kết được với dữ liệu nguồn; tín hiệu bất thường không được trình bày như kết luận chắc chắn
Tài khoản cá nhân	Đăng nhập, cập nhật profile, quản lý watchlist/label và thử truy cập dữ liệu của tài khoản khác	Thao tác hợp lệ được lưu; route bảo vệ từ chối request thiếu hoặc sai quyền; dữ liệu cá nhân không bị trả cho tài khoản khác
Cảnh báo	Tạo, sửa trạng thái và xóa cảnh báo token hoặc ví; tiếp nhận một sự kiện phù hợp	Rule được lưu đúng chủ sở hữu và điều kiện; sự kiện phù hợp được xử lý theo kênh hỗ trợ; lỗi provider không làm mất cấu hình cảnh báo

Luồng	Kịch bản cần kiểm tra	Tiêu chí chấp nhận
AI và subscription	Gửi câu hỏi có ngữ cảnh, quản lý session/prompt và sử dụng hết quota thử nghiệm	Phản hồi bám dữ liệu được cung cấp; lượt dùng được ghi nhận; chức năng bị khóa trả thông báo và hướng nâng gói rõ ràng

Phạm vi kiểm thử dự kiến gồm unit test cho các phép tính và normalizer, contract test bằng fixture cho response provider, route/service test với upstream được mock và một số component test cho trạng thái giao diện. Một smoke test xuyên suốt hành trình Market Overview – Token Pool – Token Overview – Wallet Overview – User – AI sẽ được ưu tiên vì nó kiểm tra sự liên kết giữa các miền thay vì chỉ xác nhận từng endpoint riêng lẻ. Các nội dung như load test, end-to-end toàn hệ thống hoặc benchmark chỉ được ghi là kết quả nếu nhóm thực sự chạy và lưu được bằng chứng.

3.6 Tổng kết chương

Chương này đã chuyển mục tiêu chung của Yoca thành một hành trình sử dụng và các yêu cầu có thể đối chiếu. Khách có thể khám phá thị trường, token, pool, giao dịch và ví; người dùng đã đăng ký có thêm không gian cá nhân, cảnh báo, subscription và AI. Những chức năng chuyên sâu như so sánh ví và wash trading được đặt như nhánh mở rộng từ luồng phân tích chính, không phải các sản phẩm tách rời.

Bên cạnh chức năng, nhóm chúng em đặt ra yêu cầu về khả năng kiểm chứng dữ liệu, xử lý lỗi, bảo vệ dữ liệu cá nhân và tính nhất quán của giao diện. Bảng tiêu chí chấp nhận là cầu nối sang phần kiểm thử sau này, nhưng không thay thế kết quả thực nghiệm. Chương tiếp theo sẽ giải thích cách kiến trúc Yoca đáp ứng các yêu cầu trên trong điều kiện dữ liệu phụ thuộc nhiều provider và đã thay đổi đáng kể trong quá trình phát triển.

Chương 4

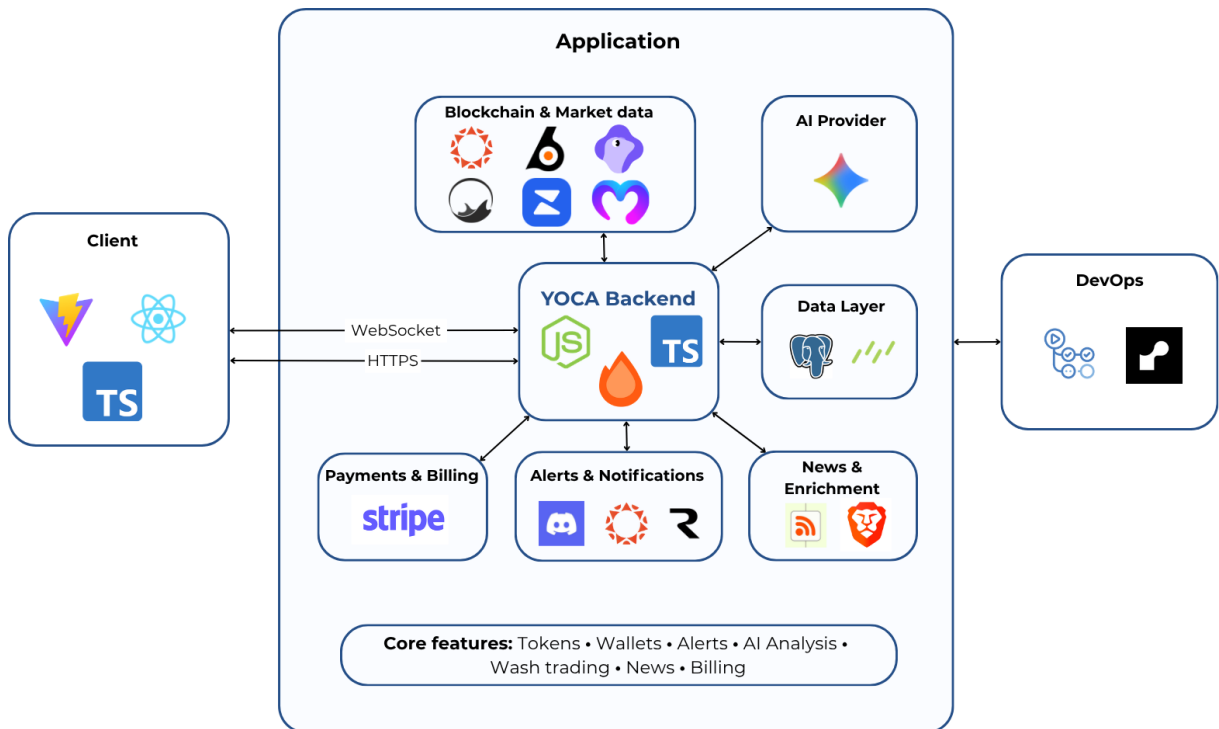
Kiến trúc và thiết kế hệ thống

Chương này trình bày cách nhóm chúng em tổ chức Yoca từ giao diện, máy chủ API đến cơ sở dữ liệu và các nhà cung cấp bên ngoài. Kiến trúc hiện tại không phải là kết quả của một bản thiết kế cố định ngay từ đầu. Nó được điều chỉnh nhiều lần khi nhóm hiểu rõ hơn dữ liệu Solana, giới hạn của các gói API và chi phí duy trì những phép phân tích ví. Vì vậy, nội dung chương tập trung vào những nguyên tắc đang thực sự chi phối mã nguồn, đồng thời chỉ rõ các điểm mà hệ thống vẫn mang dấu vết của quá trình phát triển tăng dần.

4.1 Kiến trúc tổng thể

Yoca là một ứng dụng web full-stack được quản lý trong cùng một monorepo. Phía người dùng sử dụng React và Vite; phía máy chủ là một ứng dụng Hono chạy trên Node.js, giao tiếp với client qua cả HTTPS lẫn WebSocket; dữ liệu có cấu trúc và cache bền vững được lưu trong PostgreSQL thông qua Drizzle ORM. Backend được triển khai như một monolith, nhưng mã nguồn được chia theo miền như token, pool, wallet, người dùng, cảnh báo, thanh toán và AI. Cách tổ chức này vừa đủ nhẹ cho

quy mô nhóm, đồng thời tránh chi phí vận hành nhiều dịch vụ độc lập khi các chức năng vẫn chia sẻ lược đồ và dữ liệu.



Hình 4.1: Kiến trúc tổng thể của hệ thống Yoca: backend là điểm trung chuyển duy nhất giữa client và sáu nhóm phụ thuộc ngoài, cùng DevOps là quan hệ triển khai chứ không phải một lệnh gọi runtime

Backend đóng vai trò trung chuyển duy nhất: giao diện không gọi trực tiếp API của bất kỳ nhà cung cấp nào. Sáu nhóm phụ thuộc ngoài được backend gọi tới khi cần gồm dữ liệu blockchain/thị trường (CoinGecko, Birdeye, Mobula, Zerion, Helius và Moralis, mỗi provider phụ trách một tập miền như đã trình bày ở phần tích hợp nguồn dữ liệu), AI (Gemini), lớp dữ liệu (PostgreSQL qua Drizzle), thanh toán (Stripe), cảnh báo/thông báo (Helius webhook làm nguồn sự kiện, gửi qua Resend và Discord) và tin tức/enrichment (RSS tổng hợp và Brave News). Khối DevOps ở ngoài ranh giới *Application* thể hiện quan hệ build/deploy (GitHub Actions và Render) chứ không phải một cuộc gọi runtime từ backend, khác về bản chất so với sáu nhóm còn lại.

Ranh giới trung chuyển này đặc biệt quan trọng vì các provider không chỉ khác tên trường mà còn khác cách biểu diễn token, thời gian, giá trị rỗng và phạm vi lịch sử. Nếu để từng màn hình tự xử lý những khác biệt đó, thay đổi từ một provider sẽ lan sang nhiều thành phần giao diện.

Đây là kiến trúc phân lớp theo trách nhiệm, không phải sự phân tách tuyệt đối. Trong mã nguồn còn tồn tại cả cấu trúc `routes/services` cũ và một số module mới được gom theo miền. Nhóm chúng em xem đây là kết quả của cách nâng cấp tăng dần: chức năng mới được hoàn thiện và thay thế luồng cũ, sau đó phần không còn được sử dụng mới được loại bỏ. Cách làm này giúp hệ thống tiếp tục hoạt động trong lúc dữ liệu và yêu cầu thay đổi, nhưng cũng tạo ra khoản nợ cần tiếp tục đồng nhất hóa sau đó.

4.2 Hợp đồng dữ liệu từ giao diện đến nhà cung cấp

Ứng dụng dùng TypeScript ở cả client và server. Backend xuất kiểu của toàn bộ cây route Hono, còn client khởi tạo RPC client từ chính kiểu đó. Nhờ vậy, đường dẫn, tham số và kiểu phản hồi được kiểm tra ngay trong quá trình phát triển thay vì phải duy trì thêm một bộ khai báo API viết tay. Đây là lý do nhóm chọn Hono: với quy mô hiện tại, Hono RPC tạo được hợp đồng end-to-end mà không cần sinh mã từ một đặc tả trung gian [15].

Kiểu TypeScript chỉ tồn tại trong lúc phát triển nên không thể bảo đảm dữ liệu trả về từ Internet là đúng. Trước khi đưa phản hồi của provider vào nghiệp vụ hoặc cơ sở dữ liệu, các service quan trọng sử dụng Zod để kiểm tra cấu trúc ở runtime. Khi một trường sai kiểu, thiếu ngoài dự kiến hoặc mang hình dạng khác tài liệu, hệ thống dừng tại biên tích hợp và trả lỗi thay vì tiếp tục tính toán bằng dữ liệu mơ hồ. Nhóm chúng em chủ ý chọn cách *fail-fast* này vì một kết quả PnL trông hợp lý nhưng được tính

từ dữ liệu sai nguy hiểm hơn một yêu cầu thất bại có thể truy vết.

Zod cho phép định nghĩa schema và kiểm tra dữ liệu không đáng tin cậy ngay khi chương trình đang chạy, phù hợp với vai trò bảo vệ biên tích hợp của Yoca [26].

Khi một service phát hiện lỗi ở biên tích hợp, nó không ném một **Error** chung mà trả về một lớp lỗi có kiểu riêng, mang theo mã lỗi, HTTP status và chi tiết bổ sung, ví dụ **WalletAnalysisServiceError**. Route bắt lớp lỗi này và chuyển thành một envelope JSON thống nhất dạng `{ error, code }` kèm status code tương ứng, thay vì để lỗi provider rò rỉ nguyên văn ra ngoài. Bảng 4.1 minh họa contract này cho một endpoint phân tích ví tiêu biểu.

Bảng 4.1: Ví dụ hợp đồng request/response cho endpoint phân tích ví

Trường hợp	Hình dạng phản hồi
Thành công (200)	<code>{ address, period, pnl, winRate, volumeBreakdown, ... }</code> theo đúng kiểu <code>WalletAnalysisResult</code> đã được Zod kiểm tra
Địa chỉ không hợp lệ (400)	<code>{ error: "Invalid wallet address", code: "INVALID_ADDRESS" }</code>
Upstream lỗi/hết hạn mức (502/429)	<code>{ error: "Provider request failed", code: "UPSTREAM_ERROR" }</code> , sinh ra khi <code>WalletAnalysisServiceError</code> bọc lỗi từ Mobula
Vượt giới hạn AI theo gói (429)	<code>{ errorCode, message, limit, used, remaining, resetsAt, tier, upgradePath }</code> từ <code>WalletAiDailyLimitError</code> , giúp client hiển thị đúng trạng thái thay vì lỗi chung

Việc kiểm tra chặt cũng làm lộ ra một khó khăn thực tế: tài liệu của provider không phải lúc nào cũng nói rõ trường nào có thể rỗng. Chẳng hạn, địa chỉ token là định danh chắc chắn hơn tên hoặc ký hiệu; các chỉ số biến động giá cũng có thể không tồn tại với token mới hay ít thanh khoản. Lược đồ nội bộ vì vậy không sao chép nguyên xi một phản hồi mẫu, mà chấp nhận giá trị thiếu ở nơi dữ liệu thực tế đòi hỏi và giữ ràng buộc chặt cho các định danh cốt lõi.

Một contract đại diện là API lịch sử cảnh báo. Client gọi `GET /api/alerts/history` với `page` và `limit`; server lấy `userId` từ phiên xác

thực thay vì nhận từ query, nhờ đó người dùng không thể yêu cầu lịch sử của tài khoản khác. Phản hồi gồm danh sách theo thứ tự mới nhất, tổng số bản ghi và số chưa đọc. Mỗi item giữ nguồn cảnh báo, message, severity, trạng thái email/Discord, thời điểm gửi và `readAt`. Khi cập nhật `PATCH /api/alerts/history/:id/read`, điều kiện truy vấn đồng thời dùng history ID và user ID; bản ghi không thuộc tài khoản được trả như không tồn tại.

Bảng 4.2: Contract đại diện của API Alert History

Thành phần	Dữ liệu hợp lệ	Hành vi khi không hợp lệ/không đầy đủ
List history	<code>page >= 1, 1 <= limit <= 100</code> ; user lấy từ JWT	Query sai trả lỗi validation; lỗi database trả 500; không có dữ liệu trả danh sách rỗng cùng tổng bằng 0
Read state	UUID history và boolean <code>read</code> ; update được scope bởi user	UUID sai trả 400; ID không tồn tại hoặc thuộc user khác trả 404
Delivery snapshot	Ít nhất một kênh gửi thành công; lưu attempted/succeeded riêng cho email và Discord	Partial delivery vẫn được lưu đúng trạng thái từng kênh; không kênh nào thành công thì không tạo history
Idempotency	Khóa logic từ user, transaction signature, scope và rule/ví	Webhook retry cùng sự kiện không tạo bản ghi thứ hai

4.3 Luồng truy xuất, chuẩn hóa và lưu đệm

Mẫu truy xuất phổ biến trong backend gồm hai lớp hàm. Hàm `get*` là điểm vào của route: nó tìm dữ liệu trong cơ sở dữ liệu và đánh giá độ mới. Nếu dữ liệu còn hợp lệ, kết quả được trả ngay; nếu thiếu hoặc đã cũ, hàm gọi luồng `fetch*`. Luồng thứ hai chịu trách nhiệm truy cập provider, kiểm tra phản hồi, cập nhật cơ sở dữ liệu rồi trả về cùng kiểu dữ liệu nội bộ. Không phải mọi service đều tuân theo đúng tên gọi này, nhưng đây là nguyên tắc chủ đạo đối với dữ liệu thị trường và dữ liệu ví.

Nhóm chúng em không áp dụng fallback nhiều provider một cách mặc định. Mỗi provider được chọn theo phần dữ liệu mà họ có thể mạnh và

theo hạn mức còn khả dụng. Token metadata là một trong những nơi cần nhiều nguồn bổ sung, trong khi PnL và lịch sử giao dịch ví hiện dựa chủ yếu vào Mobula; lịch sử tổng tài sản ưu tiên Mobula, còn chuỗi số dư theo từng token tiếp tục dùng Zerion. Helius vẫn giữ vai trò đối với giao dịch on-chain và webhook. Sự phân công theo miền làm giảm số nhánh khó kiểm soát, dù đồng nghĩa hệ thống vẫn phụ thuộc đáng kể vào chất lượng của từng nguồn.

Cơ sở dữ liệu vừa lưu dữ liệu nghiệp vụ, vừa là lớp cache bền vững. Thiết kế bảng không chỉ theo mục tiêu chuẩn hóa mà còn cân nhắc số lần gọi API để hoàn thiện một bản ghi. Nếu một hàng yêu cầu hai hoặc ba endpoint khác nhau nhưng các phần dữ liệu có chu kỳ cập nhật khác nhau, việc tách chúng thành các bảng nhỏ giúp chỉ làm mới phần đã cũ. Khi provider trả kèm thông tin hữu ích, service có thể cập nhật thêm bảng liên quan để tận dụng dữ liệu đã trả phí truy vấn, song việc cập nhật chéo phải đi qua khóa và ràng buộc nhất quán.

4.4 Thiết kế cơ sở dữ liệu

4.4.1 Mục tiêu và đặc điểm dữ liệu

Cơ sở dữ liệu PostgreSQL của Yoca được thiết kế theo hai lớp có mục đích khác nhau. Lớp thứ nhất là dữ liệu nền tương đối chuẩn hóa, gồm tài khoản, phương thức xác thực, watchlist, nhãn ví, cảnh báo, subscription và thanh toán. Đây là dữ liệu do người dùng hoặc hệ thống sở hữu, cần khóa ngoại và quy tắc xóa rõ ràng. Lớp thứ hai là các read model và cache bền vững cho token, pool, ví, giao dịch và kết quả phân tích. Lớp này ưu tiên khả năng đọc nhanh, tái sử dụng dữ liệu provider và làm mới từng phần hơn là loại bỏ mọi sự lặp dữ liệu.

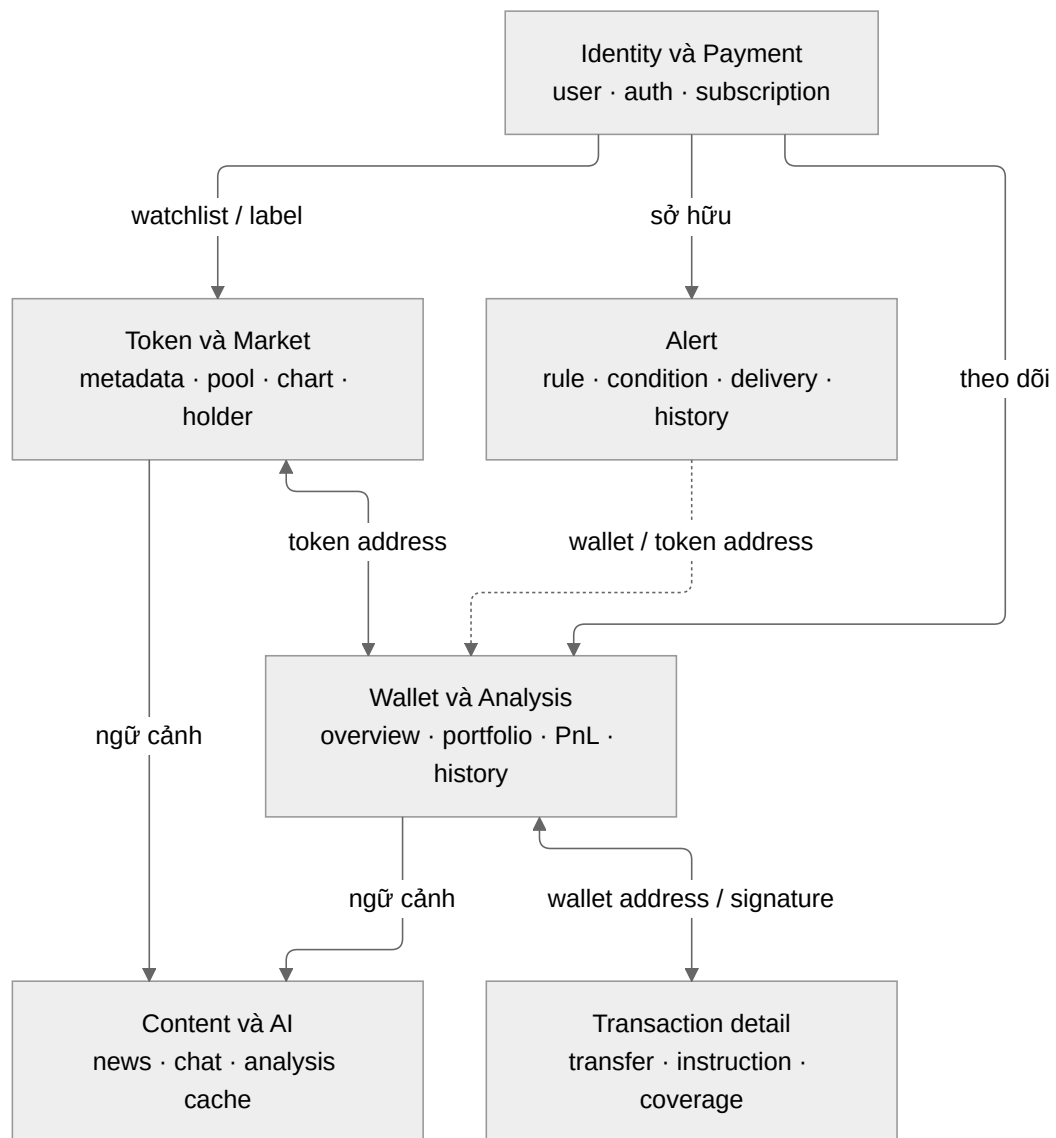
Dữ liệu trong Yoca có hai đặc điểm. Dữ liệu tài khoản và thanh toán cần quan hệ rõ ràng, ràng buộc khóa ngoại và quy tắc xóa nhất quán. Ngược lại, phần lớn dữ liệu blockchain là dữ liệu công khai được định danh

bằng token address, wallet address hoặc transaction signature. Các bảng này thường không phụ thuộc vào một bảng gốc duy nhất mà được liên kết theo khóa nghiệp vụ.

Schema được định nghĩa bằng Drizzle ORM. Kiểu dữ liệu, primary key, unique constraint và foreign key được đặt trong mã TypeScript, giúp truy vấn backend và cấu trúc PostgreSQL dùng chung một định nghĩa. Tuy nhiên, schema vật lý không phải lúc nào cũng trùng với mô hình khái niệm: wallet address, token address và signature thường là khóa nghiệp vụ liên kết nhiều bảng nhưng không bị buộc phải tham chiếu đến một bảng cha.

4.4.2 Phân nhóm dữ liệu theo miền nghiệp vụ

Cấu trúc lưu trữ được chia theo các miền identity/payment, token/-market, wallet/analysis, transaction, alert và content/AI. Hình 4.2 chỉ thể hiện quan hệ khái niệm giữa các miền; các ERD phía sau mới phân biệt foreign key vật lý với liên hệ bằng address hoặc signature.

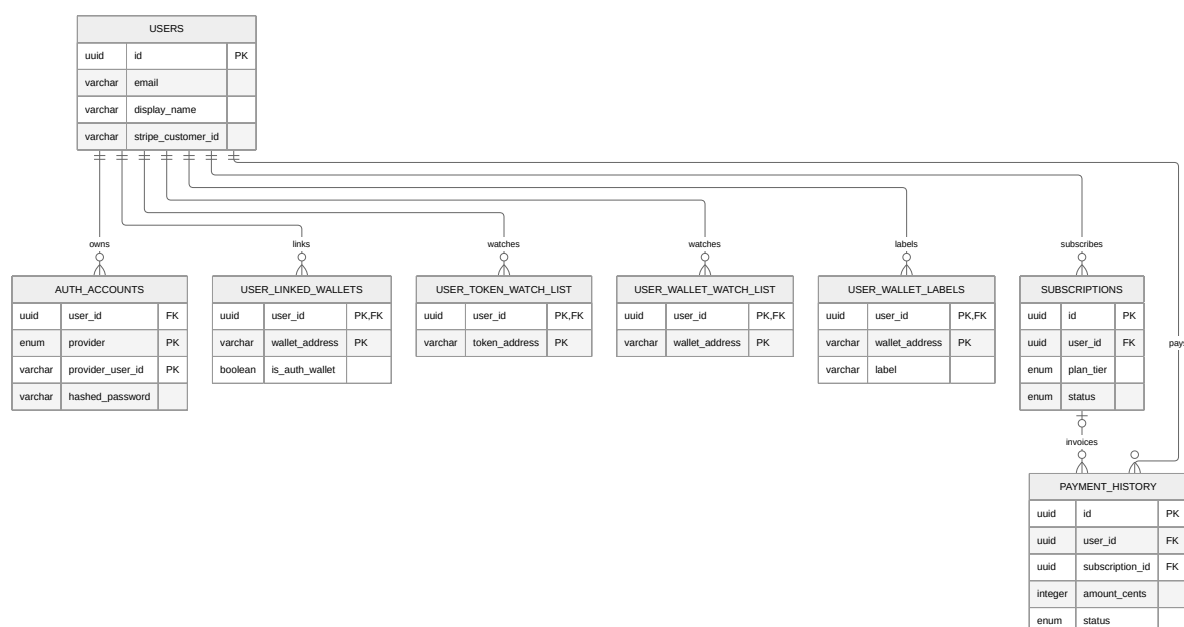


Hình 4.2: Các miền dữ liệu chính và quan hệ khái niệm

Các nhóm trên phục vụ nhu cầu truy vấn khác nhau. Vì vậy, dữ liệu có thể giao nhau về wallet address, token address hoặc transaction signature nhưng không đồng nghĩa các bảng có thể thay thế trực tiếp cho nhau. Cách phân nhóm cũng giúp ERD tổng thể không biến thành một hình quá dài theo chiều ngang và phải thu nhỏ đến mức không đọc được.

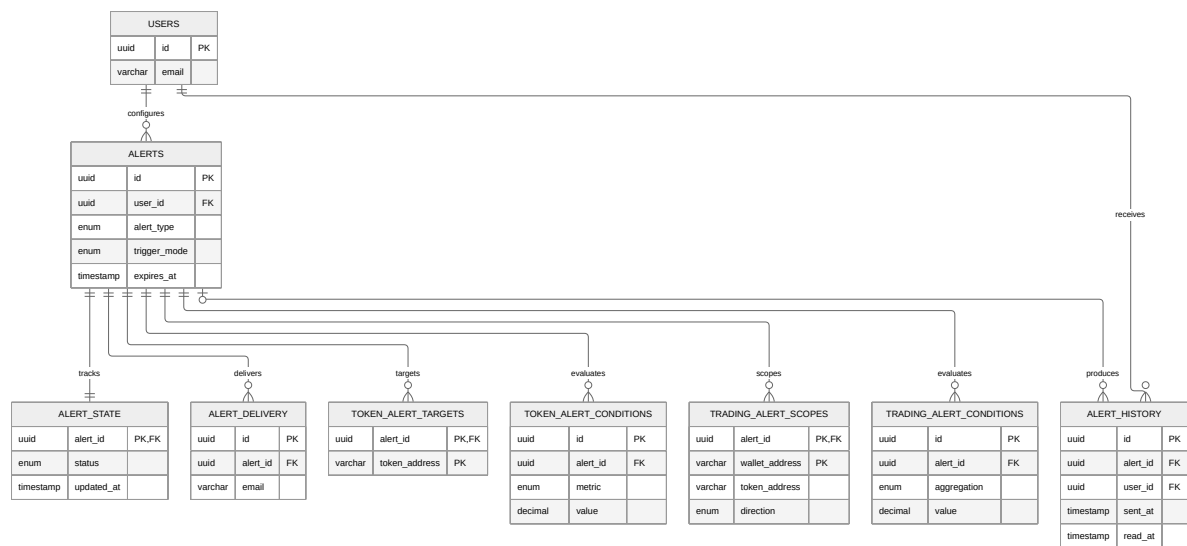
4.4.3 Sơ đồ thực thể mối quan hệ

Schema có nhiều bảng cache nên một ERD duy nhất sẽ khó đọc trên khổ A4. Phần này chia ERD theo miền nghiệp vụ và chỉ giữ primary key, foreign key cùng các cột chính. Đường liền biểu diễn foreign key được khai báo trong PostgreSQL. Đường nét đứt biểu diễn liên hệ logic thông qua address, mint hoặc signature; các liên hệ này được kiểm soát tại service thay vì bằng foreign key. Sơ đồ tổng quan ở Hình 4.2 giúp người đọc thấy toàn bộ kiến trúc, còn các hình dưới đây là những lát cắt có thể đọc được trên trang dọc A4. Cách tách này cũng cho phép đổi một miền cache mà không phải vẽ lại một ERD ngang quá lớn.



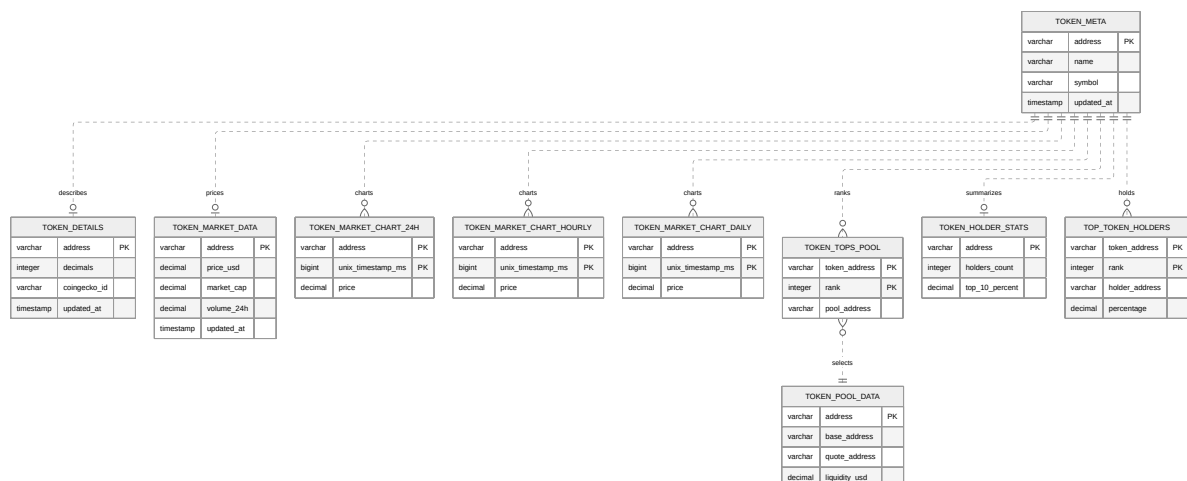
Hình 4.3: ERD nhóm tài khoản và thanh toán

Hình 4.3 thể hiện dữ liệu có tính sở hữu. Bảng **users** là gốc của dữ liệu xác thực, cấu hình cá nhân và thanh toán. Các bản ghi phụ thuộc người dùng được xóa theo quan hệ cascade; lịch sử thanh toán giữ được bản ghi khi subscription không còn tồn tại.



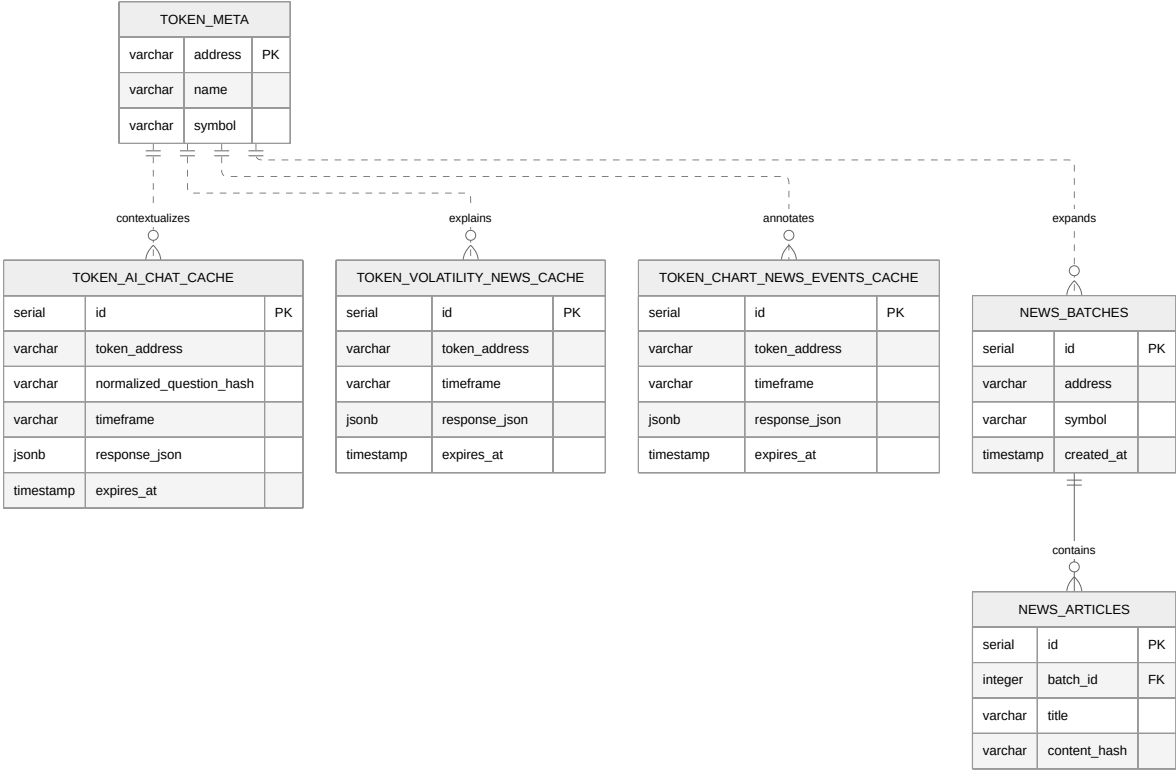
Hình 4.4: ERD nhóm cảnh báo và lịch sử cảnh báo

Hình 4.4 tách định nghĩa cảnh báo khỏi trạng thái, delivery target và điều kiện kích hoạt. **alert_history** lưu sự kiện sau khi ít nhất một kênh gửi thành công, đồng thời hỗ trợ chống ghi trùng, phân trang và trạng thái đã đọc. API chỉ cho phép người dùng truy cập hoặc cập nhật lịch sử thuộc tài khoản của mình; giao diện Profile sử dụng cùng API này để hiển thị lịch sử thực tế thay cho dữ liệu mock ban đầu.



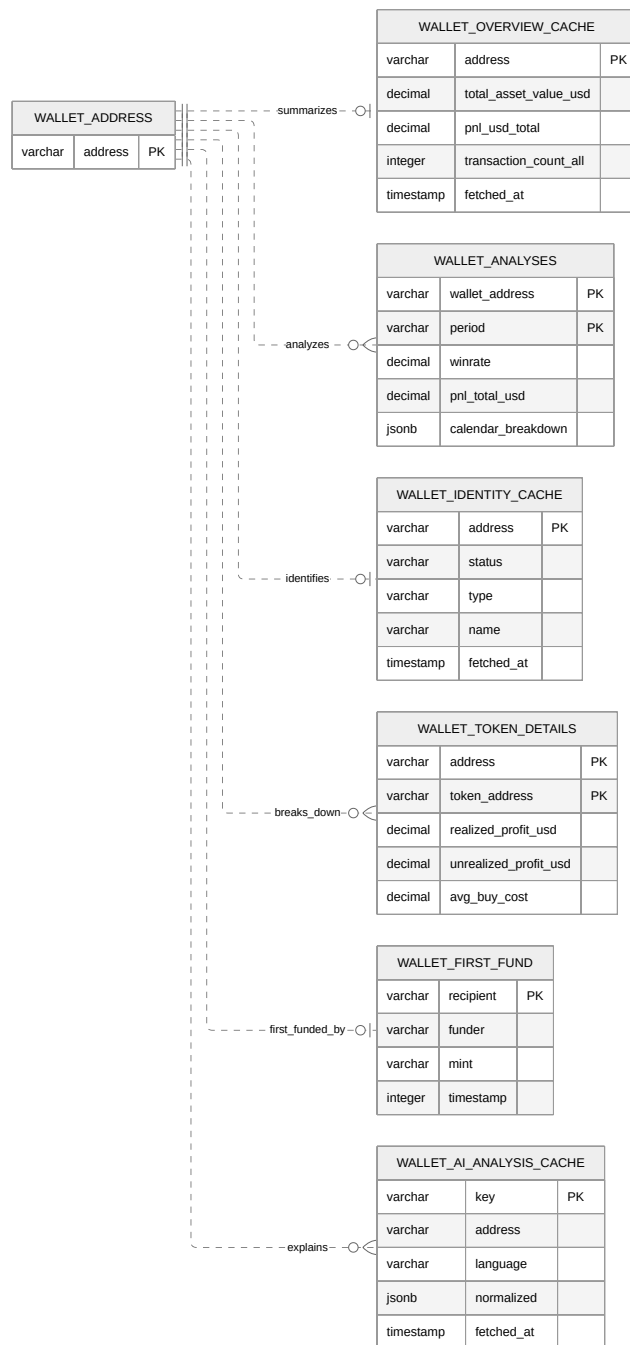
Hình 4.5: ERD nhóm token và dữ liệu thị trường

Hình 4.5 sử dụng `token_meta` như điểm tham chiếu logic. Market snapshot và chart được cập nhật theo address và mốc thời gian. Pool và holder là các read model riêng, chỉ liên kết logic với token.



Hình 4.6: ERD nhóm nội dung và AI cache theo token

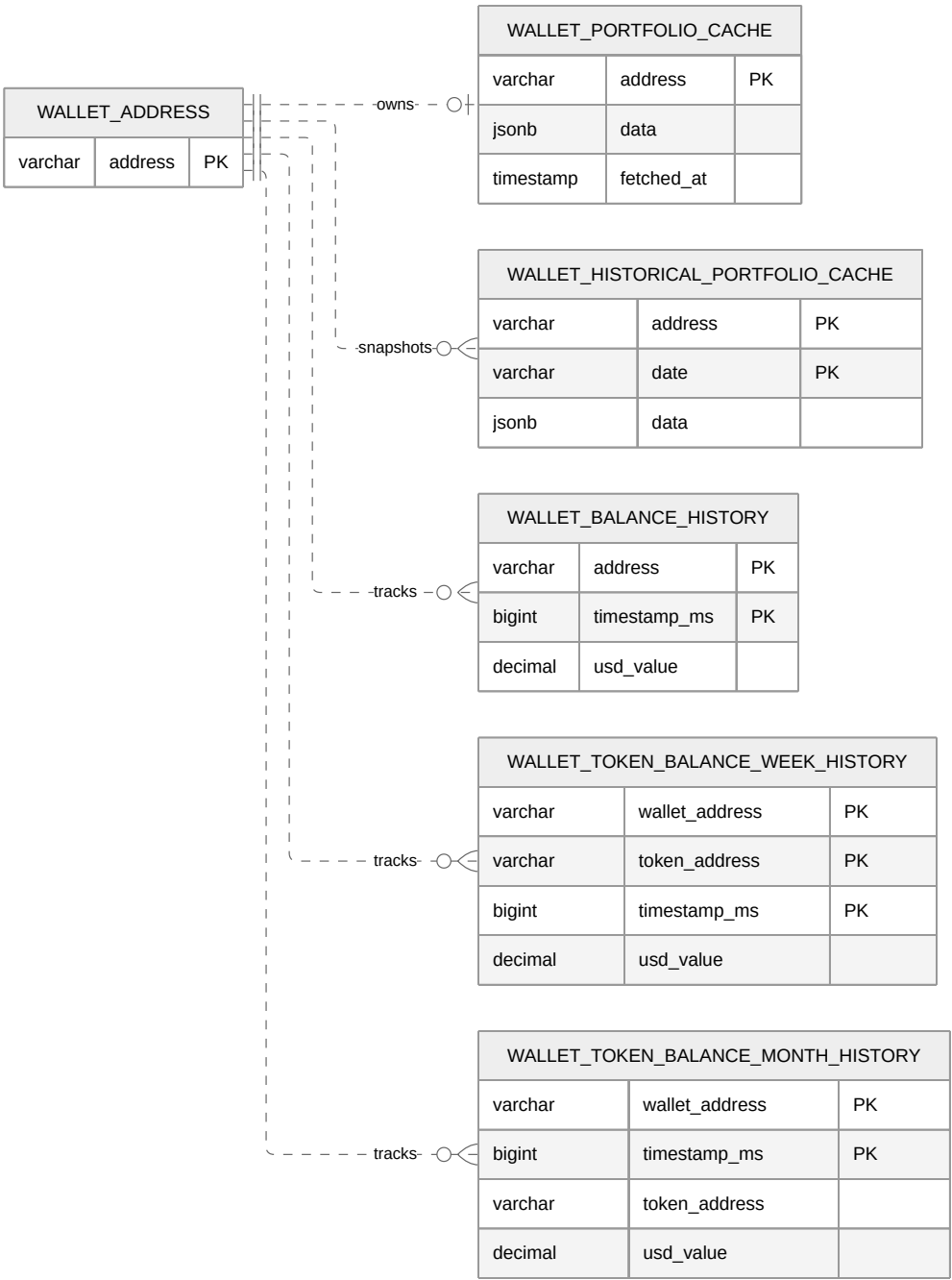
Hình 4.6 mô tả các cache phục vụ AI, news event và phần giải thích biến động. News batch có quan hệ khóa ngoại trực tiếp với article; các cache còn lại được ghép với token theo address.



Hình 4.7: ERD nhóm tổng quan và phân tích ví

Hình 4.7 dùng **WALLET_ADDRESS** như một thực thể logic, không phải bảng vật lý. **wallet_overview_cache** cung cấp bản tổng hợp cho trang overview; **wallet_analyses** lưu kết quả Mobula theo ví và kỳ phân tích; **wallet_token_details** giữ PnL ở cấp token. Identity, nguồn cấp vốn đầu

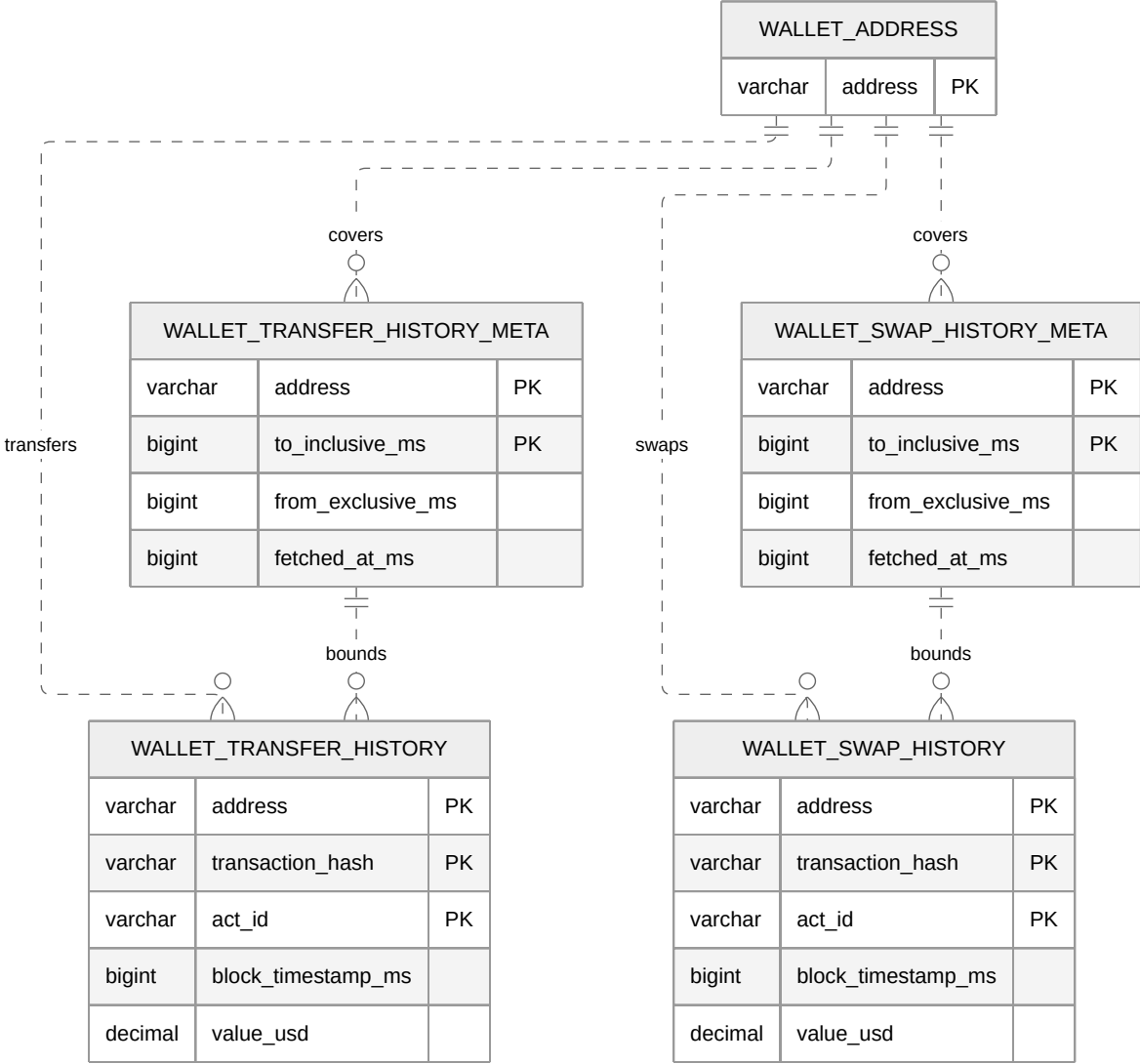
tiền và kết quả AI được tách thành cache riêng vì có nguồn, chi phí và TTL khác nhau.



Hình 4.8: ERD nhóm portfolio và lịch sử số dư

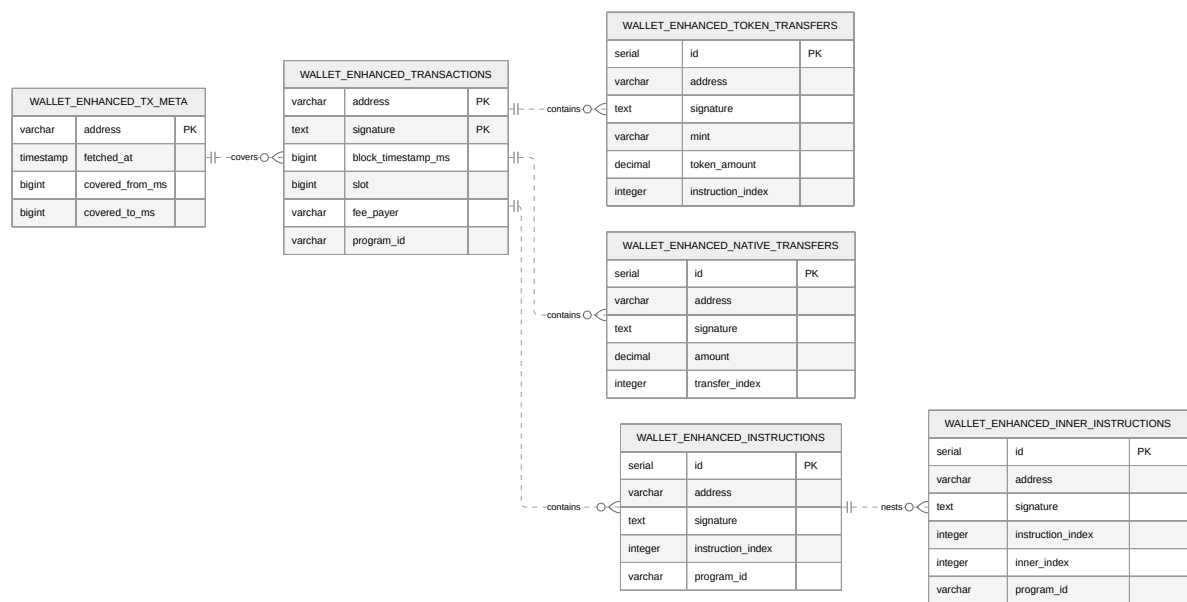
Hình 4.8 phân biệt portfolio hiện tại, snapshot theo ngày, tổng giá trị ví theo thời gian và lịch sử số dư ở cấp token. Tổng tài sản hiện ưu tiên

Mobula do chuỗi Zerion từng ghi nhận cả token spam có giá trị thanh khoản không phù hợp; lịch sử theo từng token vẫn dùng Zerion. Các bảng được ghép theo wallet address nhưng không phụ thuộc một bảng wallet vật lý.



Hình 4.9: ERD nhóm lịch sử transfer và swap

Hình 4.9 tách dữ liệu lịch sử đã chuẩn hóa khỏi metadata coverage. Các bảng này phục vụ truy vấn phân trang và chỉ tải bổ sung những khoảng thời gian còn thiếu.



Hình 4.10: ERD nhóm chi tiết giao dịch Helius

Hình 4.10 mô tả read model chi tiết giao dịch. Bảng transaction cha được ghép với token transfer, native transfer và instruction bằng address cùng signature. Nhóm này phục vụ day activity, transaction detail và wallet analysis; nó không bị thay thế bởi transfer/swap history.

4.4.4 Các bảng trọng tâm

Bảng 4.3 tóm tắt các bảng đại diện cho từng miền. Các bảng cache phụ có cùng cách tổ chức không được liệt kê lại để tránh lặp nội dung.

Bảng 4.3: Các bảng dữ liệu trọng tâm của Yoca

Bảng	Khóa chính	Vai trò
users	id	Thông tin tài khoản, cấu hình nhận cảnh báo và Stripe customer.
auth_accounts	provider,provider_user_id	Danh tính đăng nhập bằng password, OAuth hoặc ví Solana.

Bảng	Khóa chính	Vai trò
subscriptions	id	Trạng thái gói dịch vụ và chu kỳ thanh toán.
alerts	id	Định nghĩa cảnh báo, loại cảnh báo, chế độ kích hoạt và thời hạn.
alert_history	id	Lịch sử cảnh báo đã phát sinh và trạng thái đã đọc.
token_meta	address	Tên, symbol và hình ảnh cơ bản của token.
token_market_data	address	Market snapshot gần nhất, gồm giá, vốn hóa và volume.
token_market_chart_*	address,timestamp	Chuỗi giá 24 giờ, hourly và daily cho biểu đồ.
wallet_overview_cache	address	Read model tổng hợp holdings và hoạt động theo nhiều kỳ cho trang overview.
wallet_analyses	wallet_address,period	Kết quả Mobula gồm win rate, PnL, phân phối giao dịch và calendar breakdown.
wallet_balance_history	address,timestamp_ms	Tổng tài sản theo ngày từ Mobula, dùng cho balance chart và PnL theo dòng tiền.
wallet_token_balance_*.history	wallet_address,timestamp_ms (kỳ tuần bổ sung token_addresses)	Giá trị lịch sử ở cấp token lấy từ Zerion theo kỳ tuần hoặc tháng.
wallet_transfer_history	address,transaction_hash,act_id	Lịch sử chuyển token đã chuẩn hóa và hỗ trợ phân trang.
wallet_swap_history	address,transaction_hash,act_id	Lịch sử swap đã chuẩn hóa theo ví.
wallet_enhanced_transactions	address,signature	Transaction cha cho dữ liệu chi tiết từ Helius.

Bảng	Khóa chính	Vai trò
wallet_token_details	address, token_address	Read model PnL, số lần mua bán và giá vốn theo từng token.
chat_sessions	id	Phiên chat, context ví và danh sách message.
news_batches	id	Nhóm bài viết thu thập theo token và thời điểm.

wallet_analyses là read model có kích thước tương đối lớn do lưu cả chỉ số tổng hợp và hai cấu trúc JSONB là period_timeframes và calendar_breakdown. Mỗi ví chỉ có một bản ghi cho mỗi kỳ phân tích, nhờ đó dữ liệu Mobula có thể được dùng lại trong thời gian TTL mà không cần gọi provider cho mọi request. Việc gom dữ liệu ở bảng này là một ngoại lệ có chủ đích sau đợt migration gộp sang Mobula, không phải mẫu chuẩn hóa mà nhóm áp dụng cho mọi miền.

Các bảng wallet_enhanced_* lưu chi tiết giao dịch ở mức thấp hơn transfer/swap history. Instruction, inner instruction, fee payer, program ID và native transfer cần thiết cho transaction detail và phân tích hành vi, trong khi history tables ưu tiên danh sách đã chuẩn hóa và phân trang. Qua kiểm tra consumer, cả hai nhóm vẫn đang được sử dụng; vì vậy báo cáo xem chúng là các read model song song trong quá trình thay thế tăng dần, không gắn nhãn deprecated chỉ dựa vào tuổi của bảng.

Bên cạnh primary key, một số bảng truy vấn nhiều còn có index bổ sung bám theo truy vấn thực tế thay vì đánh index toàn bộ cột. wallet_transfer_history và wallet_swap_history có index kết hợp (address, block_timestamp_ms) vì truy vấn phổ biến nhất là lấy lịch sử của một ví theo thứ tự thời gian. wallet_enhanced_token_transfers và wallet_enhanced_native_transfers dùng unique index trên (address, signature, ... index) để việc ghi lại dữ liệu từ Helius không tạo bản ghi trùng khi service chạy lại. chat_sessions và chat_prompts có index trên user_id để liệt kê phiên chat hoặc prompt của một tài khoản không phải quét toàn

bảng. Báo cáo không đưa ước lượng dung lượng hay chiến lược partition vì nhóm chưa đo kích thước bảng ở quy mô sản xuất; các index nêu trên phản ánh đúng mẫu truy vấn hiện có trong schema, không phải một danh sách đầy đủ.

4.4.5 Cache và quản lý độ mới

Cache được kiểm soát bằng ba cơ chế. Cơ chế thứ nhất là TTL dựa trên thời điểm cập nhật. Dữ liệu overview, market hoặc kết quả phân tích được trả trực tiếp khi vẫn còn mới; khi hết TTL, service gọi provider và cập nhật lại bản ghi. TTL phù hợp với snapshot hoặc kết quả có thể thay thế toàn phần, nhưng không đủ để chứng minh một khoảng lịch sử đã được tải đầy đủ.

Cơ chế thứ hai là coverage metadata. Với lịch sử transfer, swap và enhanced transaction, metadata lưu biên thời gian đã đồng bộ. Service so sánh khoảng yêu cầu với các đoạn đã có, chỉ tải phần còn thiếu rồi hợp nhất coverage. Đối với chuỗi tổng tài sản Mobula, service kiểm tra điểm đầu, điểm cuối, số ngày và thời điểm cập nhật; nếu phần đầu đã đủ nhưng đuôi đã cũ, nó chỉ tải lại đoạn cuối có một khoảng chồng lấn. Cách này phù hợp hơn việc tải lại toàn bộ lịch sử của một ví có tần suất giao dịch cao.

Cơ chế thứ ba là upsert. Market snapshot, chart point và các cache theo khóa nghiệp vụ sử dụng insert kết hợp conflict handling. Bản ghi mới được thêm khi khóa chưa tồn tại; dữ liệu cũ được cập nhật khi cùng address, timestamp hoặc signature. Đây cũng là cơ sở để tận dụng thông tin dư trong một response: nếu provider trả kèm metadata đã biết, service có thể bổ sung bảng liên quan thay vì bỏ dữ liệu và gọi lại một endpoint khác. Nhóm chỉ áp dụng cách cập nhật chéo này ở các luồng đã kiểm soát được khóa và nullability, không xem nó như một quy tắc bắt buộc cho mọi service.

Thiết kế bảng còn chịu ảnh hưởng trực tiếp từ nhịp cập nhật của API.

Nhóm chúng em tránh gom một dòng dữ liệu mà chỉ có thể hoàn thiện sau hai hoặc ba lần gọi provider. Metadata, market snapshot, chart, pool và holder được tách vì chúng có nguồn và chu kỳ làm mới khác nhau. Sự tách nhỏ này có thể làm tăng số bảng, nhưng giúp một trường đã cũ không buộc hệ thống làm mới toàn bộ đối tượng.

4.4.6 Toàn vẹn và bảo mật dữ liệu

Primary key tổng hợp được dùng cho dữ liệu chuỗi thời gian và lịch sử, ví dụ address kết hợp timestamp hoặc transaction hash. Unique constraint ngăn một transaction detail hoặc chart point bị ghi nhiều lần. Foreign key với quy tắc cascade áp dụng cho dữ liệu thuộc người dùng như auth account, watchlist, alert và subscription. Với dữ liệu provider, chỉ address hoặc signature là chắc chắn trong nhiều trường hợp; symbol, logo và các phần trăm biến động có thể thiếu. Các cột này vì vậy cần cho phép rỗng khi response thực tế không bảo đảm giá trị, thay vì đặt NOT NULL chỉ dựa trên ví dụ trong tài liệu API.

Không phải mọi liên hệ logic đều được chuyển thành foreign key. Wallet address và token address đến từ blockchain, có thể xuất hiện trước khi metadata tương ứng được tải. Việc bắt buộc foreign key trong trường hợp này sẽ làm gián đoạn luồng đồng bộ. Service chịu trách nhiệm chuẩn hóa address và giữ quy tắc ghép dữ liệu nhất quán.

Dữ liệu công khai từ blockchain được tách khỏi dữ liệu cá nhân. Các truy vấn profile, watchlist, cảnh báo, chat và thanh toán phải được giới hạn theo user ID đã xác thực. Password chỉ được lưu dưới dạng hash; API key và secret của provider không được lưu trong các bảng nghiệp vụ.

4.4.7 Tổng kết thiết kế cơ sở dữ liệu

Thiết kế cơ sở dữ liệu kết hợp dữ liệu quan hệ với các read model cache theo miền nghiệp vụ. Các bảng user, payment và alert ưu tiên ràng buộc quan hệ. Các bảng token và wallet ưu tiên khả năng đồng bộ, truy vấn

theo address và tái sử dụng dữ liệu provider. Việc thay đổi schema được thực hiện theo hướng thay thế tăng dần: tạo read model mới, chuyển từng consumer sang luồng mới, theo dõi ổn định rồi mới loại bỏ cấu trúc cũ. Vì quá trình này chưa kết thúc ở mọi miền, báo cáo giữ lại các mô hình song song đang có consumer thay vì trình bày một schema lý tưởng nhưng không khớp hệ thống.

Các bảng trọng tâm được trình bày trực tiếp trong từng lát cắt ERD thay vì liệt kê lại thành một danh mục dài. Có thể quy chúng về bốn nhóm: tài khoản–cảnh báo, token–thị trường, phân tích ví và nội dung AI. Trong đó, `wallet_analyses` là một ngoại lệ có chủ đích: bảng lưu kết quả tổng hợp từ Mobula theo ví và kỳ phân tích để tránh lặp lại các lời gọi tốn kém. Các bảng lịch sử transfer/swap và chi tiết giao dịch Helius vẫn được duy trì song song vì phục vụ hai nhu cầu khác nhau, lần lượt là truy vấn danh sách đã chuẩn hóa và xem dữ liệu giao dịch ở mức chi tiết.

4.5 Lựa chọn công nghệ và giới hạn của thiết kế

React được chọn vì mô hình thành phần phù hợp với giao diện có nhiều bảng, biểu đồ và trạng thái tương tác dùng lại; Vite cung cấp máy chủ phát triển và cơ chế cập nhật mô-đun nhanh [16, 24]. ECharts đảm nhiệm phần lớn biểu đồ tài chính [1], trong khi các cấu trúc quan hệ giao dịch cần một thành phần đồ thị tương tác riêng. Ở backend, Drizzle được chọn vì cách khai báo lược đồ gần với SQL và cho phép nhóm kiểm soát trực tiếp truy vấn, điều quan trọng khi cache và dữ liệu lịch sử cần các thao tác upsert hoặc truy vấn theo khoảng [9].

Những lựa chọn trên ưu tiên tốc độ phát triển và khả năng nhìn thấy lỗi sớm, nhưng không tự động tạo ra một hệ thống hoàn toàn nhất quán. Hono RPC chỉ bảo vệ hợp đồng giữa client và server; Zod bảo vệ dữ liệu ở runtime; ràng buộc cơ sở dữ liệu bảo vệ trạng thái đã lưu; còn kiểm thử

vẫn cần thiết để xác nhận hành vi kết hợp giữa các lớp. Việc phân biệt các lớp bảo vệ này giúp nhóm không xem “strongly typed” như một sự thay thế cho kiểm thử.

Ở phía giao diện, Carbon Design System từng được chọn làm nền tảng chung, nhưng việc hiện thực không được duy trì đồng đều trong suốt quá trình phát triển. Một số màn hình mới kết hợp thành phần tự xây dựng, SCSS và utility class, trong khi một số bảng cũ vẫn giữ cấu trúc Carbon. Nhóm chúng em chấp nhận trình bày đây là hạn chế về tính nhất quán và khả năng bảo trì, thay vì mô tả giao diện như một design system đã hoàn thiện. Trong đợt hoàn thiện cuối, nhóm ưu tiên đồng nhất các luồng chính; những bảng cũ chưa chuyển đổi hoàn toàn được ghi nhận rõ như một giới hạn của phiên bản hiện tại.

Localization được tổ chức thành một lớp dùng chung độc lập với từng trang. Locale tiếng Anh đóng vai trò cấu trúc gốc; locale tiếng Việt phải giữ cùng hệ thống key và biến nội suy. Kiểu TypeScript được dùng để phát hiện key thiếu, key sai và biến interpolation không tương thích trong lúc phát triển. Hook `useLocalization` cung cấp đồng thời hàm dịch và formatter cho số, phần trăm, token unit, ngày giờ và tiền tệ. Các formatter nhận giá trị tài chính gốc bằng USD; với locale Việt Nam, lớp hiển thị áp dụng tỷ giá USD–VND rồi định dạng theo quy ước VND. Cách tách này giữ phép tính và API ở một đơn vị thống nhất, đồng thời tránh mỗi component tự quy đổi theo một công thức khác nhau.

4.6 Ranh giới tin cậy và các điểm cần bảo vệ

Yoca tiếp nhận dữ liệu và yêu cầu từ nhiều phía không đáng tin cậy như nhau: người dùng, ví Solana, webhook của provider và mô hình AI. Đăng nhập bằng ví không ký giao dịch trên chuỗi mà yêu cầu người dùng ký một thông điệp chứa nonce dùng một lần; server xác minh chữ ký ed25519

khớp với public key trước khi cấp phiên đăng nhập, sau đó xóa nonce để tránh dùng lại. Với thanh toán, webhook Stripe được xác thực bằng chữ ký ký số của Stripe trước khi xử lý sự kiện, còn giao dịch thanh toán bằng Solana được backend tự truy vấn qua RPC để đối chiếu địa chỉ nhận, số lượng và trạng thái xác nhận, thay vì tin vào thông tin do client gửi lên.

Webhook nhận sự kiện ví từ Helius hiện được bảo vệ bằng một secret cố định so sánh trên header **Authorization**, chưa phải xác minh chữ ký trên toàn bộ payload. Đây là một giới hạn bảo mật nhóm chúng em ghi nhận rõ thay vì trình bày như một cơ chế đã hoàn thiện; hướng nâng cấp hợp lý là chuyển sang xác minh chữ ký payload nếu Helius hỗ trợ hoặc bổ sung kiểm tra IP/replay.

Đối với AI, ngữ cảnh gửi đến mô hình được xây dựng thành các kiểu dữ liệu riêng cho từng miền (ví dụ **TokenAiContext**), và phần nội dung không đáng tin cậy như lịch sử hội thoại hay dữ liệu tool được bọc trong dấu phân định rõ ràng kèm chỉ dẫn chỉ tuân theo hướng dẫn hệ thống, không thực thi chỉ dẫn nằm trong phần dữ liệu đó. Phản hồi của mô hình sau đó được kiểm tra và chuẩn hóa lại cấu trúc trước khi trả về client. Đây là các lớp giảm thiểu rủi ro prompt injection ở mức thực dụng, không phải một cơ chế phòng thủ đã được kiểm chứng độc lập. Dữ liệu cá nhân như watchlist, nhãn ví, cảnh báo và lịch sử thanh toán tiếp tục được giới hạn theo user ID đã xác thực như đã trình bày ở phần cơ sở dữ liệu.

4.7 Kết chương

Chương 4 đã xác định Yoca là một monolith phân lớp theo miền, có hợp đồng kiểu xuyên suốt giữa client và server, biên kiểm tra runtime cho dữ liệu bên ngoài, cùng cơ chế lưu trữ ưu tiên tái sử dụng dữ liệu đã đồng bộ. Kiến trúc này phản ánh sự cân bằng giữa lược đồ chuẩn hóa, hạn mức API và khả năng phát triển của nhóm. Chương cũng chỉ rõ những ranh giới tin cậy đã được kiểm chứng trong mã nguồn cùng các điểm còn là giới hạn, thay vì trình bày hệ thống như đã phòng thủ toàn diện. Trên nền

thiết kế đó, Chương 5 trình bày kết quả triển khai, những khó khăn đã gặp và mức độ đáp ứng của hệ thống.

Chương 5

Triển khai thực nghiệm và Đánh giá

Chương này trình bày quá trình triển khai thực nghiệm hệ thống Yoca dựa trên kiến trúc và các luồng nghiệp vụ đã được thiết kế ở Chương 4, bao gồm môi trường triển khai, kết quả hiện thực các chức năng cốt lõi, những thách thức kỹ thuật và giải pháp được áp dụng. Đồng thời, chương này ghi nhận kết quả kiểm thử và những giới hạn còn tồn tại về khả năng vận hành, hiệu năng và độ ổn định.

5.1 Môi trường và công cụ triển khai

5.1.1 Tổ chức môi trường triển khai

Yoca được triển khai theo mô hình ứng dụng web full-stack, gồm phần giao diện và máy chủ API được tổ chức trong cùng một monorepo. Hai thành phần cùng sử dụng TypeScript và chia sẻ các kiểu dữ liệu cần thiết, qua đó duy trì sự thống nhất trong quá trình trao đổi dữ liệu và phát triển chức năng.

Phần giao diện được xây dựng bằng React 19 và Vite 7, đảm nhiệm việc trình bày dữ liệu và xử lý tương tác của người dùng. Phần máy chủ

vận hành trên Node.js với Hono, chịu trách nhiệm xác thực, xử lý nghiệp vụ, chuẩn hóa dữ liệu và kết nối với các dịch vụ bên ngoài. PostgreSQL được sử dụng làm cơ sở dữ liệu chính, kết hợp với Drizzle ORM để quản lý lược đồ, truy vấn và lưu các cache bền vững cần tái sử dụng giữa nhiều yêu cầu.

Các thành phần trực quan hóa được xây dựng chủ yếu bằng ECharts; riêng dữ liệu có cấu trúc mạng lưới như luồng giao dịch và quan hệ giữa các ví được biểu diễn dưới dạng đồ thị tương tác. Hệ thống cũng hỗ trợ kết xuất dữ liệu và kết quả phân tích thành PDF, bảng tính và hình ảnh để phục vụ việc lưu trữ, đối chiếu và lập báo cáo.

5.1.2 Tích hợp nguồn dữ liệu

Yoca tổng hợp dữ liệu theo miền nghiệp vụ thay vì tìm một provider thay thế cho mọi chức năng. CoinGecko và nhóm API on-chain của họ đảm nhiệm phần lớn metadata, market, pool và lịch sử giá; Birdeye còn được sử dụng ở một số luồng market, recent trade và lịch sử. Đối với Wallet, Mobula cung cấp wallet analysis, activity và lịch sử tổng tài sản; Zerion cung cấp lịch sử ở cấp token; Helius tiếp tục phục vụ portfolio, identity, funded-by, webhook và transaction detail. Sự phân công này phản ánh trạng thái source hiện tại, đồng thời cho phép thay một miền mà không làm giao diện phụ thuộc trực tiếp vào response riêng của provider.

Tin tức được thu thập từ các luồng nội dung rồi chuẩn hóa và loại trùng trước khi lưu. Gemini được tích hợp cho các chức năng diễn giải dữ liệu, còn Stripe xử lý thanh toán thẻ. Giao dịch thanh toán bằng Solana được backend xác minh độc lập qua RPC trước khi ghi nhận quyền sử dụng.

Cấu hình hệ thống được tách thành hai nhóm rõ ràng. Các giá trị công khai như địa chỉ máy chủ API, khóa công khai Stripe và cấu hình mạng Solana được cung cấp cho giao diện thông qua biến môi trường Vite (tiền tố `VITE_`). Các giá trị nhạy cảm như khóa API nhà cung cấp, chuỗi kết nối cơ sở dữ liệu, khóa ký phiên đăng nhập và khóa bí mật Stripe chỉ được

nạp ở phía máy chủ và không bao giờ được gửi về giao diện.

5.2 Quy trình tích hợp và triển khai hệ thống

5.2.1 Quản lý mã nguồn và môi trường

Mã nguồn client và server được quản lý trong cùng một Git repository dưới dạng npm workspace. Cấu trúc này cho phép nhóm phát triển hai phần độc lập nhưng vẫn dùng chung phiên bản TypeScript và hợp đồng kiểu Hono. Các thay đổi về route hoặc schema thường cần được đối chiếu với consumer tương ứng trước khi tích hợp, đặc biệt ở các miền Wallet và Payment có nhiều phụ thuộc.

Cấu hình runtime được truyền qua biến môi trường. Client chỉ nhận các giá trị có tiền tố `VITE_` và được phép xuất hiện trong browser; khóa provider, chuỗi kết nối PostgreSQL, JWT secret và Stripe secret chỉ thuộc server. Repository cung cấp script riêng cho dev, test, lint, build và đồng bộ schema Drizzle. Trong phạm vi báo cáo, nhóm không đưa nội dung tệp `.env` hoặc giá trị secret vào phụ lục hay ảnh minh họa.

5.2.2 Tích hợp liên tục bằng GitHub Actions

Workflow CI được đặt tại `.github/workflows/ci.yml` và chạy khi có thay đổi được đẩy lên nhánh `main`, `dev` hoặc khi pull request hướng vào `main`. Pipeline gồm sáu job kiểm tra độc lập: `typecheck`, `lint`, `test server`, `test client`, `build server` và `build client`. Mỗi job checkout cùng revision, cài dependency theo npm workspace rồi thực hiện script tương ứng. Cách tách này giúp nhóm nhìn rõ lỗi thuộc hợp đồng kiểu, quy tắc mã nguồn, regression test hay production build, thay vì gộp mọi bước vào một lệnh khó chẩn đoán.

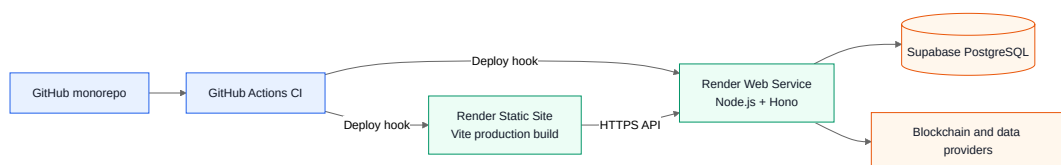
Các job test chạy Vitest trong từng workspace; build client tạo static

artifact bằng Vite, còn build server biên dịch TypeScript và xử lý path alias trước khi Node khởi động. Concurrency được cấu hình theo Git reference để một lần chạy mới có thể hủy lần chạy cũ trên cùng nhánh. Secret provider và deploy hook không được ghi trực tiếp vào workflow; chúng được quản lý bằng GitHub Actions Secrets và biến môi trường của nền tảng triển khai.

Một job thứ bảy, **deploy**, phụ thuộc vào cả sáu job trên (**needs**) và chỉ chạy khi sự kiện là **push** trên nhánh **main**. Job này gọi hai **curl POST** tới deploy hook của Render (một cho Static Site, một cho Web Service) được lưu trong GitHub Actions Secrets, tức bước liên kết CI với Render đã được hiện thực chứ không còn là kiến trúc dự kiến.

5.2.3 Triển khai và kiểm tra sau triển khai

Mã nguồn vẫn được giữ trong một monorepo nhưng được triển khai thành hai đơn vị độc lập trên Render. Client là một Static Site: Render checkout source, chạy **npm install** và **npm run client:build**, sau đó phân phối thư mục **client/build** do Vite tạo ra. Vì vậy production không sử dụng Vite development server. Server là một Web Service: Render chạy **npm run server:build** rồi khởi động **node server/build/main.js**. Cách tổ chức này giữ lợi ích chia sẻ kiểu của monorepo nhưng cho phép client và server có vòng đời build, biến môi trường và khả năng triển khai riêng.

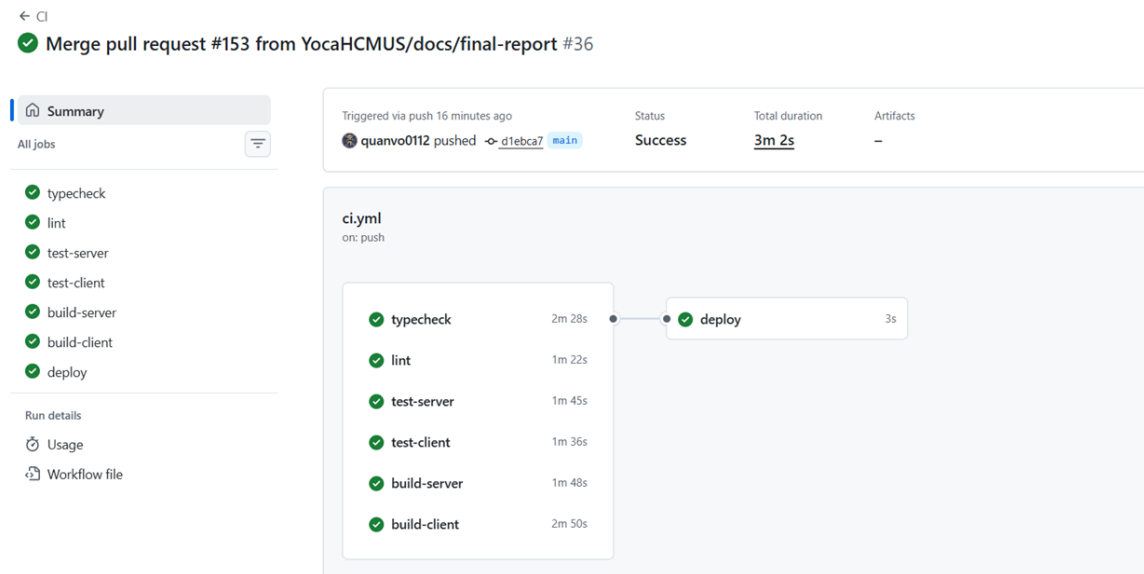


Hình 5.1: Kiến trúc tích hợp và triển khai của Yoca

Sau khi các cổng CI đạt, hai deploy hook bí mật được dùng để yêu cầu Render build revision mới cho Static Site và Web Service. Client chỉ nhận

các biến `VITE_` công khai tại build time; API key, JWT/Stripe secret và chuỗi kết nối database chỉ được cấu hình cho Web Service. PostgreSQL được lưu trên Supabase Free; trong môi trường demo có thể tạo lại, schema Drizzle được đồng bộ bằng quy trình `db:reset` thay vì duy trì dữ liệu cũ qua migration. Thao tác này không được dùng trên database cần bảo toàn dữ liệu.

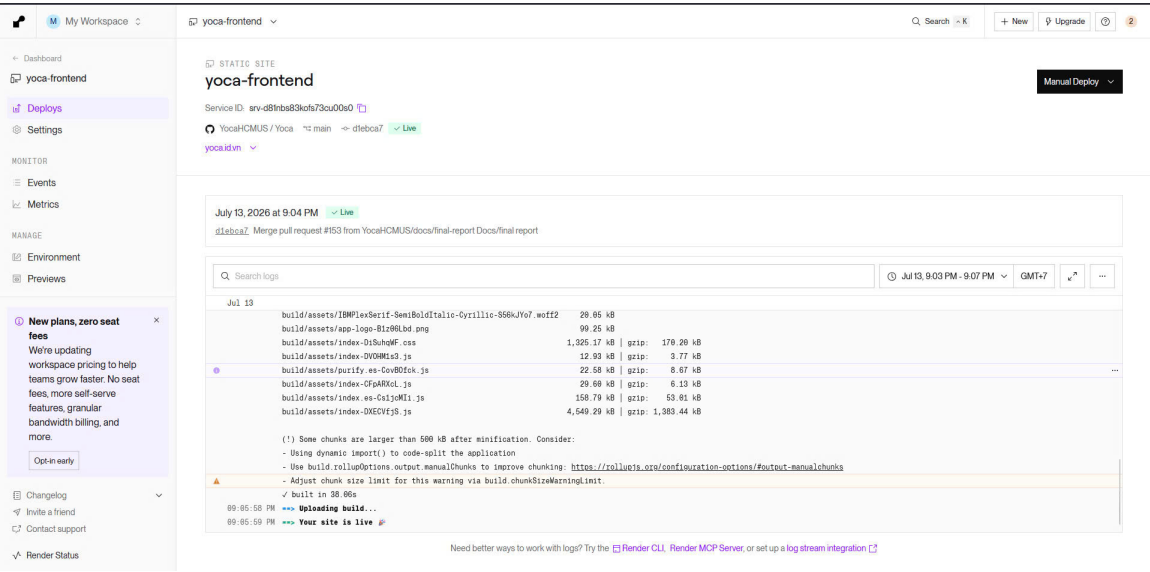
Sau triển khai, nhóm kiểm tra endpoint `/api`, CORS giữa hai domain, SPA rewrite của Static Site và hành trình Market Overview – Token Pool – Token Overview – Wallet – User – AI. Free tier có thể tạo cold start hoặc giới hạn tài nguyên, nên báo cáo không xem thời gian phản hồi của lần gọi đầu tiên là một SLA ổn định và không tuyên bố có rollback hay monitoring tự động ngoài phạm vi Render cung cấp sẵn.



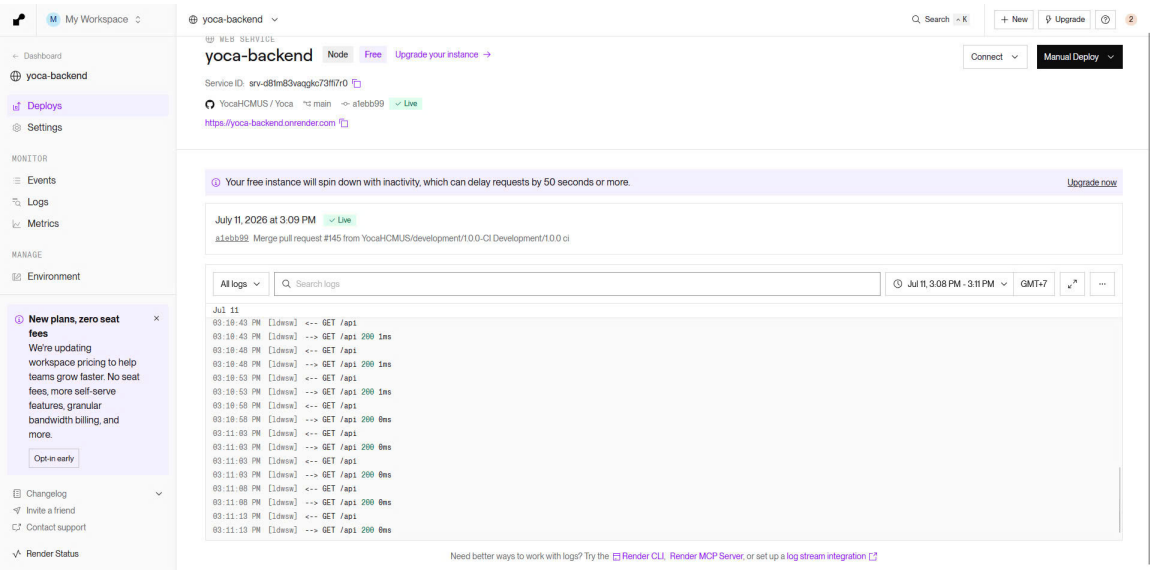
Hình 5.2: Một lần chạy GitHub Actions thành công trên `main`: bảy job `typecheck`, `lint`, `test server/client`, `build server/client` và `deploy` đều đạt, tổng thời gian 3m2s

Job `deploy` gọi hai deploy hook của Render; Hình 5.3 và Hình 5.4 xác nhận cả hai service đã build xong và đang ở trạng thái Live. Yoca hiện chạy thật trên tên miền `yoca.id.vn`: Static Site `yoca-frontend` phục vụ giao diện tại địa chỉ này, Web Service `yoca-backend` phục vụ API tại

yoca-backend.onrender.com và đã ghi nhận các request GET /api trả về 200 trong log thời gian thực.



Hình 5.3: Render Static Site yoca-frontend ở trạng thái Live, phục vụ tại tên miền yoca.id.vn



Hình 5.4: Render Web Service yoca-backend ở trạng thái Live tại yoca-backend.onrender.com, log cho thấy các request GET /api thật trả về 200

5.3 Kết quả triển khai các chức năng cốt lõi

Các chức năng cốt lõi của Yoca đã được kết nối thành một chuỗi sử dụng liên tục. Người dùng có thể bắt đầu từ tìm kiếm hoặc tổng quan thị trường, đi sâu vào token và pool, phân tích ví và giao dịch, sau đó lưu đối tượng quan tâm hoặc thiết lập cảnh báo.

5.3.1 Xác thực tài khoản

Yoca hỗ trợ ba phương thức xác thực gồm đăng nhập bằng email, đăng nhập bằng Google và xác thực thông qua ví Solana, đáp ứng nhu cầu của nhiều nhóm người dùng trong hệ sinh thái blockchain.

Với phương thức email, người dùng có thể đăng ký tài khoản và đăng nhập bằng địa chỉ email cùng mật khẩu. Hệ thống cũng hỗ trợ luồng quên mật khẩu, cho phép người dùng đặt lại mật khẩu thông qua mã xác nhận được gửi qua email.

Đối với đăng nhập bằng Google, người dùng có thể xác thực nhanh mà không cần tạo tài khoản riêng.

Với xác thực bằng ví Solana, người dùng ký một thông điệp xác thực để chứng minh quyền sở hữu ví. Quá trình này không thực hiện giao dịch trên blockchain và chỉ được sử dụng để thiết lập phiên đăng nhập.

Giao diện xác thực được thiết kế dưới dạng hộp thoại, cho phép chuyển đổi linh hoạt giữa đăng nhập và đăng ký, đồng thời hỗ trợ hiển thị thông báo lỗi và tương thích với cả giao diện sáng và tối.

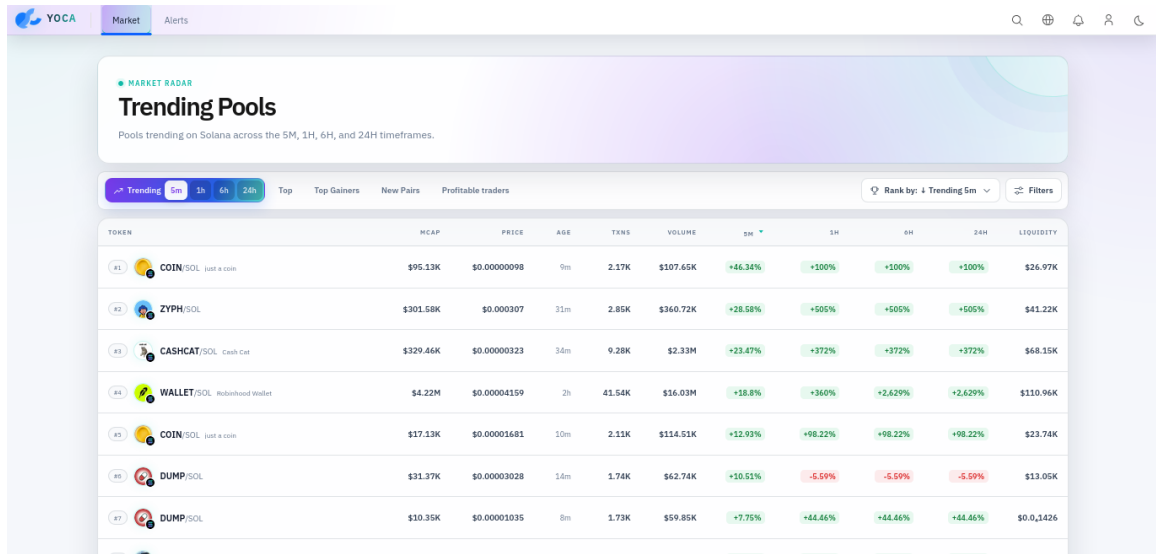
5.3.2 Trang tổng quan thị trường

Trang tổng quan thị trường của Yoca được thiết kế xoay quanh hoạt động giao dịch thực tế thay vì chỉ xếp hạng token theo vốn hóa, nhằm phản ánh trung thực hơn những gì đang diễn ra trên các sàn giao dịch phi tập trung. Giao diện tổ chức dữ liệu thành sáu nhóm có thể chuyển đổi độc lập:

- Trending: Token hoặc pool đang thu hút sự chú ý theo lượng tìm kiếm và hoạt động giao dịch.
- Top Volume/Transactions: Nhóm dẫn đầu theo khối lượng giao dịch hoặc số lần giao dịch.
- Gainers/Losers: Nhóm tăng hoặc giảm giá đáng chú ý trong khoảng thời gian quan sát.
- New Pairs: Các cặp giao dịch mới xuất hiện gần đây trên các sàn phi tập trung.
- Profitable Traders: Những nhà giao dịch đạt lợi nhuận hoặc chịu thua lỗ lớn nhất.

Bộ lọc thời gian cho phép người dùng quan sát dữ liệu Trending theo các cửa sổ 5 phút, 1 giờ, 6 giờ và 24 giờ. Nhóm Profitable Traders hỗ trợ khoảng quan sát 1 ngày, 7 ngày, 30 ngày và 90 ngày. Mỗi dòng dữ liệu hiển thị đầy đủ: tên token hoặc cặp giao dịch, vốn hóa thị trường, định giá pha loãng hoàn toàn (FDV), giá hiện tại, tuổi của pool, số giao dịch 24 giờ, khối lượng 24 giờ, biến động theo 5 phút, 1 giờ, 6 giờ và 24 giờ, cùng thanh khoản hiện có.

Người dùng có thể sắp xếp bảng theo bất kỳ cột nào và áp dụng bộ lọc theo khoảng thanh khoản, vốn hóa, khối lượng, số giao dịch, tuổi pool hoặc mức biến động. Việc chọn một token, pool hoặc địa chỉ nhà giao dịch từ danh sách sẽ điều hướng đến trang phân tích tương ứng, đảm bảo trang tổng quan không chỉ là nơi quan sát mà còn là điểm khởi đầu của quá trình nghiên cứu sâu hơn.



Hình 5.5: Market Overview: nhóm Trending, bộ lọc theo khung thời gian và bảng token/pool có thể điều hướng tiếp

5.3.3 Tìm kiếm

Thanh tìm kiếm toàn cục là công cụ điều hướng trung tâm của hệ thống, hỗ trợ tra cứu đồng thời ba loại đối tượng: token, pool giao dịch và địa chỉ ví.

Mỗi kết quả kèm theo thông tin cơ bản tương ứng với từng loại đối tượng: token hiển thị ký hiệu, giá, biến động 24 giờ, vốn hóa thị trường, khối lượng và biểu đồ sparkline 7 ngày; pool hiển thị tên cặp, sàn giao dịch và các chỉ số thanh khoản; ví hiển thị địa chỉ và thông tin nhận diện. Người dùng có thể xem thông tin này ngay trong giao diện tìm kiếm mà không cần mở trang riêng.

Người dùng có thể điều khiển danh sách kết quả bằng bàn phím hoặc chuột. Khi lựa chọn một kết quả, hệ thống tự động điều hướng đến trang tổng quan token, trang chi tiết pool hoặc trang phân tích ví tương ứng.

5.3.4 Trang tổng quan token

Trang tổng quan token cung cấp bức tranh toàn diện về một token, bao gồm thông tin nhận diện, thống kê thị trường, biểu đồ giá và các phân tích chuyên sâu theo nhiều chiều dữ liệu.

Phần đầu trang hiển thị tên, ký hiệu, địa chỉ hợp đồng, ảnh đại diện và các liên kết chính thức khi có. Khu vực thống kê nhanh trình bày giá và biến động, giá cao/thấp trong 24 giờ, thứ hạng vốn hóa, vốn hóa thị trường, FDV, khối lượng giao dịch 24 giờ, ATH và ATL lịch sử.

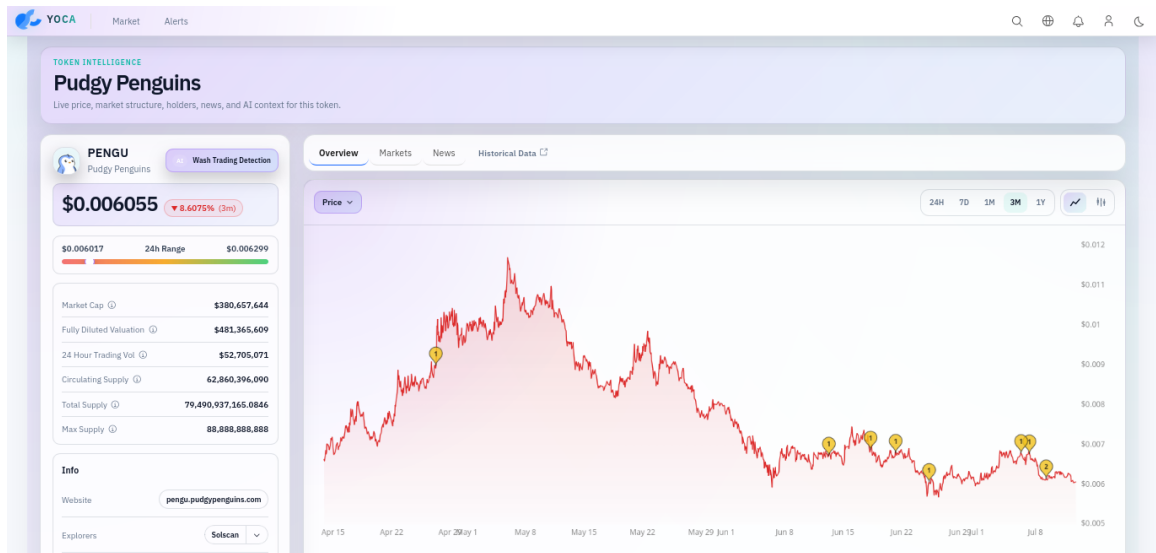
Biểu đồ hỗ trợ ba chế độ: đường giá, đường vốn hóa và nến. Hai chế độ đường hỗ trợ các khoảng 24 giờ, 7 ngày, 30 ngày, 90 ngày và 1 năm. Biểu đồ nến sử dụng pool ưu tiên của token để phản ánh diễn biến giao dịch thực tế. Các sự kiện tin tức được đánh dấu trực tiếp trên trục thời gian; khi chọn một sự kiện, phần tóm lược và danh sách bài viết liên quan hiện ngay phía dưới mà không cần rời khỏi ngữ cảnh biến động giá.

Phần nội dung chuyên sâu được tổ chức thành nhiều thẻ:

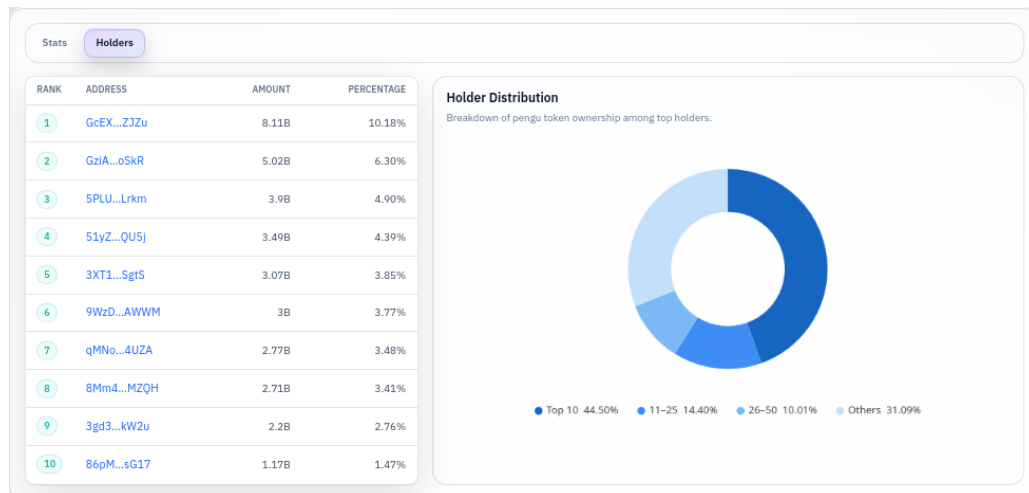
- Overview: giải thích các chỉ số về khối lượng, biến động, vốn hóa, nguồn cung lưu hành, FDV và khoảng cách hiện tại so với ATH/ATL.
- Holders: hiển thị danh sách địa chỉ nắm giữ lớn và biểu đồ phân bố nguồn cung theo bốn lớp: 10 ví lớn nhất, nhóm ví từ vị trí 11 đến 25, nhóm từ vị trí 26 đến 50, và phần còn lại. Cách phân lớp này giúp người dùng đánh giá mức độ tập trung nguồn cung một cách trực quan mà không cần đọc từng địa chỉ riêng lẻ.
- Tokenomics: trình bày cơ cấu phân bổ token, tỷ trọng từng nhóm, lượng token mở khóa tích lũy theo thời gian và các đợt mở khóa sắp tới.
- Investors: liệt kê các tổ chức hoặc cá nhân hậu thuẫn token, bao gồm vai trò, loại hình, quốc gia và mô tả.

- Markets: liệt kê các pool nổi bật theo sàn giao dịch, cặp, giá, biến động 24 giờ, khối lượng và thanh khoản. Lựa chọn một pool sẽ chuyển đến trang chi tiết pool tương ứng.
- News: tổng hợp bài viết từ nhiều nguồn RSS và kết quả tìm kiếm, phân biệt giữa tin tức chính thức và nội dung chỉ đề cập đến token trên web.

Khu vực quy đổi tỷ giá cho phép nhập một giá trị và xem ngay kết quả quy đổi sang nhiều đơn vị tiền tệ khác. Trợ lý AI được tích hợp trực tiếp trong trang, nhận bối cảnh token hiện tại làm đầu vào để hỗ trợ người dùng đặt câu hỏi phân tích.



Hình 5.6: Token Overview: identity, thống kê giá/thị trường và biểu đồ giá của một token



Hình 5.7: Tab Holders: danh sách địa chỉ nắm giữ lớn và biểu đồ phân bố nguồn cung theo bốn lớp

The screenshot shows the 'Ask Yoca AI' interface. A user has asked the question 'Is this token risky?'. The AI has provided a response with a 'Medium confidence' level. The response includes a 'TLDR' section with three points.

Ask Yoca AI
Ask about this token using Yoca market, news, volatility, and on-chain context.

Why is this token moving? What is the latest news? What are the bullish and bearish signals? Is this token risky? Should I buy this token now? What should I watch next?

Explain this token simply.

Is this token risky?

20/500 4/5 questions remaining today (Free)

RISK OVERVIEW
Yoca AI answer for Pudgy Penguins (PENGU) As of Jul 12, 09:11 PM, using Yoca market, news, volatility, and available on-chain context.

Deep mode Gemini gemini-2.5-flash Cache miss

AI TLDR

- PENGU has experienced a price **decline** of nearly 3% in the last 24 hours and over 9% in the last 7 days, indicating recent **bearish** sentiment.
- Holder concentration is high, with the top **10 holders** controlling 44.50% of the supply, which could **increase** price volatility and distribution **risk**.
- The token's mint and freeze authorities are both revoked, which generally reduces certain types of centralized control **risks** for the token contract.

Hình 5.8: Trợ lý AI trả lời câu hỏi về token kèm trạng thái giới hạn lượt hỏi còn lại trong ngày (gói Free)

Ngoài ra, trang token cũng cung cấp mục dữ liệu lịch sử, nơi giá đóng cửa, vốn hóa và khối lượng giao dịch được trình bày theo ngày dưới dạng bảng, sắp xếp mới nhất lên đầu. Người dùng chọn khoảng quan sát từ 7 ngày đến 1 năm và có thể tải thêm dữ liệu theo từng trang, phù hợp khi cần tra cứu giá trị tại một thời điểm cụ thể hoặc lấy số liệu thô cho phân tích bên ngoài.

5.3.5 Trang chi tiết pool

Trang chi tiết pool phục vụ việc phân tích token trong bối cảnh một cặp thanh khoản cụ thể. Trang này được tách biệt với trang tổng quan token nhằm phân biệt các chỉ số của toàn bộ token với các chỉ số chỉ áp dụng cho một pool giao dịch.

Người dùng có thể chuyển giữa các pool nổi bật của cùng một token thông qua bộ chọn pool. Mỗi lựa chọn hiển thị sàn giao dịch (DEX), tên cặp, địa chỉ pool, thanh khoản và khối lượng giao dịch trong 24 giờ, giúp việc so sánh giữa các pool trở nên thuận tiện.

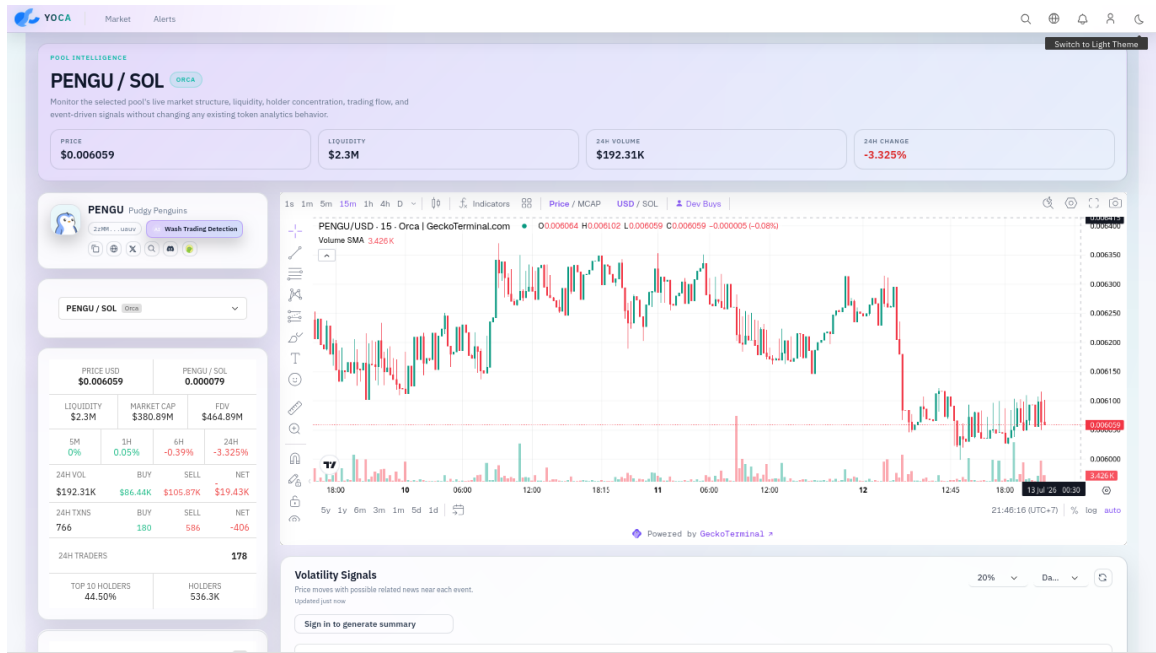
Sau khi lựa chọn pool, hệ thống hiển thị khu vực thống kê với các chỉ số quan trọng như giá token, thanh khoản, vốn hóa, FDV, biến động giá, khối lượng mua và bán, tỷ lệ mua-bán, số lượng giao dịch, số nhà giao dịch, tỷ lệ nắm giữ của các holder lớn và nguồn cung.

Biểu đồ nền phản ánh diễn biến của cặp giao dịch đang được chọn. Đồng thời, khối tín hiệu biến động tổng hợp các thay đổi đáng chú ý trong dữ liệu gần đây, hỗ trợ người dùng nhanh chóng nhận biết các biến động nổi bật của thị trường.

Ngoài thông tin tổng quan, hệ thống còn cung cấp bảng giao dịch gần đây với các lệnh mua và bán, kèm theo số lượng token, giá, giá trị giao dịch, địa chỉ ví và mã giao dịch. Từ mỗi giao dịch, người dùng có thể mở phần chi tiết để theo dõi luồng chuyển tài sản trên blockchain.

Bên cạnh lịch sử giao dịch, tab Bubblemaps hiển thị biểu đồ phân bố holder, giúp trực quan hóa mối quan hệ giữa các ví nắm giữ token mà không cần rời khỏi trang. Điều này cho phép người dùng kết hợp quan sát hoạt động giao dịch với cấu trúc phân bố holder trong cùng một ngữ cảnh.

Cuối cùng, trang tiếp tục tích hợp tin tức liên quan, dữ liệu holder và trợ lý AI, giúp người dùng phân tích hoạt động của pool trong bối cảnh tổng thể của token.



Hình 5.9: Token Pool: pool selector, market metrics và biểu đồ nến của cặp giao dịch đang chọn

5.3.6 Trang phân tích ví

Trang phân tích ví là một trong những chức năng trọng tâm của Yoca, cung cấp cái nhìn toàn diện về tài sản và hoạt động của một địa chỉ ví Solana. Phần đầu trang hiển thị thông tin ví cùng các thao tác như theo dõi, so sánh, chia sẻ, xuất dữ liệu và phân tích bằng AI.

Người dùng có thể lựa chọn khoảng thời gian 7, 30 hoặc 90 ngày để theo dõi các chỉ số tổng quan như tổng giá trị tài sản, số token đang nắm giữ hoặc đã giao dịch, tổng lãi/lỗ, lãi/lỗ đã thực hiện, lãi/lỗ chưa thực hiện, khối lượng giao dịch cùng số lần mua và bán.

Tiếp theo, hệ thống trực quan hóa dữ liệu thông qua biểu đồ số dư và biểu đồ PnL, giúp người dùng theo dõi sự thay đổi giá trị danh mục cũng như hiệu quả giao dịch theo từng giai đoạn.

Bên cạnh các biểu đồ, bảng danh mục tài sản liệt kê những token đang được nắm giữ cùng số dư, giá hiện tại và giá trị quy đổi. Từ đây, người dùng có thể chuyển trực tiếp đến trang tổng quan của từng token để tiếp

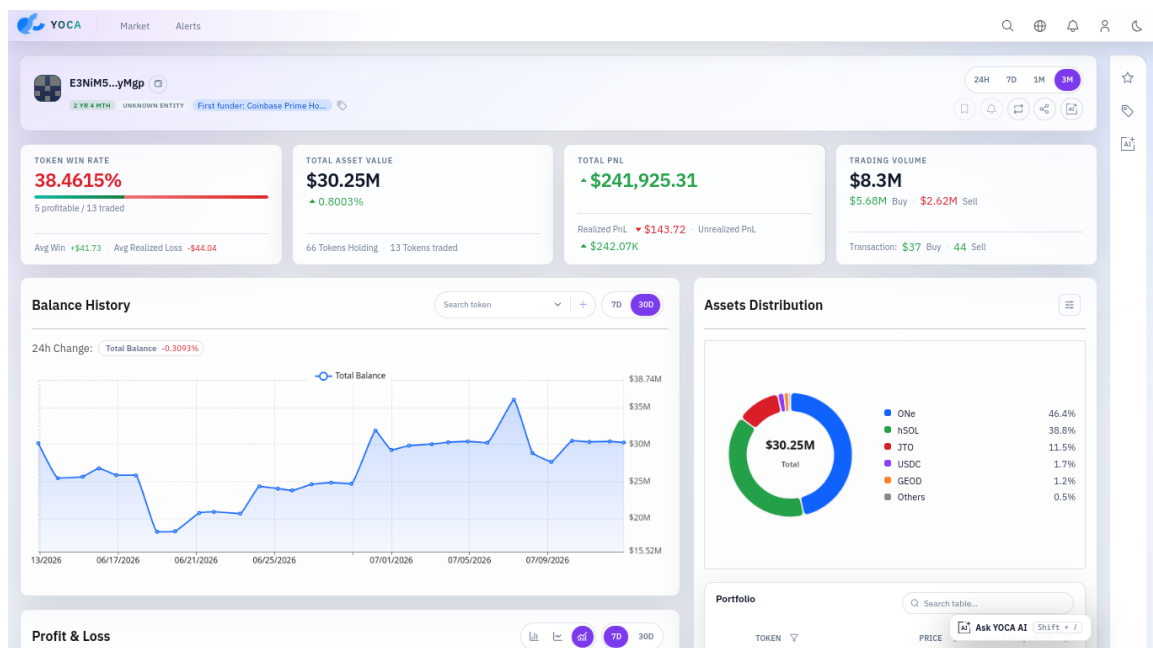
tục phân tích.

Ngoài thông tin tổng quan, hệ thống còn cung cấp phần phân tích chi tiết cho từng vị thế token, bao gồm số dư hiện tại, lãi/lỗ, khối lượng mua và bán, giá trị ròng, số lần giao dịch cùng giá mua và giá bán trung bình. Lịch sử của từng token cũng có thể được xem theo nhiều khoảng thời gian khác nhau.

Hoạt động của ví được chia thành hai nhóm gồm các giao dịch swap và chuyển tài sản. Mỗi giao dịch đều hiển thị đầy đủ thông tin về tài sản, giá trị, thời gian, địa chỉ ví liên quan và mã giao dịch, đồng thời cho phép mở xem chi tiết mà không làm mất trạng thái hiện tại của trang.

Ngoài việc cung cấp dữ liệu giao dịch, hệ thống còn hỗ trợ gán nhãn ví, nhận diện tự động một số đặc trưng hành vi và tích hợp trợ lý AI để diễn giải hoạt động của ví dựa trên dữ liệu đang hiển thị. Kết quả AI đóng vai trò hỗ trợ phân tích và luôn được đặt cạnh các bảng hoặc biểu đồ tương ứng để người dùng dễ dàng đối chiếu với dữ liệu gốc.

Người dùng cũng có thể xuất dữ liệu của ví dưới nhiều định dạng khác nhau nhằm phục vụ nhu cầu lưu trữ hoặc phân tích bên ngoài.



Hình 5.10: Wallet Overview: win rate, tổng tài sản, PnL, khối lượng giao dịch, biểu đồ số dư và phân bổ tài sản của một ví demo

Last traded tokens											
Tokens with recent trading activity											
TOKEN / TIME	BALANCE	PROFIT	REALIZED PROFIT	UNREALIZED PROFIT	TOTAL BOUGHT	TOTAL SOLD	NET VALUE	TRANSACTIONS	AVG BUY/SELL PRICE	GRAPH	
 PYUS... 4d	\$0 0	\$0 0%	\$0 0%	\$0	\$0 0	\$10.63 10.6182	+\$10.63	0 / 1	\$0 \$1		
 USDC 1mo	\$0 0	-\$150.99 -0.0001%	-\$150.99 -0.0001%	\$0	\$1.18M 1.18M	\$1.06M 1.06M	-\$117.58K	6 / 12	\$0.9999 \$0.9998		
 PRIM... 4d	\$3.92 3.7416	+\$0.1 +0.0262%	\$0 0%	+\$0.1	\$7.46M 7.31M	\$0 0	-\$7.46M	5 / 0	\$1.02 \$0		
 WBTC 19d	\$0 0	+\$208.64 +0.0044%	+\$208.64 +0.0044%	\$0	\$104.83K 1.6548	\$47.3K 0.7434	-\$57.53K	4 / 1	\$63.35K \$63.63K		
 ONE 1mo	\$14.01M 12.51M	+\$357.26K +0.0261%	\$0 0%	+\$357.26K	\$2.06M 1.88M	\$0 0	-\$2.06M	37 / 0	\$1.09 \$0		
 GEO 3mo	\$358.69K 1.79M	+\$119.59K +0.4996%	\$0 0%	+\$119.59K	\$39.08K 293.01K	\$0 0	-\$39.08K	39 / 0	\$0.1334 \$0		
 WTF 1d	\$0 0	\$0 0%	\$0 0%	\$0	\$0 0	\$0.1468 3.04K	+\$0.1468	0 / 1	\$0 \$0.04824		
 ANSE... 8d	\$149.72K 627.27K	-\$80.09K -0.3445%	\$0 0%	-\$80.09K	\$232.46K 627.27K	\$0 0	-\$232.46K	5 / 0	\$0.3706 \$0		
 CLAW... 1mo	- 0	\$0 0%	\$0 0%	\$0	\$0 0	\$0.1569 55.56K	+\$0.1569	0 / 1	\$0 \$0.02824		

Hình 5.11: Phân tích chi tiết theo từng vị thế token: số dư, lãi/lỗ đã và chưa thực hiện, khối lượng và giá mua/bán trung bình

5.3.7 Trang so sánh nhiều ví

Trang so sánh cho phép quan sát từ hai đến nhiều địa chỉ ví trong cùng một hệ quy chiếu thời gian và thước đo, giúp nhận diện sự khác biệt về chiến lược, hiệu quả và mức độ rủi ro giữa các ví.

Người dùng thêm ví bằng cách nhập địa chỉ hoặc chọn từ danh sách ví đã theo dõi. Các biểu đồ được tổ chức thành ba nhóm chức năng:

- Nhóm tổng quát: Xu hướng số dư nhiều ví theo thời gian, khối lượng giao dịch tổng, phân bố khối lượng và khối lượng trung bình trên mỗi giao dịch.
- Nhóm tài sản: Phân bố danh mục tài sản theo ví và tỷ lệ stablecoin của từng ví.
- Nhóm rủi ro và hiệu quả: PnL theo cửa sổ thời gian, tỷ lệ giao dịch có lãi và mức suy giảm tài sản tối đa (drawdown).

Mỗi biểu đồ có thể được đưa vào ngữ cảnh của trợ lý AI để đặt câu hỏi trực tiếp về dữ liệu đang hiển thị. Người dùng cũng có thể mở lại trang

phân tích riêng của từng ví hoặc xuất toàn bộ phần so sánh sang định dạng PDF.

5.3.8 Trang cảnh báo và theo dõi ví

Trang cảnh báo yêu cầu người dùng đăng nhập và cung cấp cơ chế theo dõi ví theo thời gian thực thông qua tích hợp webhook Helius.

Quản lý danh sách ví theo dõi: Người dùng thêm địa chỉ ví cần theo dõi và đặt nhãn phân biệt. Khi một ví được thêm hoặc xóa khỏi danh sách, hệ thống tự động đồng bộ cấu hình với Helius Webhook: các địa chỉ được phân phối vào các nhóm webhook (shards) theo giới hạn 25 địa chỉ mỗi shard. Quá trình đồng bộ tạo, cập nhật hoặc xóa shard phù hợp để luôn phản ánh đúng danh sách hiện tại mà không yêu cầu người dùng thao tác thủ công.

Mỗi quy tắc cảnh báo được tạo qua hai bước rõ ràng. Ở bước đầu (cấu hình điều kiện), người dùng chỉ định:

- Ví cần áp dụng quy tắc;
- Loại hoạt động kích hoạt: swap, chuyển tài sản hoặc tất cả;
- Khoảng giá trị tối thiểu và tối đa;
- Đơn vị giá trị: USD hoặc SOL;
- Kiểu kích hoạt: một lần (one-shot) hoặc lặp lại (recurring);
- Thời điểm hết hạn của quy tắc.

Ở bước thứ hai (cấu hình thông báo), người dùng chọn kênh nhận thông báo. Theo mặc định, thông báo được gửi đến email đã đăng ký. Người dùng có thể chỉ định email riêng hoặc URL Discord webhook cho từng quy tắc. Ngoài ra, người dùng đặt tên quy tắc và xem trước nội dung thông báo sẽ nhận được.

ALERTS Create new alert

(1) Trading events — (2) Delivery

Trader (wallet address)
3nMNd89AxxHUs1AFvQGqohRkxFEQsTgEYqKFHyH

Action type
Swap

Volume from: 10 Volume to: 100

Volume unit
USD (server converts SOL via WEBHOOK_SOL_PRICE_USD)

Trigger
☒ Only once ☐ Always

Expires
13/08/2026 09:26 CH

Cancel Next

(a) Bước 1: điều kiện kích hoạt

ALERTS Create new alert

(1) Trading events — (2) Delivery

Use default Discord & email settings
☒ On

Alert name
Trading Event 1

Message (auto)
 Wallet 3nMNd89AxxHUs1AFvQGqohRkxFEQsTgEYqKFHyH on Solana has a swap with value greater than 10\$ and less than 100\$.

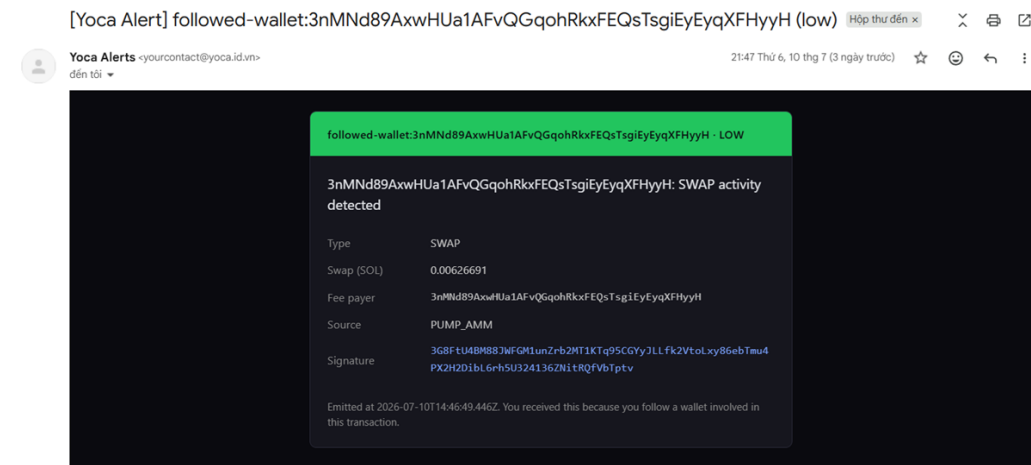
Back Save

(b) Bước 2: kênh nhận và xem trước nội dung

Hình 5.12: Tạo một alert rule thật: theo dõi swap của một ví trong khoảng giá trị 10–100 USD

Danh sách quy tắc hiển thị ví, loại hoạt động, khoảng giá trị, kiểu kích hoạt và thời hạn; từ đó người dùng có thể tắt hoặc xóa quy tắc bất kỳ lúc nào.

Sau khi ít nhất một kênh gửi thành công, hệ thống ghi Alert History theo đúng tài khoản. Mỗi bản ghi giữ nội dung, mức độ, transaction signature, ví liên quan, thời điểm và kết quả email/Discord. Người dùng có thể xem lịch sử theo thứ tự mới nhất, đánh dấu đã đọc/chưa đọc hoặc đánh dấu toàn bộ; khóa idempotency ngăn webhook retry tạo thông báo trùng.



Hình 5.13: Email cảnh báo nhận được thật sau khi rule ở Hình 5.12 kích hoạt: sự kiện SWAP trên ví theo dõi, kèm nguồn, số lượng và transaction signature

Hai hình trên xác nhận luồng end-to-end (tạo rule – Helius phát sinh sự kiện – gửi thành công) đã chạy được trên môi trường thật.

5.3.9 Trang hồ sơ cá nhân

Trang hồ sơ cá nhân là trung tâm quản lý tài sản và tùy chỉnh trải nghiệm, được tổ chức thành các thẻ chức năng liên kết chặt chẽ với nhau.

Bắt đầu từ thẻ Portfolio (Overview), người dùng thiết lập danh sách các ví cần theo dõi. Thẻ này cung cấp cái nhìn nhanh về tổng giá trị của từng ví, cho phép thêm mới hoặc gỡ bỏ các ví không còn nhu cầu giám sát.

Thẻ Dashboard là nơi tổng hợp toàn bộ dữ liệu từ các ví của người dùng thành một bảng điều khiển trung tâm duy nhất. Chức năng này cung cấp bức tranh toàn cảnh về sức khỏe tài chính thông qua các biểu đồ phân tích chuyên sâu được tạo sẵn, như phân bố khối lượng giao dịch, ma trận cơ cấu tài sản, và đối sánh lãi/lỗ (PnL). Nhờ đó, người dùng nắm bắt được hiệu suất của toàn bộ danh mục đầu tư mà không cần thao tác thiết lập phức tạp.

Bên cạnh tài sản cá nhân, người dùng có thể theo dõi thị trường thông qua thẻ Watchlist. Thẻ này hoạt động như một sổ tay lưu trữ các token và

ví tiềm năng đang trong tầm ngắm, giúp dễ dàng mở nhanh để xem biến động hoặc chỉnh sửa nhãn phân loại.

Tiếp theo là thẻ Subscriptions, thẻ này giúp quản lý gói dịch vụ và lịch sử thanh toán minh bạch. Hệ thống hỗ trợ thanh toán qua thẻ ngân hàng hoặc bằng tiền mã hóa. Khi thanh toán bằng Solana, giao dịch chuyển tiền từ ví người dùng sẽ được máy chủ tự động truy xuất và xác minh nghiêm ngặt trên chuỗi khối (địa chỉ nhận, số tiền, trạng thái) trước khi cấp quyền dịch vụ.

Cuối cùng, mọi thiết lập định danh và bảo mật được quản lý tập trung tại thẻ Settings. Tại đây, người dùng có thể tùy chỉnh tên hiển thị, theo dõi và liên kết các phương thức đăng nhập đa dạng (như Email, tài khoản Google hoặc ví Solana). Thẻ này cũng cho phép thay đổi mật khẩu định kỳ hoặc thực hiện quy trình xóa tài khoản an toàn với bước xác nhận bổ sung, đảm bảo người dùng luôn có toàn quyền kiểm soát dữ liệu cá nhân của mình.

5.3.10 Mô-đun phát hiện dấu hiệu wash trading

Mô-đun phát hiện dấu hiệu wash trading hỗ trợ nhận diện các mẫu giao dịch có khả năng tạo khối lượng giả thông qua phân tích quan hệ giữa các ví trên blockchain. Đây là chức năng bổ sung cho các chỉ số thị trường truyền thống, giúp người dùng quan sát hành vi giao dịch ở mức độ sâu hơn.

Người dùng nhập địa chỉ mint của token, ký hiệu token và lựa chọn khoảng thời gian phân tích 24 giờ, 7 ngày hoặc 30 ngày. Từ dữ liệu giao dịch thu thập được, hệ thống xây dựng đồ thị ví-giao dịch và tính toán điểm rủi ro dựa trên nhiều đặc trưng, bao gồm:

- Chu trình giao dịch (circular patterns);
- Tính đều đặn về thời gian (time regularity);
- Mức tương đồng về số lượng giao dịch (amount similarity);

- Giao dịch tự quay lại (self-loop);
- Vai trò của ví trung tâm (hubness);
- Tín hiệu bất thường về khối lượng (volume anomaly).

Sau khi phân tích hoàn tất, hệ thống hiển thị các chỉ số tổng hợp như tổng số giao dịch, số lượng ví tham gia, khối lượng wash trading ước tính, số ví đáng ngờ, số cụm giao dịch vòng và độ tin cậy của kết quả phân tích.

Bên cạnh các chỉ số thống kê, đồ thị ví-giao dịch được sử dụng để trực quan hóa mối quan hệ giữa các ví. Người dùng có thể phóng to, thu nhỏ, di chuyển đồ thị hoặc lựa chọn từng ví để xem các đặc trưng nổi bật và điểm rủi ro tương ứng. Giao diện cho phép chuyển đổi giữa ba bộ trọng số được đặt tên theo GCN, GAT và GraphSAGE để so sánh cách mỗi cấu hình nhấn mạnh khác nhau vào chu trình, tính đều đặn thời gian hay vai trò trung tâm của ví khi tổng hợp điểm rủi ro. Đây là bộ trọng số lấy cảm hứng từ các kiến trúc graph neural network chứ chưa phải mô hình đồ thị đã huấn luyện; nhóm chúng em xem việc thay bằng một GNN được huấn luyện trên dữ liệu gắn nhãn là hướng phát triển tiếp theo của mô-đun này.

Ngoài kết quả phân tích, hệ thống tích hợp trợ lý AI để diễn giải các dấu hiệu được phát hiện dựa trên bối cảnh của đồ thị và các chỉ số đã tính toán. Phần giải thích luôn được hiển thị cùng với dữ liệu nguồn, giúp người dùng dễ dàng đối chiếu và sử dụng kết quả như một tín hiệu hỗ trợ đánh giá, thay vì kết luận độc lập về hành vi gian lận.

5.3.11 Quá trình đồng nhất hóa giao diện

Carbon Design System được nhóm lựa chọn từ giai đoạn đầu với mong muốn có sẵn component, trạng thái tương tác và quy tắc trình bày nhất quán. Tuy nhiên, quyết định này được đưa ra khi navigation và phạm vi từng trang còn chưa ổn định. Bản phác thảo ban đầu vì vậy mới thể hiện bố cục tổng quát, chưa đủ để thống nhất cách bảng, bộ lọc, modal và trạng thái dữ liệu sẽ hoạt động khi sản phẩm phát triển đầy đủ.

Trong quá trình hiện thực, các thành viên tiếp cận giao diện theo những cách khác nhau. Một số phần tiếp tục dùng component Carbon và behavior đi kèm; một số phần được xây mới bằng SCSS module hoặc utility class để nhanh đạt hình thức mong muốn. Việc thay đổi giữa dự án không chỉ làm màu sắc và khoảng cách thiếu đồng nhất mà còn tạo nhiều contract component, loading state, empty state và interaction behavior khác nhau. Đây là lý do một thay đổi nhìn bề ngoài như “làm mới giao diện” lại đòi hỏi migration ở cả cấu trúc component chứ không thể chỉ sửa stylesheet.

Qua kiểm tra source hiện tại, Landing Page, Market Overview và Wallet là các khu vực đã được đổi mới nhiều nhất. Market vẫn tái sử dụng một số primitive Carbon cho tab, loading và bảng, nhưng đã có lớp trình bày riêng bằng SCSS module. Wallet kết hợp component Carbon ở các modal và trạng thái tải với các component phân tích được tổ chức lại theo từng miền. Ở Token Overview và Token Pool, một số bảng và component cũ vẫn giữ style hoặc behavior trước đợt đổi mới. Vì vậy, nhóm chúng em không xem migration giao diện là đã hoàn tất.

Kinh nghiệm rút ra là design system chỉ tạo ra tính nhất quán khi nhóm thống nhất cả component contract, trạng thái loading/error/empty, accessibility và quy trình review. Việc có tên một thư viện trong dependency không đủ để bảo đảm toàn bộ sản phẩm tuân theo thư viện đó. Trong phạm vi còn lại, nhóm ưu tiên giữ luồng sử dụng ổn định, chuyển dần component cũ và tránh tiếp tục thay toàn bộ nền tảng giao diện trong một lần.

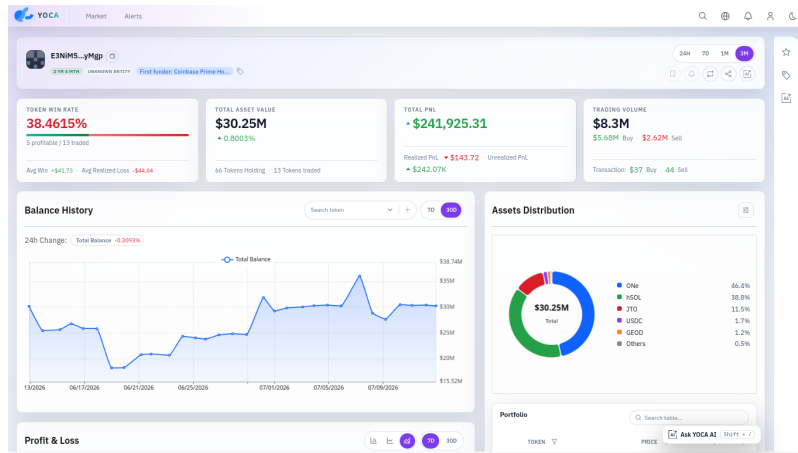
5.3.12 Localization cho người dùng Việt Nam

Yoca hiện cung cấp hai locale tiếng Anh và tiếng Việt thông qua một context dùng chung. Khi người dùng đổi ngôn ngữ, các component sử dụng localization hook được render lại với chuỗi, quy tắc số và mẫu ngày giờ của locale tương ứng. Cùng một giá trị tài chính được hiển thị bằng USD ở locale tiếng Anh và được quy đổi, làm tròn theo VND ở locale tiếng Việt;

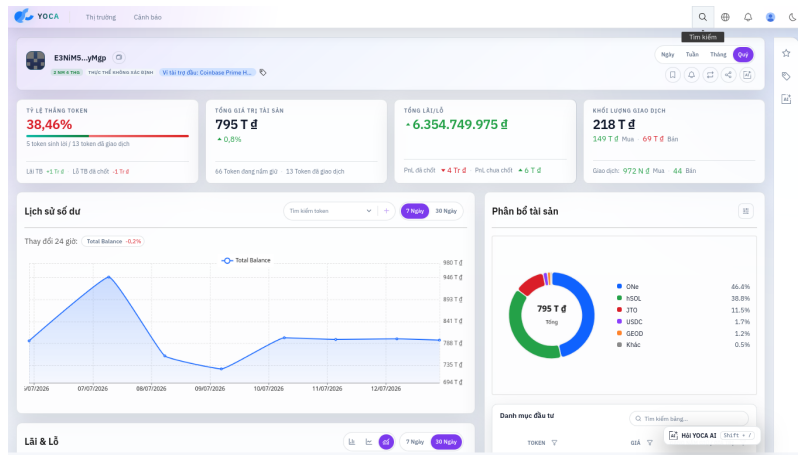
dữ liệu gốc từ backend không bị thay đổi.

Phần triển khai không chỉ gồm hai tệp bản dịch. Cấu trúc key của locale tiếng Việt được kiểm tra dựa trên locale gốc, biến nội suy được ràng buộc kiểu và formatter xử lý thống nhất giá trị rỗng, compact notation, phần trăm, token unit và thời gian tương đối. Cách làm này giúp nhóm sửa thuật ngữ hoặc quy tắc trình bày tại một nơi thay vì chỉnh từng bảng và biểu đồ.

Mức độ áp dụng hiện chưa tuyệt đối. Một số component cũ và một số luồng được phát triển độc lập vẫn còn chuỗi tiếng Anh hoặc nhãn USD viết trực tiếp. Vì vậy, nhóm chúng em xem nền tảng localization đã được hiện thực và sử dụng ở nhiều màn hình cốt lõi, còn việc loại bỏ toàn bộ hard-coded text là phần cần tiếp tục trong quá trình đồng nhất hóa giao diện.



(a) Giao diện tiếng Anh, đơn vị USD



(b) Giao diện tiếng Việt, đơn vị VND

Hình 5.14: Cùng trang Wallet Overview ở hai locale: chuỗi giao diện, định dạng ngày/thời lượng và giá trị tài chính được quy đổi và làm tròn theo VND

5.4 Giải quyết các thách thức kỹ thuật

5.4.1 Xử lý giới hạn truy vấn và điều phối khóa truy cập

Các nhà cung cấp dữ liệu đều áp dụng giới hạn về số lượng yêu cầu trong một khoảng thời gian nhất định. Trong khi đó, một thao tác như mở trang phân tích ví có thể đồng thời yêu cầu nhiều dữ liệu từ nhiều

endpoint và nhiều nhà cung cấp khác nhau, dễ dẫn đến lỗi HTTP 429 và làm gián đoạn quá trình tải dữ liệu.

Để giải quyết vấn đề này, Yoca xây dựng lớp điều phối `ThrottledQueue` riêng cho từng nhà cung cấp nhằm giới hạn số lượng yêu cầu đồng thời, thực hiện thử lại với khoảng chờ tăng dần khi gặp lỗi tạm thời và tuân theo thời gian chờ do nhà cung cấp trả về. Bên cạnh đó, cơ chế gộp yêu cầu (request coalescing) hợp nhất các yêu cầu giống nhau phát sinh đồng thời thành một lần gọi duy nhất, giúp giảm số lượng truy vấn không cần thiết.

Đối với các dịch vụ sử dụng nhiều khóa API, hệ thống luân phiên sử dụng các khóa để phân tán tải và hạn chế vượt quá giới hạn của từng khóa. Đồng thời, các loại lỗi như giới hạn truy vấn, lỗi mạng và lỗi dữ liệu được xử lý theo các chiến lược khác nhau nhằm duy trì khả năng cung cấp dữ liệu ngay cả khi một số nguồn gặp sự cố.

Các cơ chế trên tạo nền tảng để giảm truy vấn đồng thời và xử lý lỗi tạm thời có kiểm soát. Tuy nhiên, nhóm chưa có benchmark hoặc log tổng hợp đủ để định lượng mức giảm lỗi 429, vì vậy hiệu quả ở đây chỉ được đánh giá ở mức thiết kế và qua các test retry riêng lẻ.

5.4.2 Tái sử dụng bộ đệm theo khoảng thời gian

Dữ liệu lịch sử ví thường được truy vấn theo các khoảng thời gian chồng lấn. Nếu mỗi lần thay đổi khoảng thời gian đều tải lại toàn bộ dữ liệu, hệ thống sẽ phát sinh nhiều truy vấn lặp, làm tăng thời gian phản hồi và tiêu tốn tài nguyên.

Để khắc phục vấn đề này, Yoca áp dụng cơ chế *partial-range cache reuse*, trong đó dữ liệu giao dịch và phạm vi thời gian đã đồng bộ được lưu đồng thời trong bộ đệm. Khi nhận yêu cầu mới, hệ thống chỉ xác định và truy xuất phần dữ liệu còn thiếu, sau đó hợp nhất với dữ liệu hiện có dựa trên chữ ký giao dịch để loại bỏ các bản ghi trùng lặp.

Cơ chế này cũng được áp dụng cho dữ liệu biểu đồ token. Nhờ chỉ đồng

bộ phần dữ liệu còn thiếu, hệ thống giảm đáng kể số lần gọi đến nhà cung cấp, đồng thời cải thiện tốc độ phản hồi khi người dùng chuyển đổi giữa các khoảng thời gian.

Qua đó, dữ liệu được tái sử dụng hiệu quả hơn, góp phần nâng cao hiệu năng của hệ thống và giảm tải cho các dịch vụ bên ngoài.

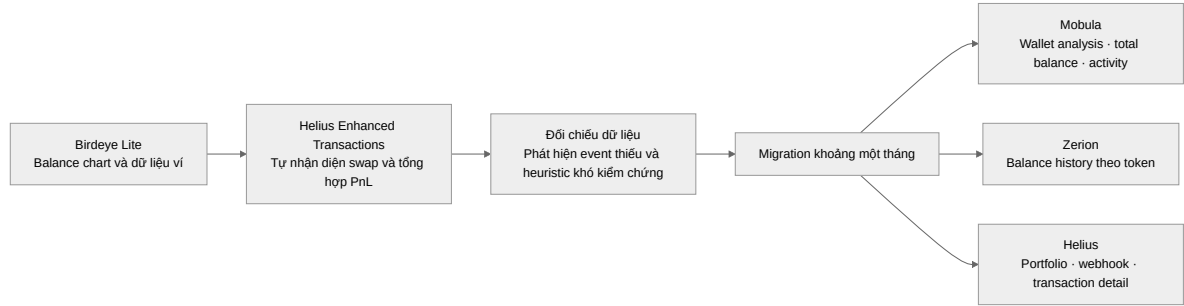
5.4.3 Từ tự tổng hợp giao dịch đến dịch vụ phân tích ví

Ở giai đoạn đầu, nhóm chúng em thử tự tổng hợp PnL từ Helius Enhanced Transactions. Dữ liệu đã được Helius giải mã giúp đọc token transfer, native transfer và instruction thuận tiện hơn transaction thô, nhưng lớp sự kiện dẫn xuất không phải lúc nào cũng chứa đủ thông tin mua, bán hoặc swap. Khi đối chiếu một số giao dịch với explorer và các nguồn dữ liệu khác, nhóm nhận thấy trường event có thể rộng hoặc không phản ánh đầy đủ các leg của giao dịch.

Để tiếp tục phát triển, nhóm xây dựng heuristic dựa trên thay đổi ròng của token balance, chiều chuyển tài sản, SOL vào/ra và program data. Cách này có thể nhận diện nhiều trường hợp mà event không mô tả, nhưng không phải một decoder tổng quát đã được kiểm chứng. Giao dịch đi qua nhiều program, tạo token account hoặc có nhiều tài sản cùng thay đổi dễ tạo ra trường hợp mơ hồ. Với ví hoạt động mạnh, số trang lịch sử và số token cần định giá cũng tăng khó dự đoán, làm thời gian phân tích kéo dài và khiến việc kiểm tra thủ công từng kết quả không còn khả thi.

Khó khăn lớn hơn nằm ở phạm vi dữ liệu. PnL yêu cầu lịch sử giao dịch đủ sâu, giá tại thời điểm giao dịch và cách xác định giá vốn nhất quán. Tần suất hoạt động của một địa chỉ không thể biết trước khi người dùng nhập ví, do đó nhóm không thể bảo đảm tự thu thập toàn bộ lịch sử cho mọi ví trong phạm vi thời gian và hạ tầng của đồ án. Heuristic Helius vẫn có giá trị cho transaction detail, cảnh báo và một số phân tích hành vi, nhưng không còn được xem là nguồn duy nhất cho các chỉ số tài chính

tổng hợp.



Hình 5.15: Quá trình điều chỉnh nguồn dữ liệu cho mô-đun Wallet

5.4.4 Migration Mobula–Zerion và phân tách ý nghĩa PnL

Khi gói Birdeye Lite không còn được hỗ trợ toàn phần, nhóm dành khoảng một tháng để chuyển các chức năng Wallet sang Mobula và Zerion. Việc chuyển đổi ảnh hưởng trực tiếp đến Overview, Balance Chart và PnL nên trong giai đoạn này các chức năng phân tích quan trọng bị gián đoạn. Nhóm không thay toàn bộ Birdeye bằng một API duy nhất, mà chọn lại nguồn theo loại dữ liệu như Hình 5.15.

Trong phiên bản hiện tại, các chỉ số PnL tổng hợp, realized/unrealized PnL, volume, số giao dịch và breakdown theo ngày được lấy từ wallet analysis của Mobula rồi cache theo địa chỉ và kỳ phân tích. Đường PnL theo ngày có ý nghĩa khác: hệ thống lấy biến động tổng tài sản từ lịch sử Mobula và trừ dòng tiền ròng vào/ra trong ngày. Công thức sử dụng là

$$PnL_d = (B_{d,end} - B_{d,start}) - NetFlow_d, \quad (5.1)$$

trong đó B là tổng giá trị ví và $NetFlow_d$ là giá trị tài sản nhận vào trừ tài sản chuyển ra trong ngày. Kết quả này phản ánh biến động danh mục sau khi loại ảnh hưởng của nạp/rút, không đồng nhất hoàn toàn với realized

PnL theo từng giao dịch.

Balance Chart cũng được phân tách theo độ phủ của provider. Trước đây Birdeye cung cấp cả tổng tài sản và lịch sử theo token. Sau migration, total balance ưu tiên Mobula, còn lịch sử từng token tiếp tục dùng Zerion. Nhóm từng thử dùng Zerion cho tổng tài sản nhưng phát hiện kết quả có thời điểm lệch lớn so với portfolio Helius do một số tài sản nhỏ hoặc token spam được tính với giá trị không phù hợp. Sau khi trao đổi với phía hỗ trợ mà dữ liệu chưa ổn định kịp tiến độ, nhóm chuyển total balance sang Mobula và chỉ giữ Zerion ở phạm vi đã đối chiếu phù hợp hơn.

5.4.5 Giá trị kỹ thuật sau khi thu hẹp phạm vi tự tính toán

Việc sử dụng dịch vụ tổng hợp từng khiến nhóm băn khoăn liệu phần phân tích còn đủ ý nghĩa nếu các chỉ số quan trọng đến từ provider. Sau quá trình thử nghiệm, nhóm nhận ra giá trị của đề tài không nhất thiết nằm ở việc tự xây một indexer giao dịch có độ phủ tương đương dịch vụ thương mại. Phần nhóm có thể kiểm soát tốt hơn là lựa chọn nguồn theo miền, kiểm tra response ở runtime, chuẩn hóa hợp đồng nội bộ, quản lý cache và coverage, phát hiện sai lệch qua đối chiếu, đồng thời giữ khả năng thay provider khi quota hoặc chất lượng dữ liệu thay đổi.

Giải pháp hiện tại vẫn có giới hạn. PnL tổng hợp phụ thuộc vào cách tính của Mobula; đường PnL theo balance phụ thuộc vào độ đầy đủ của transfer và giá trị USD; dữ liệu theo token phụ thuộc vào mapping fungible của Zerion. Báo cáo vì vậy xem các kết quả này là dữ liệu phân tích có nguồn và phạm vi xác định, không phải sổ cái kế toán tuyệt đối của ví.

5.4.6 Chuẩn hóa dữ liệu từ nhiều nguồn

Các nhà cung cấp dữ liệu thường sử dụng những định dạng và cấu trúc khác nhau để biểu diễn cùng một loại thông tin. Nếu xử lý trực tiếp dữ

liệu thô, các chức năng phía trên sẽ phải phụ thuộc vào từng nhà cung cấp và khó đảm bảo tính nhất quán.

Để khắc phục điều này, Yoca thực hiện chuẩn hóa dữ liệu trước khi lưu trữ và trước khi hiển thị. Các giao dịch được quy về các hành động chuẩn như swap, gửi và nhận; trong khi dữ liệu thị trường và tin tức cũng được chuyển về một mô hình thống nhất trước khi sử dụng.

Nhờ đó, toàn bộ các mô-đun trong hệ thống có thể xử lý dữ liệu theo cùng một cấu trúc, giảm sự phụ thuộc vào từng nhà cung cấp và tạo thuận lợi cho việc mở rộng hệ thống trong tương lai.

5.4.7 Duy trì tính giải thích của kết quả AI

Các mô hình AI khó khai thác hiệu quả dữ liệu blockchain ở dạng thô, đồng thời kết quả sinh ra cần có khả năng đối chiếu với dữ liệu nguồn để đảm bảo tính minh bạch.

Để giải quyết vấn đề này, trước khi gửi yêu cầu đến mô hình AI, Yoca xây dựng một bối cảnh có cấu trúc gồm các chỉ số tổng hợp, giao dịch tiêu biểu, khoảng thời gian phân tích và các bằng chứng liên quan. Ngữ cảnh này được định nghĩa bằng kiểu dữ liệu riêng cho từng miền, ví dụ `TokenAiContext` cho token và các DTO tương ứng cho ví, thay vì truyền thẳng response thô của provider vào prompt. Việc này giúp mô hình tập trung vào những thông tin quan trọng thay vì xử lý toàn bộ dữ liệu blockchain.

Vì lịch sử hội thoại và dữ liệu do các tool trả về đều là nội dung không hoàn toàn đáng tin cậy, hệ thống bọc các phần này trong dấu phân định rõ ràng kèm chỉ dẫn rằng mô hình chỉ tuân theo hướng dẫn hệ thống, không thực thi chỉ dẫn nằm trong phần dữ liệu đó. Đây là một ranh giới prompt-injection ở mức thực dụng, không phải cơ chế đã được kiểm chứng độc lập. Sau khi AI tạo kết quả, phần diễn giải luôn được hiển thị cùng với bảng dữ liệu hoặc biểu đồ tương ứng để người dùng dễ dàng đối chiếu. Đối với mô-đun wash trading, kết quả được liên kết trực tiếp với các đặc

trung và các ví đáng ngờ; trong khi ở các trang phân tích ví và token, AI chỉ diễn giải dữ liệu của đối tượng đang được quan sát.

Qua đó, hệ thống vừa tận dụng được khả năng diễn giải của AI, vừa đảm bảo người dùng luôn có thể kiểm chứng các nhận định dựa trên dữ liệu gốc, giúp tăng tính minh bạch và độ tin cậy của kết quả phân tích.

5.4.8 Giảm thiểu prompt injection và kết quả AI sai cấu trúc

Các luồng AI có một ranh giới tin cậy khác với API thông thường: câu hỏi hiện tại, lịch sử hội thoại và nội dung do search/provider trả về đều có thể chứa văn bản giống câu lệnh. Trong Wallet Chat, nhóm chúng em đặt quy tắc bất biến trong `systemInstruction`, còn user message, history và tool result được đóng gói trong các khối UNTRUSTED có marker kết thúc rõ ràng. Bước chọn tool chỉ chấp nhận tên nằm trong allowlist; dữ liệu đầu vào tiếp tục được chuẩn hóa và giới hạn trước khi gọi service.

Phản hồi của mô hình được yêu cầu ở dạng JSON và được kiểm tra lại trước khi đưa vào giao diện. Output không đúng hợp đồng bị từ chối thay vì hiển thị nguyên văn; citation không có nguồn tương ứng bị loại bỏ, còn nguồn hiển thị được lấy lại từ kết quả search thật thay vì tin vào danh sách do mô hình tự sinh. Token AI và Wash Trading Chat cũng dùng system instruction riêng, giới hạn phạm vi đối tượng và kiểm tra JSON đầu ra. Các biện pháp này giảm khả năng instruction ẩn trong user/history/tool data điều khiển luồng xử lý, nhưng không được xem là bảo đảm ngăn chặn tuyệt đối; vì vậy nhóm vẫn giới hạn tool, context, độ dài và dữ liệu bí mật được phép đi vào mô hình.

5.5 Kết quả kiểm thử và đánh giá hệ thống

Yoca sử dụng Vitest ở cả client và server. Client chạy trong môi trường jsdom và kết hợp Testing Library để kiểm tra component, thao tác người

dùng và trạng thái hiển thị. Server chạy trong môi trường Node; phần lớn test cô lập route hoặc service bằng mock cho database, provider, Stripe, Helius và AI. Cách tổ chức này có giá trị đối với regression ở logic nội bộ, nhưng không tương đương kiểm thử end-to-end với PostgreSQL và external API thật.

5.5.1 Phạm vi kiểm thử hiện có

Ở client, test bao phủ một số hành vi bảng như sort/search/filter, chart interaction, Wallet holding/topbar, Quick AI, subscription và các luồng thanh toán thẻ hoặc Solana. Ở server, phạm vi rộng hơn gồm auth và password reset, subscription/payment, alert và Helius webhook, AI usage, chat normalizer/guardrail, token AI/news, rate-limit retry, balance route và một số phép tính hoặc coverage của Wallet/PnL.

Các test sử dụng fixture và mock để tạo trường hợp thành công, dữ liệu rỗng, lỗi xác thực, lỗi provider và giới hạn truy vấn. Một số test đã kiểm tra status 400, 401, 429, 500 và kỳ vọng chuyển lỗi upstream thành 502. Zod schema và kiểu TypeScript hỗ trợ phát hiện sai lệch dữ liệu, nhưng được xem là lớp validation chứ không được tính thay cho test.

Một số kịch bản biên cụ thể đã có test riêng: webhook ví ghi nhận đúng số sự kiện trùng thay vì chặn toàn bộ khi Helius gửi lại cùng signature; phần hướng dẫn hệ thống của chat được kiểm tra để bảo đảm mô hình không làm theo chỉ dẫn chèn trong dữ liệu tool hoặc lịch sử hội thoại. Các kịch bản còn lại như ví có lịch sử rất dài, token thiếu giá lịch sử hoặc payment thất bại giữa chừng hiện được xử lý ở mức thiết kế (phân trang, giá trị rỗng có kiểm soát, xác minh giao dịch trước khi cấp quyền) nhưng chưa có test riêng; nhóm xem đây là phần cần bổ sung trước khi mở rộng phạm vi kiểm thử.

5.5.2 Kết quả chạy test và vấn đề còn tồn tại

Ngày 12/07/2026, nhóm chạy hai lệnh `npm test -w client --run` và `npm test -w server` trên source hiện tại. Kết quả được trình bày trong Bảng 5.1; đây là số liệu của đúng lần chạy này, không phải coverage hay tỷ lệ ổn định dài hạn.

Suite	Test file	Test case	Kết quả	
Client	16/16 đạt	178/178 đạt	Các test component, payment, profile, chart interaction, utility và Alert History API đều đạt trong lần chạy ngày 12/07/2026.	
Server	29/29 đạt	204/204 đạt	Các test auth/entitlement, payment, alert/webhook/history, AI guardrail và prompt injection, Wallet, provider mapping, pagination cùng route/service đều đạt trong lần chạy ngày 12/07/2026.	

Bảng 5.1: Kết quả chạy test trên source hiện tại

Kết quả cho thấy repository đã có nền tảng kiểm thử đáng kể và hai suite hiện đều ở trạng thái xanh. Trong đợt rà cuối, nhóm chúng em sửa các expectation đã lỗi thời sau thay đổi localization/theme, thống nhất contract lỗi upstream và bổ sung test cho Alert History, provider mapper cùng cursor/offset pagination. Kết quả này đủ để dùng test làm một công kiểm tra trong CI, nhưng chưa thay thế E2E trên môi trường triển khai hay đánh giá tải thực tế.

5.5.3 Đánh giá và phạm vi kiểm thử còn thiếu

Các test hiện tại chưa đo đầy đủ cache hit/miss trên môi trường có PostgreSQL thật, chưa gọi provider thật theo contract regression và chưa chạy toàn bộ user journey trong browser. Repository cũng chưa cung cấp load test, benchmark có thể lặp lại, staging ổn định hay số liệu về độ trễ

cảnh báo. Vì vậy, chương này không đưa ra con số cải thiện hiệu năng hoặc khẳng định độ ổn định toàn hệ thống.

Phạm vi hoàn thiện vừa đủ gồm bốn lớp: unit test cho normalizer và phép tính; provider contract test bằng response fixture đã ẩn dữ liệu nhạy cảm; route/service test cho fresh/stale/error và các status 200, 400, 429, 500, 502, invalid JSON, schema mismatch; cuối cùng là smoke test cho hành trình demo. Suite tự động hiện đã xanh; phần còn lại trước bản nộp là chạy smoke test trên database và môi trường deploy thật, đặc biệt với delivery và lịch sử cảnh báo.

5.6 Tổng kết chương

Chương 5 đã trình bày quá trình triển khai thực nghiệm hệ thống Yoca, bao gồm môi trường vận hành, tích hợp các nguồn dữ liệu, quy trình tích hợp và triển khai mã nguồn, cùng kết quả hiện thực các nhóm chức năng cốt lõi. Các chức năng được tổ chức thành luồng sử dụng thống nhất, từ khám phá thị trường, phân tích token và pool đến theo dõi ví, giao dịch, cảnh báo và hỗ trợ diễn giải bằng AI.

Chương cũng làm rõ những giải pháp được áp dụng để xử lý các thách thức của dữ liệu blockchain, gồm giới hạn truy vấn, dữ liệu lịch sử chồng lấn, định giá danh mục trong quá khứ và sự khác biệt giữa các nhà cung cấp. Trong đó, mô-đun phát hiện dấu hiệu wash trading thể hiện khả năng kết hợp phân tích đồ thị, chấm điểm rủi ro, trực quan hóa quan hệ giữa các ví và giải thích kết quả dựa trên dữ liệu nguồn.

Kết quả chạy test cho thấy các luồng auth, entitlement, payment, alert, AI, component và Wallet đã có regression test và hai suite đều đạt. Giới hạn còn lại nằm ở E2E, load test, benchmark và bằng chứng triển khai thật, thay vì các test tự động đang thất bại. Những kết quả và tồn tại này là cơ sở để Chương 6 tổng kết đóng góp, giới hạn và hướng phát triển tiếp theo cho Yoca.

Chương 6

Kết luận và hướng phát triển

Chương này tổng kết những gì Yoca đã đạt được sau quá trình phát triển, đồng thời nhìn nhận các giới hạn còn tồn tại. Thay vì lặp lại danh sách chức năng của Chương 5, nội dung tập trung vào giá trị của luồng phân tích, các quyết định kỹ thuật có khả năng tái sử dụng và những công việc cần ưu tiên nếu hệ thống tiếp tục được hoàn thiện.

6.1 Kết quả và đóng góp chính

Yoca đã hình thành một luồng phân tích liên tục từ Market Overview đến Token Pool, Token Overview, Wallet, User và các chức năng AI. Điểm quan trọng của luồng này không nằm ở số lượng trang, mà ở việc người dùng có thể đi từ biến động thị trường đến pool cụ thể, quan sát ví tham gia, kiểm tra giao dịch và quay lại dữ liệu nguồn mà không phải tự ghép nhiều công cụ rời rạc. Các chức năng tài khoản, watchlist, nhãn ví, subscription và cảnh báo bổ sung lớp cá nhân hóa cho quá trình sử dụng.

Đóng góp kỹ thuật đầu tiên là lớp dữ liệu nhiều provider được đặt sau một hợp đồng nội bộ. CoinGecko, Birdeye, Helius, Mobula, Zerion và Moralis được chọn theo miền thay vì tạo fallback tràn lan. Response

bên ngoài được kiểm tra bằng Zod trước khi đi vào nghiệp vụ; Hono RPC và TypeScript giữ hợp đồng giữa client với server; PostgreSQL và Drizzle quản lý dữ liệu nền cùng các read model cache. Cách tổ chức này không loại bỏ sự phụ thuộc provider, nhưng giới hạn nơi sự khác biệt về schema và semantics lan vào hệ thống.

Đóng góp thứ hai là cách xem cache như một phần của thiết kế dữ liệu. Snapshot có thể dùng TTL, còn lịch sử transfer, swap và transaction cần coverage metadata để xác định khoảng đã tải. Các bảng được tách theo nguồn và nhịp cập nhật nhằm tránh việc một hàng chỉ hoàn chỉnh sau nhiều lần gọi API. Quá trình migration Wallet cũng cho thấy khả năng thay thế tăng dần: Mobula hiện cung cấp các chỉ số phân tích và lịch sử tổng tài sản, Zerion giữ lịch sử theo token, còn Helius tiếp tục phục vụ portfolio, webhook và transaction detail.

Đối với phân tích ví, nhóm chúng em đã đi từ thử nghiệm tự giải mã Enhanced Transactions và heuristic balance change đến một phạm vi thực tế hơn. PnL tổng hợp được đọc từ dữ liệu Mobula đã kiểm tra và cache; đường PnL theo ngày được phân biệt rõ với realized PnL, dựa trên biến động tổng tài sản sau khi trừ dòng tiền ròng. Việc ghi rõ nguồn, kỳ phân tích và độ phủ giúp kết quả có thể được diễn giải đúng giới hạn thay vì được xem như sổ cái kế toán tuyệt đối.

Yoca còn kết hợp dữ liệu định lượng với lớp diễn giải AI. Dữ liệu token, ví và giao dịch được chuẩn bị thành ngữ cảnh có cấu trúc trước khi gửi đến mô hình. Kết quả được đặt cạnh bảng, biểu đồ hoặc giao dịch liên quan để người dùng có cơ sở đối chiếu. Mô-đun wash trading mở rộng cách quan sát từ từng bản ghi sang quan hệ đồ thị và các tín hiệu như chu trình, hubness, mức tương đồng và bất thường khối lượng; đây là tín hiệu hỗ trợ đánh giá, không phải kết luận gian lận tự động.

Về khả năng tiếp cận, lớp localization dùng chung hỗ trợ tiếng Anh, tiếng Việt, định dạng số/ngày giờ và quy đổi USD sang VND tại lớp trình bày. Cấu trúc translation key và biến nội suy được kiểm tra trong lúc phát triển. Đây là phần đóng góp gần trực tiếp với mục tiêu giúp người

dùng Việt Nam tiếp cận dữ liệu blockchain quen thuộc hơn, dù một số component cũ vẫn chưa được bản địa hóa hoàn toàn.

6.2 Giới hạn còn tồn tại

Giới hạn lớn nhất của hệ thống là chất lượng và tính liên tục của dữ liệu vẫn phụ thuộc vào provider. Quota, giá gói, phạm vi lịch sử, trường nullable và cách phân loại token có thể thay đổi. Kinh nghiệm chuyển từ Birdeye sang Mobula–Zerion cho thấy thay provider không chỉ sửa endpoint mà còn ảnh hưởng adapter, cache, schema và cách giải thích chỉ số. Một số read model, đặc biệt `wallet_analyses`, vẫn mang hình dạng gần với provider vì được xây dựng trong giai đoạn migration gấp.

Kết quả phân tích tài chính cũng có giới hạn về độ phủ. Token mới hoặc ít thanh khoản có thể thiếu giá lịch sử; transaction phức tạp có thể không được lớp Enhanced Transactions mô tả đầy đủ; PnL theo balance phụ thuộc vào giá trị tổng tài sản và transfer có giá USD. Nhân ví, AI và tín hiệu wash trading đều cần được xem cùng dữ liệu nguồn, không nên sử dụng như nhận định chắc chắn về chủ thể hay hành vi.

Giao diện hiện chưa có một design system được áp dụng trọn vẹn. Carbon, SCSS module, utility class và component tự xây vẫn cùng tồn tại. Landing, Market và Wallet đã được đổi mới nhiều hơn, trong khi một số component Token/Pool còn style và behavior cũ. Localization cũng còn chuỗi hard-code. Vấn đề này liên quan đến component contract, accessibility và trạng thái loading/error/empty, không chỉ là màu sắc.

Phạm vi kiểm thử và vận hành chưa đủ để khẳng định hệ thống ổn định ở quy mô lớn, dù lần chạy ngày 12/07/2026 đã đạt 178/178 test phía client và 204/204 test phía server. Hệ thống chưa có E2E đầy đủ, load test, benchmark cache trên PostgreSQL thật hoặc staging ổn định. GitHub Actions đã cấu hình các bước kiểm tra liên tục và job `deploy` đã chạy thật trên `main`, đưa cả Static Site lẫn Web Service lên trạng thái Live tại tên miền `yoca.id.vn`. Alert History đã được nối với delivery Helius,

có ownership API, read state và khóa chống trùng; phần còn lại là reset database demo, smoke test delivery và chụp bằng chứng giao diện.

6.3 Hướng phát triển

Ưu tiên đầu tiên là khép lại các contract đang không thống nhất. Nhóm cần quyết định rõ cách biểu diễn lỗi provider, sửa đường Zerion balance để phân biệt response rỗng, malformed JSON và upstream failure, sau đó đưa các fixture này vào contract regression test. Test suite phải trở về trạng thái xanh trước khi CI trở thành cổng merge. Bước tiếp theo là bổ sung smoke test cho user journey chính và E2E chọn lọc cho auth, Wallet, payment và alert; số liệu hiệu năng chỉ được đưa ra sau khi có benchmark lặp lại được.

Ở lớp tích hợp, hệ thống cần bổ sung observability cho thời gian phản hồi, tỷ lệ lỗi, retry, quota và cache hit/miss theo provider. Retry phải có giới hạn; circuit breaker và stale-data policy cần được định nghĩa cho những endpoint quan trọng. Adapter nội bộ và read model nên tiếp tục giảm coupling với response provider, đặc biệt ở Wallet, nhưng không nên refactor đồng loạt khi consumer cũ vẫn hoạt động.

Ở giao diện, hướng phát triển phù hợp là thống nhất design token, quy định component contract và trạng thái dữ liệu, sau đó migration theo màn hình ưu tiên thay vì thay toàn bộ thư viện một lần nữa. Cùng lúc đó, nhóm cần loại bỏ chuỗi hard-code, bổ sung kiểm tra parity locale và hoàn thiện bộ ảnh chức năng bằng dữ liệu demo ổn định.

Alert History cần tiếp tục được kiểm chứng trên môi trường triển khai bằng một luồng delivery thật và retry webhook, dù contract, persistence, ownership và read state đã được cài đặt và kiểm thử tự động. Pipeline GitHub Actions và Render đã chạy thật, đưa hệ thống lên `yoca.id.vn`; phần còn lại là smoke check các callback/webhook (Google, Stripe, Helius) trên đúng môi trường production.

Về dài hạn, mô-đun wash trading có thể được đánh giá trên tập dữ liệu

đã kiểm chứng và kết hợp thêm biến động giá, thanh khoản cùng hành vi qua nhiều pool. AI có thể tăng khả năng dẫn chứng bằng cách liên kết từng nhận định với giao dịch hoặc vùng biểu đồ cụ thể. Những mở rộng này chỉ có ý nghĩa khi nền tảng dữ liệu, kiểm thử và quan sát vận hành đã đủ tin cậy.

Nhìn chung, Yoca đã tạo được nền tảng cho một sản phẩm phân tích Solana có luồng sử dụng rõ ràng và có chủ đích đối với người dùng Việt Nam. Kết quả quan trọng nhất của đề tài là kinh nghiệm tổ chức dữ liệu nhiều nguồn, nhận diện giới hạn của việc tự giải mã, điều chỉnh kiến trúc khi provider thay đổi và giữ cho kết quả phân tích có thể đối chiếu. Các giới hạn còn lại được xem là điểm xuất phát cụ thể cho giai đoạn hoàn thiện tiếp theo, thay vì được che khuất bằng những khẳng định chưa có bằng chứng.

Tài liệu tham khảo

- [1] Apache ECharts. *Apache ECharts Documentation*. <https://echarts.apache.org/>. Accessed: 2026-06-19.
- [2] Arkham Intelligence. *Intel Platform / Arkham*. <https://intel.arkm.com/>.
- [3] Birdeye. *Token Tracker / Live Prices, Charts & Trades / Birdeye*. <https://birdeye.so/>.
- [4] Birdeye Data Services. *Data Accessibility by Packages*. <https://docs.birdeye.so/docs/data-accessibility-by-packages>. Truy cập ngày 13/07/2026.
- [5] Birdeye Data Services. *Pricing*. <https://docs.birdeye.so/docs/pricing>. Truy cập ngày 11/07/2026.
- [6] CoinGecko. *API Pricing Plans*. <https://www.coingecko.com/en/api/pricing>. Truy cập ngày 13/07/2026.
- [7] CoinGecko. *CoinGecko API Data Delivery Methods*. <https://docs.coingecko.com/docs/data-delivery-methods>. Truy cập ngày 11/07/2026.
- [8] CoinGecko. *Cryptocurrency Prices, Charts, and Crypto Market Cap / CoinGecko*. <https://www.coingecko.com/>.
- [9] Drizzle Team. *Drizzle ORM Documentation*. <https://orm.drizzle.team/>. Accessed: 2026-06-19.
- [10] Dune. *Make onchain data work for you / Dune*. <https://dune.com/>.

- [11] Google. *Gemini API Pricing*. <https://ai.google.dev/gemini-api/docs/pricing>. Truy cập ngày 13/07/2026.
- [12] Groq. *Pricing*. <https://groq.com/pricing>. Truy cập ngày 13/07/2026.
- [13] Helius. *Get Enhanced Transactions*. <https://www.helius.dev/docs/api-reference/enhanced-transactions/gettransactions>. Truy cập ngày 11/07/2026.
- [14] Helius. *Pricing*. <https://www.helius.dev/pricing>. Truy cập ngày 13/07/2026.
- [15] Hono. *Hono - Web framework built on Web Standards*. <https://hono.dev/>. Accessed: 2026-06-19.
- [16] Meta Open Source. *React Documentation*. <https://react.dev/>. Accessed: 2026-06-19.
- [17] Mobula. *Data APIs*. <https://docs.mobula.io/data-introduction>. Truy cập ngày 11/07/2026.
- [18] Mobula. *Pricing*. <https://docs.mobula.io/pricing>. Truy cập ngày 11/07/2026.
- [19] Moralis. *Blockchain and Onchain Data API Plans*. <https://moralis.com/pricing/>. Truy cập ngày 11/07/2026.
- [20] Nansen. *Nansen / Onchain Analytics for Crypto Investors & Teams*. <https://www.nansen.ai/>.
- [21] RunPod. *GPU Cloud Pricing*. <https://www.runpod.io/pricing>. Truy cập ngày 13/07/2026.
- [22] Together AI. *Pricing*. <https://www.together.ai/pricing>. Truy cập ngày 13/07/2026.
- [23] Vantage. *AWS EC2 g5.xlarge Instance Pricing*. <https://instances.vantage.sh/aws/ec2/g5.xlarge>. Truy cập ngày 13/07/2026.

- [24] Vite. *Vite Documentation*. <https://vite.dev/>. Accessed: 2026-06-19.
- [25] Zerion. *Zerion API for Wallet Data*. <https://zerion.io/api>. Truy cập ngày 11/07/2026.
- [26] Zod. *Zod Documentation*. <https://zod.dev/>. Truy cập ngày 11/07/2026.

Phụ lục A

Phân tích chi phí vận hành và mô hình kinh doanh

Phụ lục này trình bày cách nhóm đưa pricing của Yoca về sát với chi phí vận hành thật và chuẩn bị cho các mốc mở rộng quy mô tiếp theo, thay vì giữ một mức giá cố định từ đầu dự án. Pricing được thiết kế lại với lựa chọn tháng/năm, các gói chỉ ghi phần khác biệt so với gói liền trước, và toàn bộ chi phí/lợi nhuận giả định được suy ra trực tiếp từ source code (cache, TTL, giới hạn truy vấn từng provider) cùng giá/hạn mức công khai tại thời điểm khảo sát 13/07/2026, để nhóm có căn cứ điều chỉnh giá khi lưu lượng thật tăng lên. Phần chưa có số đo thật (lưu lượng truy cập, tỷ lệ trả phí) được ghi rõ là giả định minh họa.

A.1 Bản đồ chi phí dữ liệu hiện tại

Mọi yêu cầu đi qua backend, được kiểm tra so với dữ liệu đã lưu trong PostgreSQL trước khi gọi provider. Cache theo mô hình cache-aside, khóa theo TTL cấu hình trong `constants.ts`. Cache được khóa theo địa chỉ

token/ví, không theo người dùng: nhiều người cùng xem một token phổ biến trong TTL chỉ tốn một lần gọi provider; dữ liệu ví thì gần với cache một-đối-một hơn, vì mỗi người dùng thường chỉ theo dõi một tập nhỏ ví họ quan tâm.

Rà lại luồng gọi provider theo từng trang cho thấy tải nặng nhất thật sự đến từ CoinGecko (metadata, market data, chart, tìm kiếm) và Mobula (lịch sử swap/transfer, PnL, token nắm giữ trong ví); Birdeye chỉ còn xuất hiện ở vài điểm hẹp (widget thị trường dùng cache chung, tab gainers, chart tổng tài sản ví, fallback giá swap) — thấp hơn nhiều so với giả định ban đầu dựa theo lịch sử migration Birdeye–Mobula–Zerion ở Chương 5.

A.2 Điều chỉnh TTL và đôn bẫy phía nhà cung cấp

Phần lớn TTL hiện tại đã hợp lý, không cần đại tu. Một khoảng trống chi phí được phát hiện và đã vá: mô-đun wash trading gọi Gemini sinh kết luận nhưng không có TTL, khác mọi tính năng AI còn lại (đều cache 3–24 giờ); đã bổ sung cache theo cặp token và khung thời gian phân tích.

Mỗi provider đều hỗ trợ xoay nhiều khóa API miễn phí để tăng giới hạn truy vấn hiệu dụng, trừ Zerion (một khóa) và Gemini (tính theo token, không theo khóa). Lịch sử hoạt động ví qua Mobula dùng phân trang offset, hiện chạy tuần tự do giới hạn 1 request/giây của gói miễn phí; nâng lên gói trả phí thấp nhất cho phép tải song song, rút thời gian tải một ví từ khoảng mười giây xuống dưới một giây.

Bảng A.1 tóm tắt giới hạn miễn phí và gói trả phí gần nhất, theo nguyên tắc chỉ nâng gói để tăng giới hạn trên đúng endpoint đang dùng, không đổi endpoint để tránh chi phí migrate.

Bảng A.1: Giới hạn miễn phí và gói trả phí gần nhất theo provider

Provider	Giới hạn miễn phí (đo trong code)	Gói trả phí gần nhất
Helius	2 request/giây	Developer, 49 USD/tháng, 10 request/giây [14]
Birdeye	1 request/giây (đã sửa cấu hình limiter cho đúng)	Không cần nâng gói ở các mốc đã khảo sát [5, 4]
Moralis	5 request đồng thời, 1,2 triệu compute unit/tháng	Starter, 49 USD/tháng, 2 triệu compute unit/tháng [19]
Zerion	1 request/giây, một khóa duy nhất	Builder, 149 USD/tháng, 50 request/giây [25]
Mobula	1 request/giây, một khóa duy nhất	Start-up, 50 USD/tháng, 30 request/giây [18]
CoinGecko	1 request/1,2 giây	Basic, 35 USD/tháng, 300 request/phút [6]
Google Gemini	Trả theo token, không có bậc miễn phí/trả phí cố định	Xem Bảng A.3 [11]

A.3 Kịch bản chi phí theo quy mô người dùng

Ba mốc MAU giả định (300 / 3.000 / 30.000, cách nhau mười lần) minh họa công thức suy luận chi phí: $\text{request/tháng} = \text{MAU} \times \text{phiên/tháng} \times \text{tỷ lệ vào trang} \times \text{tỷ lệ cache-miss} \times \text{số lệnh gọi provider/lượt xem}$. Các hệ số đầu vào là giả định, chưa có analytics thật từ hệ thống.

Bảng A.2: Chi phí dữ liệu và AI ước tính theo mức quy mô người dùng

Mức MAU	Chi phí dữ liệu ngoài	Chi phí Gemini AI	Tổng/tháng
300	0 USD (giới hạn miễn phí đủ dùng)	~3 USD	~3 USD
3.000	~85 USD (Mobula 50 + CoinGecko 35)	~27 USD	~112 USD
30.000	~228 USD (Helius 49 + Mobula 50 + CoinGecko 129)	~270 USD	~498 USD

Bảng chỉ tính chi phí dữ liệu và AI; chưa gồm hạ tầng máy chủ, cơ sở dữ liệu, nhân sự và marketing.

A.4 Giả thuyết chuyển sang mô hình AI tự vận hành

Đây là giả thuyết cho định hướng dài hạn, không phải cam kết migrate.

Bảng A.3: Giá Gemini API theo token (USD/1 triệu token) [11]

Model	Input	Output	Cached input
gemini-2.5-flash	0,30	2,50	0,03 + phí lưu trữ
gemini-2.5-flash-lite	0,10	0,40	0,01 + phí lưu trữ
gemini-3.1-flash-lite	0,25	1,50	0,025 + phí lưu trữ

Ba ứng viên mã nguồn mở phù hợp quy mô hiện tại: Qwen3-8B/Qwen3.5-9B (Apache 2.0, năng lực tương đương Gemini Flash-Lite), DeepSeek-R1-Distill Qwen 14B/32B (MIT, hợp cho audit cần suy luận nhiều bước), Llama-3.3-70B-Instruct (chất lượng cao hơn nhưng cần hạ tầng GPU lớn). Ở mức 50.000–200.000 lượt gọi/tháng, dịch vụ suy luận

trả theo token trên hạ tầng dùng chung (Together AI, Groq) tốn 4–52 USD/tháng, cùng bậc với Gemini Flash-Lite; thuê GPU riêng (RunPod L4 24GB) tốn khoảng 285 USD/tháng cố định, chỉ hợp lý ở lưu lượng lớn hơn nhiều [22, 12, 21, 23].

Điểm hòa vốn ước tính giữa thuê GPU riêng và trả theo token rơi vào khoảng 1,3–4,5 triệu lượt gọi/tháng, cao hơn 15–50 lần so với mốc 30.000 MAU. Ở quy mô hiện tại, dùng Gemini theo token vẫn kinh tế hơn; tự vận hành AI mã nguồn mở chỉ hợp lý khi lưu lượng tăng thêm một đến hai bậc, hoặc khi ưu tiên kiểm soát dữ liệu hơn tối ưu chi phí.

A.5 Mô hình giá và dòng tiền giả định

A.5.1 Bảng giá theo gói, chỉ ghi phần khác biệt

Bảng giá đã được áp dụng trực tiếp vào trang giá của sản phẩm, chỉ ghi phần thay đổi so với gói liền trước.

Bảng A.4: Bảng giá theo gói, chỉ ghi phần khác biệt so với gói liền trước

	Free	Lite – thêm so với Free	Plus – thêm so với Lite	Pro – thêm so với Plus
Giá/tháng	0	39 USD	79 USD	149 USD
Giá/năm	0	390 USD/năm	790 USD/năm	1.490 USD/năm
Lượt hỏi AI/ngày	5 / 5 / 10 / 5	20 / 20 / 25 / 20	50 / 50 / 50 / 50	100 / 100 / 100 / 100
AI ví và wash trading	Khóa	Khóa (không đổi)	Mở khóa, 50/ngày	100/ngày

Giá năm = $10 \times$ giá tháng (tương đương hai tháng miễn phí), chọn qua nút chuyển đổi trên trang giá, gắn với Stripe Price đúng chu kỳ. Giá Plus/Pro được điều chỉnh từ 199/499 USD (cũ, ngoài dải giá thị trường) xuống 79/149 USD, đối chiếu theo mặt bằng các sản phẩm phân tích ví/on-chain cá nhân cùng phân khúc (Nansen Pro, Dune Pro: 30–100 USD/tháng) [20, 10].

A.5.2 Dòng tiền giả định theo quy mô

Giả định 8% người dùng trả phí (5%/2%/1% cho Lite/Plus/Pro), tỷ giá tham khảo 26.000 VND/USD.

Bảng A.5: Doanh thu và lợi nhuận gộp giả định theo mức quy mô (chỉ tính chi phí dữ liệu/AI)

Mức MAU	Doanh thu/tháng	Chi phí dữ liệu + AI	Lợi nhuận gộp
300	~1.496 USD	~3 USD	~1.493 USD
3.000	~14.960 USD	~112 USD	~14.848 USD
30.000	~149.596 USD	~498 USD	~149.098 USD

Đây là lợi nhuận gộp chỉ tính chi phí dữ liệu/AI, chưa trừ hạ tầng, nhân sự, marketing; lợi nhuận ròng thực tế thấp hơn đáng kể.

A.5.3 Nguồn vốn

Yoca hiện vận hành hoàn toàn bằng nguồn lực tự có của nhóm, không có tài trợ hay gọi vốn nào; hạ tầng và API đang chạy trên gói miễn phí hoặc chi phí cá nhân của thành viên.

A.6 Phạm vi và giới hạn của mô hình

Số liệu trong phụ lục chứng minh phương pháp suy luận chi phí có căn cứ, không phải dự báo tài chính. Lưu lượng/số người dùng và tỷ lệ trả phí (8%) đều là giả định minh họa, chưa có số đo thật; lợi nhuận ở Bảng A.5 chỉ giới hạn ở chi phí dữ liệu/AI. Khi có số liệu thật, các bảng có thể cập nhật bằng cách thay giá trị đầu vào, không cần đổi phương pháp.